面向面试能力构建的机器学习系统知识综述：从并行计算、算子与编译到大模型训练与推理

ShaneLiu04

***Abstract*—机器学习系统（ML systems）方向的面试，不论是训练 infra、推理引擎、算子与编译器还是 AI 芯片岗，考的几乎是同一种能力：给定模型、硬件和负载，能不能在几分钟内用第一性原理估出时间、显存和通信的下界，指出瓶颈在哪里，并说清楚换一种设计会改变什么。本文把 Stanford CS149（并行计算）、Stanford CS229S（机器学习系统）和 CMU 15-849（机器学习系统研讨课）三门课的内容，围绕「算得过来吗、放得下吗、传得过来吗」这三个问题重新组织成 18 个模块。内容从性能模型、并行硬件与 GPU 编程模型讲起，经过 GEMM 与 FlashAttention 等算子、专用加速器、ML 编译器与框架，一直到分布式训练（数据、张量、流水线、序列并行与 ZeRO）、LLM 推理与服务（KV cache、PagedAttention、chunked prefill、PD 分离、推测解码、量化）、模型效率（LoRA、MoE、状态空间模型、scaling law）以及集群调度与可靠性。和一般的综述不同，本文的每个结论都配有能复算的白板估算。数字只有三个来源：正文里的推导，配套计算器 `tools/mlsys_calc.py` 的输出，以及原论文和本文配套的 9 个 CPU 实验的原始 CSV；文中始终区分「模型输出」和「GPU 实测」。全文有 57 个「面试追问」框（每个都附常见错答）、16 个白板估算、15 幅图和 19 张表。此外，本文给出一个把工程能力和研究能力分开刻画的能力矩阵、一个「结论、估算、瓶颈、权衡、证据」五步答题模板，以及把课程实验变成面试故事和研究练习的方法。文中也纠正了几个流传很广的误解：FlashAttention 的 HBM 流量在序列长度上仍是 $\Theta(T^2d^2/M)$，并没有降到线性；1F1B 并不减少流水线气泡；ZeRO-3 的通信量是数据并行的 1.5 倍，而不是 3 倍。**

***Index Terms*—机器学习系统，roofline，FlashAttention，分布式训练，ZeRO，张量并行，流水线并行，LLM 推理，KV cache，PagedAttention，推测解码，量化，ML 编译器，面试，能力建设**

I. 引言

去五年，深度学习系统要解决的问题变了。以前的目标过是「把模型跑起来」，现在要同时摆平一个万卡集群、一个千亿参数的模型和一组严格的服务等级目标（SLO，比如首 token 时延和每 token 时延的上限）。规模上的变化很直观：GPT-3 一类模型的训练算力达到 10^{23} FLOP 量级 [1]，Llama 3 的最大模型在约 1.6 万张 H100 上训练了 15.6T token [2]。推理这边，光是 KV cache 的显存管理和调度，就已经成了一个独立的系统研究方向 [3]–[6]。

ML 系统岗位的面试也随之形成了一种相当稳定的风格。面试官很少问「某篇论文第几节写了什么」，更常问的是「这个数字从哪来、瓶颈在哪、换一种做法会怎样」。这种问法有清楚的工程理由：大模型系统的设计空间很大，但绝大多数候选方案在白板上算几行就能排除。举三个例子。用混合精度 Adam 训练一个 7B 模型，光静态显存就至少要 $16N = 112$ GB，一张 80 GB 的 GPU 放不下。batch 为 1 的一次 decode 要把 16 GB 权重从 HBM 读一遍，在带宽 3.35 TB/s 的 H100 上，这一步的下界约为 5 ms。8 卡之间用 ring all-reduce 同步 14 GB 梯度，即使走 450 GB/s 的 NVLink，也要约 54 ms。会做这类估算的工程师，在打开 profiler 之前就知道该优化哪里，也能判断一篇论文报告的加速比是否可信。

作者 GitHub：https://github.com/ShaneLiu04。本文全部代码、白板计算器与原始实验数据见 https://gitee.com/liu-xingyan04/mlsys-infra。本文基于 Stanford CS149（Fall 2024）、Stanford CS229S（Fall 2023）与 CMU 15-849（Spring 2022）的公开课程材料整理，作者与上述大学无任何隶属关系，内容不代表课程方观点。

A. 三个问题的框架

本文把 ML 系统里几乎所有的问题归结为三个问题（图 1）：

1) 算得过来吗（compute）：要算多少 FLOPs，算术强度多高，在 roofline 上处在哪个位置。这一层不会算，「为什么 decode 受访存限制」之类的回答就只能靠背。
2) 放得下吗（memory）：权重、梯度、优化器状态、激活和 KV cache 各占多少字节，放不下时怎么切分、重算或卸载。
3) 传得过来吗（communication）：all-reduce 要传多少数据，流水线有多少气泡，PCIe、NVLink、InfiniBand 的带宽和拓扑够不够。

写成公式，一个训练步或推理步的时间下界和显存约束是

$$T_{\text{step}} \gtrsim \max\left(\frac{F}{P_{\text{peak}}},\ \frac{B_{\text{HBM}}}{\text{BW}_{\text{HBM}}},\ \frac{B_{\text{net}}}{\text{BW}_{\text{net}}}\right), \qquad M_{\text{static}} + M_{\text{dyn}} \le M_{\text{HBM}}, \tag{1}$$

其中 F 是浮点运算量，B_{HBM} 和 B_{net} 分别是在 HBM 和网络上搬运的字节数。式 (1) 取最大值，是假设计算、访存和通信能完全重叠；如果不能重叠，就应该把三项相加。后面各节做的事，几乎都是把某种具体的系统设计代入这个式子的某一项。

B. 三门课程如何互补

三门课各管一层（表 I）。CS149 [7] 讲机器怎样并行：多核、SIMD、GPU、cache coherence、内存一致性和专用硬件，它给「算得过来」打地基。CS229S [8] 把这些原理用到 Transformer 上，讲 hardware-aware 的算法设计、FlashAttention、KV cache、推测解码、量化和新架构。CMU 15-849 [9] 是论文研讨课，覆盖框架、编译器、数据/模型/流水线并行、ZeRO、重计算和压缩，主要回答「放得下」和「传得过来」。

本文没有按课程分章，而是按系统栈自下而上排成 18 个模块。课程结束之后又出现了一批系统，到 2025 年以后已经是推理岗默认要掌握的内容，本文也把它们补了进来，例如 vLLM [3]、SGLang [10]、FlashAttention-3 [11]、chunked prefill [5] 和 PD 分离 [6], [12]。

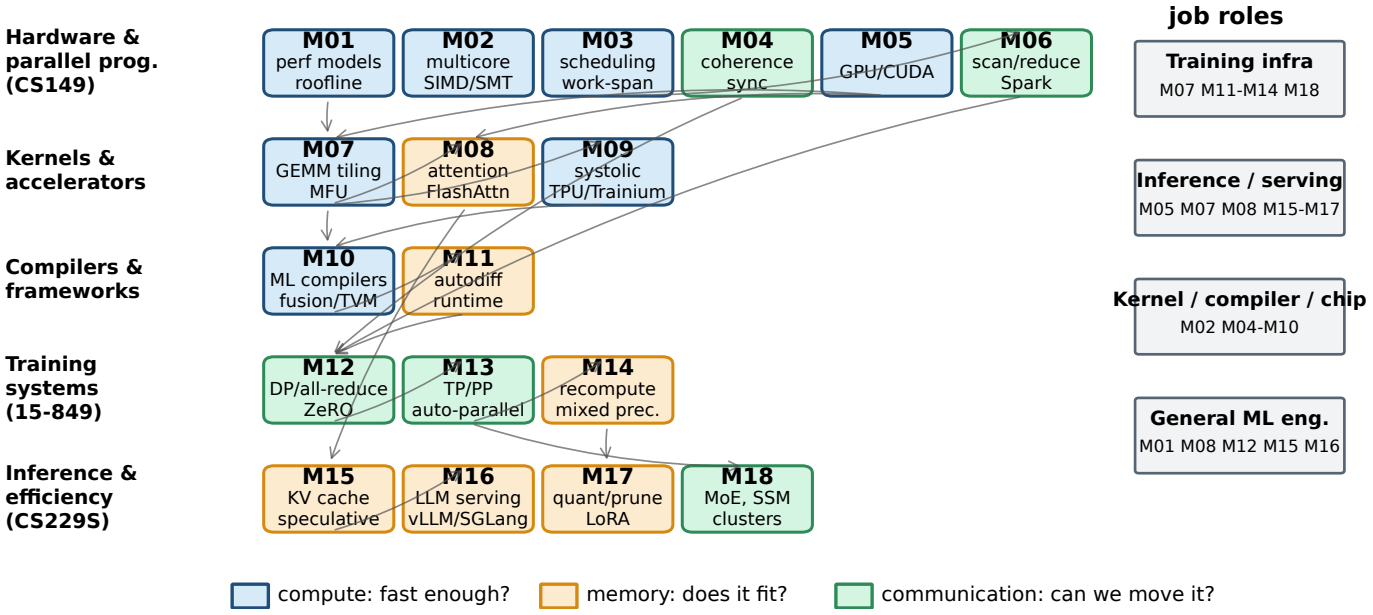


Fig. 1. 18 个模块的知识地图。纵向按系统栈自下而上排列：硬件与并行编程、算子与加速器、编译器与框架、训练系统，最上层是推理与效率。颜色表示该模块主要回答哪个问题：蓝色是「算得过来」，橙色是「放得下」，绿色是「传得过来」。箭头表示先修依赖，右侧列出四类岗位各自的主线模块。准备面试时，建议沿着箭头自下而上补齐，不必按课程顺序线性地读。

TABLE I
18 个模块与三门课程、三个问题以及配套实验的对应关系（「问题」一列：算、放、传分别指算得过来、放得下、传得过来）

#	模块	CS149 Fall 2024	CS229S Fall 2023	CMU 15-849 S22	问题	本文配套实验
01	性能模型与 roofline	Why Parallelism	Hardware-aware algorithm design	Intro to DL systems	算	—
02	多核、SIMD 与多线程	Multi-core, ISPC	—	—	算	CS149 A1
03	并行编程与调度	Work distribution, scheduling	—	—	算	CS149 A2
04	局部性、一致性与同步	Coherence, consistency, locks	—	—	传	CS149 A2
05	GPU 架构与 CUDA	GPU architecture, CUDA	GPU 存储层次	—	算	CS149 A3
06	数据并行原语	Data-parallel thinking, Spark	—	Parameter server	传	CS149 A3
07	矩阵乘与分块	Efficiently evaluating DNNs	Hardware-aware design	ML hardware	算	CS149 A4
08	Attention 与 FlashAttention	DNN 融合	FlashAttention	Transformers	放	CS229S P1
09	专用加速器	Hardware specialization	—	TPU	算	CS149 A4
10	ML 编译器	Domain-specific prog. systems	TVM（客座）	ML compilation, TASO	算	15-849 编译
11	框架与自动微分	—	FLOPs 与反传	Systems for DL	放	15-849 编译
12	数据并行与 ZeRO	—	Parallelism fundamentals	Data parallel, ZeRO	传	15-849 分布式
13	张量/流水线并行	—	Narayanan（客座）	Pipeline, Megatron, GSPMD	传	15-849 分布式
14	显存优化	—	Dettmers（客座）	Rematerialization, Offload	放	15-849 分布式
15	KV cache 与推测解码	—	Analyzing Transformers	—	放	CS229S P2
16	LLM 推理服务	—	Scheduling ML clusters	推理库（客座）	放	15-849 服务
17	压缩、量化与 PEFT	—	Memory-efficient NN	Model compression	放	15-849 编译
18	MoE、SSM、集群与数据	—	Attention-free, MoE	GNN, ML for systems	传	—

C. 本文的定位与贡献

现有的综述大多沿着「技术谱系」来写，比如专讲分布式训练，或者专讲推理优化。本文的读者不一样：他们在准备 ML infra 面试，或者想从工程走向研究。所以本文给每个技术点定的验收标准是两条，一是能不能当场估算，二是能不能说出常见的错答。围绕这个目标，本文做了四件

事：

1) 用一个统一的三问题框架组织内容。18 个模块都落到式 (1) 的三项里，读者遇到陌生问题时，可以把它归约成熟悉的估算。
2) 让数字可以复算。正文中的每个数字，要么在文中推导，要么由配套计算器 `tools/mlsys_calc.py` 算出（图表由 `reports/survey/make_figures.py` 直接调用这个计算器生成），要么引自论文或配套实验的 CSV。
3) 用配套实验支撑结论，并如实交代口径。本文复用了仓库中 9 个 CPU 实验的结果，包括 FlashAttention IO 模型、KV cache 与推测解码、分布式训练模拟、PagedAttention 服务模拟、编译/自动微分/压缩等。每条结果都标明它是正确性验证、模型输出还是 CPU 实测；CPU 实测的绝对数字不能外推到 GPU。
4) 给出面试能力框架。第 XI 节提供区分工程能力和研究能力的能力矩阵、答题模板、常见错答清单、把实验讲成 STAR 故事的方法，以及研究能力的练习。

面试追问 1：面试官为什么总让你「估一下」，而不是直接问概念？

答：因为概念可以背，估算背不了。一道估算题同时检验三件事：你知不知道硬件的数量级（H100 约 1 PFLOP/s、3.35 TB/s）；会不会把系统行为写成公式（比如 decode 一步至少要把权重读一遍）；能不能从数字推出设计结论（比如 batch 不大时，量化比换卡更有效）。合格的回答在 30 秒内先报出量级，再交代假设和误差来源。

常见错答：直接报一个经验数字（「大概 100 ms 吧」）却不给推导；或者反过来，花 5 分钟推导，迟迟不先说结论。

D. 论文组织

第 II 节建立性能模型，第 III 节讨论并行硬件和 GPU 编程模型，第 IV 节讨论 GEMM、FlashAttention 与算子融合，第 V 节讨论专用加速器，第 VI 节讨论编译器与框架。接下来是大模型的部分：第 VII 节讲分布式训练，第 VIII 节讲 LLM 推理与服务，第 IX 节讲模型效率与架构，第 X 节讲集群、可靠性与数据。第 XI 节给出面试能力框架，第 XII 节讨论开放问题，附录收录公式速查、硬件参数和两周学习计划。每节末尾有两个框：「面试要点」用于考前复习，「研究视角」把这一节的知识变成可以动手做的实验。

II. 性能模型：先算上界，再谈优化

同样一个 kernel，为什么换一个 batch 就快了十倍？为什么换了一张算力翻三倍的新卡，它却几乎没变快？要回答这类问题，先得有一个能算上界的模型。性能模型的用处不在于精确预测运行时间，而在于给出一个上界：实测结果如果「超过」了上界，说明测量有 bug；离上界很远，说明还有优化空间。本节介绍面试中最常用的五个模型：roofline、Amdahl 与 Gustafson 定律、work–span 模型、Little 定律，以及 I/O 复杂度下界。

A. *Roofline* 与算术强度

Williams 等人提出的 roofline 模型 [13] 把一个 kernel 简化成两个数：浮点运算量 F，以及它在某一级存储（通常是 DRAM 或 HBM）上搬运的字节数 B。两者之比 $I = F/B$

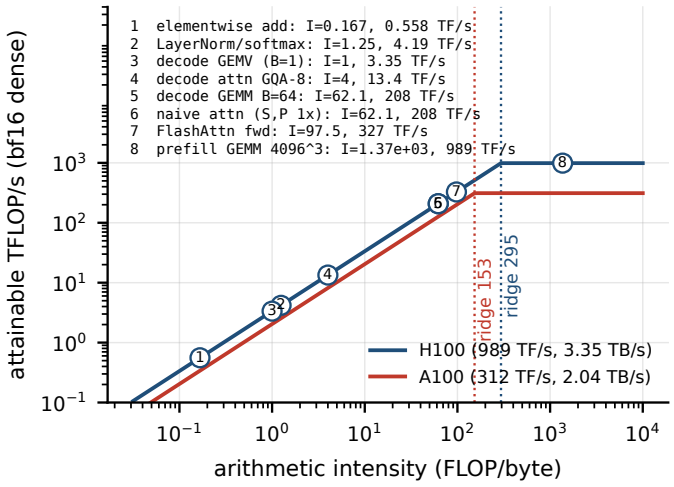


Fig. 2. A100 与 H100 的 roofline（BF16 稠密，以 HBM 为存储层级），以及 Transformer 典型算子在 H100 上的位置。强度由 `mlsys_calc.gemm_intensity` 和配套的 FlashAttention IO 模型（$T = 4096, d = 128$，SRAM 100 KiB）算出。只有大 GEMM 越过了拐点；和 decode 有关的算子全部落在斜线段上，离拐点差一到两个数量级，所以它们的速度由带宽决定。

叫算术强度（arithmetic intensity），意思是每从这一级存储读写一个字节，能做多少次浮点运算，单位是 FLOP/byte。假设计算和访存可以重叠，就有

$$t \geq \max\left(\frac{F}{P_{\text{peak}}},\ \frac{B}{\text{BW}}\right), \qquad P_{\text{attain}} = \min(P_{\text{peak}},\ I \cdot \text{BW}). \tag{2}$$

两项相等的那一点 $I^* = P_{\text{peak}}/\text{BW}$ 叫拐点（ridge point）。强度低于拐点的 kernel 受访存限制（memory-bound），再多的算力也用不上；高于拐点的受算力限制（compute-bound）。

代入本文统一使用的参数（附录表 XVII）：H100 SXM 的稠密 BF16 峰值是 989 TFLOP/s，HBM3 带宽 3.35 TB/s，拐点 $I^* \approx 295$ FLOP/B；A100 SXM 是 312 TFLOP/s 和 2.04 TB/s，拐点 $I^* \approx 153$ FLOP/B [14], [15]。从 A100 到 H100，算力涨了 3.2 倍，带宽只涨了 1.6 倍，所以拐点右移了将近一倍。这带来一个反直觉的后果：同一个 kernel 换到新硬件上，反而更容易变成访存受限。这就是「内存墙」 [16] 在单张卡上的表现。

那么 Transformer 里的算子都落在哪里？图 2 把几类典型算子画到了 H100 的 roofline 上。最左边是逐元素加法：bf16 下读两个元素、写一个元素，共 6 字节，只做 1 次 FLOP，强度 0.17，最多只能用到峰值的 0.06%。LayerNorm 和 softmax 的强度在 1 的量级；batch=1 的 decode GEMV 约为 1；GQA-8 的 decode attention 约为 4（推导见第 VIII 节）；batch=64 的 decode GEMM 约为 62。只有 4096^3 的 prefill GEMM 达到 1365，远在拐点右侧。图中还有一个值得留意的细节：FlashAttention 前向（点 7）和「S、P 各读写一次」的朴素实现（点 6）强度相差不大。这和很多人的印象不符，第 IV 节会用配套实验解释原因。

用 roofline 时有三个常见的误用，面试时可以主动指出。第一，B 必须对应某一级具体的存储。对 L2 或 shared memory 也可以画 roofline，只是拐点不同，这叫层次化 roofline。第二，P_{peak} 必须对应实际用到的计算单元。A100 不走 tensor core 的 FP32 只有 19.5 TFLOP/s，比 BF16

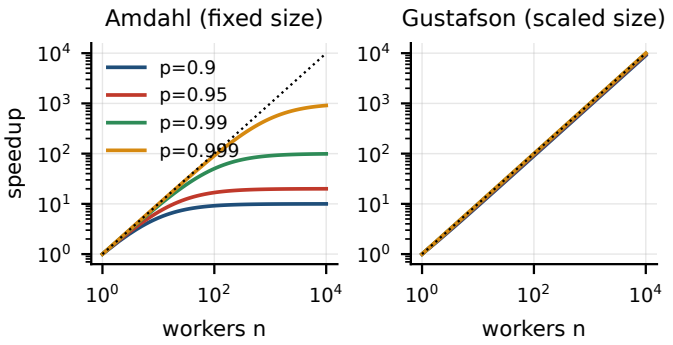


Fig. 3. Amdahl（固定规模）与 Gustafson（规模随处理器数增长）的加速比，虚线为线性加速。即使 $p = 0.99$，1024 个处理器的 Amdahl 加速比也只有 91 倍（`mlsys_calc.amdahl_speedup`）：剩下 1% 的串行部分决定了上限。

tensor core 低 16 倍；FlashAttention-2 想办法减少非矩阵乘的 FLOPs，动机正在于此 [17]。第三，「GPU 利用率 100%」不等于算力打满，因为 `nvidia-smi` 的 utilization 只表示采样窗口内有 kernel 在运行。

面试追问 2：为什么「GPU 利用率 100%」不代表算力打满？应该看什么？

答：因为 `nvidia-smi` 报的只是「有 kernel 驻留的时间比例」，一个访存受限的逐元素 kernel 同样能显示 100%。真正该看的是 Nsight Compute 里 SM 吞吐和 DRAM 吞吐各占峰值的比例，或者直接算 MFU（模型 FLOPs 利用率，见第 IV 节）。如果 DRAM 吞吐接近峰值而 SM 吞吐很低，就说明受访存限制，下一步应该做融合或降低精度，调 block size 没有用。

常见错答：「100% 说明已经最优」；或者把 HFU（把重计算的 FLOPs 也算进去）当成 MFU 来报。

B. Amdahl 与 Gustafson 定律

加卡之后能快多少？Amdahl 定律给出的答案很悲观。设程序中可并行部分的占比为 p，用 n 个处理器时，固定问题规模下的加速比是 [18]

$$S_{\mathrm{A}}(n) = \frac{1}{(1-p) + p/n} \xrightarrow{n\to\infty} \frac{1}{1-p}. \tag{3}$$

Gustafson [19] 换了一个角度：实际中问题规模往往随资源一起增长。如果并行部分的工作量也放大 n 倍，加速比就变成 $S_{\mathrm{G}}(n) = (1-p) + pn$。

图 3 对比了两种视角。$p = 0.95$ 时，按 Amdahl 定律 8 核只有 5.9 倍，无穷多核也只有 20 倍；按 Gustafson 定律，加速比却随 n 近似线性增长。两者并不矛盾，只是回答的问题不同：Amdahl 问「同一个任务能快多少」，Gustafson 问「同样的时间里能做多大的任务」。大模型训练中通过增大 global batch 来做弱扩展，用的就是 Gustafson 的视角；但它受临界 batch size 的限制，batch 大到一定程度后，收敛所需的步数不再减少，所以也不能无限扩展。

面试追问 3：Amdahl 定律在大模型训练里体现在哪里？

答：体现在那些不随卡数缩小的部分上。例如 dataloader 的串行段、优化器步进里的全局梯度范数、日志和 checkpoint、all-reduce 的延迟项 $2(p-1)\alpha$、流水线气泡，以及负载不均时最慢的那个 rank。卡数上去之后，正是这些部分决定了 MFU 的上限。

常见错答：「训练是完全数据并行的，没有串行部分」。

C. Work–span 模型与调度

Amdahl 定律只把程序分成串行和并行两段。更细致的做法是把并行计算看成一张有向无环图：T_1 是所有节点的总工作量（work），T_∞ 是最长依赖链的长度（span，也叫关键路径），两者之比 T_1/T_∞ 就是并行度。Brent 定理 [20] 说明，任何贪心调度都满足

$$T_P \le \frac{T_1}{P} + T_\infty, \tag{4}$$

Blumofe 和 Leiserson 进一步证明，随机 work stealing 在期望意义下能达到 $T_1/P + O(T_\infty)$ [21]。work stealing 的做法是：每个线程从自己的双端队列尾部取任务，空闲时随机挑一个别人的队列，从头部「偷」一个。偷到的往往是大块工作，所以偷的次数少，线程之间的争抢也少。式 (4) 还有一个直接推论：只有当 $P \ll T_1/T_\infty$ 时，才能得到接近线性的加速。

scan（前缀和）是检验 work–span 思维的标准例子 [22], [23]。Hillis–Steele 算法的 span 是 $\log_2 n$，work 是 $n \log_2 n$；Blelloch 算法的 work 是 $2(n-1)$，span 是 $2\log_2 n$。换句话说，前者用更多的工作换更短的关键路径，后者反过来。**本文配套实验**（CS149 A3）中，计数器的读数和这些公式逐一相等。$n = 2^{20}$ 时，Blelloch 做了 2.10M 次加法、走了 40 步；Hillis–Steele 做了 19.9M 次、走了 20 步。哪个更快取决于处理器数：处理器比数据还多时（$n = 1000, P = 2048$），Hillis–Steele 的 Brent 上界是 15，低于 Blelloch 的 21；处理器少时 work 占主导，Blelloch 胜出。落到 GPU 上，工程结论是：大数组用 work-efficient 的 Blelloch；warp 内只有 32 个元素，线程本来就空着，用 Hillis–Steele 更合适。

work–span 的另一面是负载均衡。**本文配套实验**（CS149 A1）在 Mandelbrot 上比较了三种划分方式。4 个 worker 时，连续块划分只有 2.93 倍加速，交错行划分 3.93 倍，动态分块 3.76 倍。原因在于图像各处的计算量很不均匀：8 块静态划分时，中间块耗时 0.233 s，边缘块只有 0.063 s，连续块划分下拿到中间块的 worker 拖慢了整体。这个实验还有一段插曲。我的第一版测出 8 个 worker 有 21 倍的「超线性加速」。查下来，原因是块按 worker 数来切，切小以后 numpy 中间数组的缓存行为变了，串行基线因此并不公平。这是一个很好的面试故事，它说明看到超线性加速时，首先要怀疑基线。

面试追问 4：静态分配和动态分配怎么选？work stealing 为什么好？

答：看任务代价能不能预测。代价可预测而且均匀时用静态划分，没有任何调度开销；不可预测时用动态分配。动态分配如果用一个集中式队列，队列本身会成为争抢热点。work stealing 让每个线程优先处理本地任务，只在空闲时去偷，而且偷的是队列另一端的大块任务，所以既均衡又少争抢，满足 $T_P \le T_1/P + O(T_\infty)$。

常见错答：「动态一定比静态好」。任务粒度很细时，入队出队的开销可能比任务本身还大。

D. Little 定律与服务容量

做推理服务的容量规划时，第一个要用的公式是 Little 定律 [24]：对任何处于稳态的排队系统，$L = \lambda W$，即系统

中的平均请求数等于到达率乘以平均停留时间。它不依赖到达和服务的具体分布，所以几乎总能用。举个例子：平均输出 500 token，每 token 时延（TPOT）30 ms，首 token 时延（TTFT）0.5 s，那么一个请求的停留时间 $W \approx 15.5$ s。如果 KV cache 的显存预算只允许 64 条并发，能承受的最大到达率就是 $\lambda_{\max} \approx 64/15.5 \approx 4.1$ req/s。

同一个定律也解释了 GPU 为什么需要大量线程来隐藏延迟。要维持带宽 BW，而每次访存的延迟是 ℓ，那么任何时刻在途的字节数必须达到 $\mathrm{BW} \cdot \ell$。occupancy 和异步拷贝之所以存在，就是为了凑够这么多在途请求（第 III 节）。

白板估算 1：H100 需要多少在途字节才能吃满 HBM？

取 HBM 带宽 3.35 TB/s，访存延迟约 500 个周期，也就是约 0.3 µs（附录表 XVII 的数量级）。由 Little 定律，在途字节数 $\approx 3.35 \times 10^{12} \times 3 \times 10^{-7} \approx 1$ MB。分到 132 个 SM 上，每个 SM 约要 7.6 KB 在途。如果每个线程每次发起 16 字节的向量加载，每个 SM 就需要约 480 个线程同时有访存在途。所以访存受限的 kernel 要么需要较高的 occupancy，要么需要 TMA 或 `cp.async` 这类异步批量拷贝，一次发出大块请求。

E. I/O 复杂度下界

roofline 告诉我们字节数 B 决定了访存受限 kernel 的时间，那么 B 最少能是多少？这就是 I/O 复杂度下界要回答的问题。Hong 和 Kung 用红蓝卵石游戏 [25] 证明：快速存储的容量为 M 时，$n \times n$ 矩阵乘至少需要 $\Omega(n^3/\sqrt{M})$ 次慢速存储访问。这个下界说明分块不是可有可无的技巧，而是达到最优的必要条件：任何达到下界的算法，都必须让每个加载进快存的元素被复用 $\Theta(\sqrt{M})$ 次。FlashAttention 论文给出了 attention 的类似下界 $\Omega(T^2d^2/M)$ [26]。第 IV 节会看到，当 d^2/M 不算太小时，这个下界意味着 HBM 流量仍是 T 的二次函数。

面试要点

- roofline：$t \geq \max(F/P, B/\mathrm{BW})$。H100 拐点 295、A100 拐点 153 FLOP/B；用的时候要说明对应哪个计算单元、哪一级存储。
- Amdahl 给固定规模的上限，Gustafson 给扩展规模的视角；训练中的串行项包括通信延迟和流水线气泡。
- work–span：$T_P \leq T_1/P + T_\infty$；scan 的两种算法分别优化 span 和 work。
- Little 定律 $L = \lambda W$ 既能做服务容量规划，也能解释 GPU 的延迟隐藏。

研究视角

- roofline 假设计算和访存完全重叠。对 FlashAttention 这类「矩阵乘与逐元素运算交替」的 kernel，重叠不完美会带来多大偏差？可以用 Nsight 的指令级采样，把时间分解为 tensor core 在忙、在等 TMA、在等 softmax 三部分。
- 设计实验检验 Little 定律在 LLM 服务里的适用边界：在 continuous batching 下，停留时间 W 本身依赖并发数 L（batch 越大，每一步越慢），系统是否仍然存在唯一的稳态？

III. 并行硬件与编程模型

上一节的 roofline 只告诉我们「上界在哪」。一个 kernel 离上界很远时，要解释它为什么慢，就得知道硬件是怎样把晶体管变成吞吐的。本节对应 CS149 的前半部分，依次讨论多核、SIMD、硬件多线程、cache coherence 和内存一致性，再把这些概念对应到 GPU 的 SIMT 模型上。这一层的面试题看起来很基础，但回答「这个 kernel 为什么慢」时，能依靠的只有这些知识。

A. 多核、SIMD 与硬件多线程

现代处理器用三种相互独立的手段提高吞吐 [27]。第一种是多核，把整条取指–执行流水线复制多份，利用任务并行。第二种是 SIMD，一条指令同时驱动 W 个 ALU（AVX2 是 8 个 fp32 lane，AVX-512 是 16 个），取指和译码的开销因此摊到 W 个数据上，利用的是数据并行。第三种是硬件多线程（SMT），一个核上保存多个线程的上下文，某个线程等内存时就切换到另一个，用来隐藏延迟。GPU 把这三种手段都推到了极致：H100 有 132 个 SM，每个 SM 最多同时驻留 64 个 warp，每个 warp 的 32 个线程以 SIMT 方式一起执行 [28], [29]。

数据并行最主要的效率损失来自 SIMD 发散（divergence）。一组 W 个 lane 必须一起等到组内最慢的 lane 做完，所以利用率是

$$U = \frac{\sum_i w_i}{W \cdot \max_{i \in \text{gang}} w_i}, \tag{5}$$

其中 w_i 是第 i 个 lane 的有效迭代数。这个代价可以非常大。**本文配套实验**（CS149 A1）的模拟显示，每个 8 宽的 gang 里只要有一条慢 lane，利用率就跌到 0.163，加速只有 1.30 倍，而不是 8 倍。在 numpy 中实测也是如此：对「每 16 个元素里有 1 个慢元素」的输入，掩码执行和「全部元素都慢」一样贵（353 对 356 ns/元素）；先把活跃元素压缩到一起再算，就降到了 29 ns。GPU 的 warp 发散和这完全是同一回事。工程上的解法是按工作量排序或分桶，让同一个 warp 内各线程的工作量接近；或者改成 persistent kernel，让线程持续从队列里拉取新工作。

面试追问 5：SIMD/warp 发散的代价怎么算？if/else 只执行一个分支，为什么还有代价？

答：因为同一个 warp 的线程共享一个程序计数器，两个分支只能先后执行，不该执行的 lane 被掩码屏蔽。Volta 之后有了独立线程调度，但发散的路径仍然是串行化的。代价是 $W \cdot \max w_i / \sum w_i$ 倍，lane 之间的工作量差得越多，代价越高。

常见错答：「if/else 两个分支只执行一个，所以没有代价」；或者以为发散只发生在 GPU 上，CPU 的 SIMD 不会发散。

硬件多线程是用并发换延迟。假设访存延迟是 ℓ 个周期，每个线程两次访存之间平均有 c 个周期的计算，那么大约需要 ℓ/c 个线程才能让核一直有活干。代价是每个线程都要占用寄存器和上下文，片上存储被分摊得更薄。GPU 的 occupancy，也就是每个 SM 实际驻留的 warp 数与上限之比，就是这件事的量化指标。

B. Cache coherence、false sharing 与同步

多个核共享内存时，每个核私有的 cache 里可能有同一份数据的副本，于是需要一致性协议。MESI 一类基于失效的协议 [30] 保证同一个地址上的写是串行化的：一个核要写某条 cache line，必须先让其他核的副本失效。这条规则带来了 false sharing（伪共享）：两个核写的是同一条 64 字节 cache line 上的不同变量，协议却会让这条 line 在两

个核之间来回迁移，每次耗时数十到上百 ns。它的症状是线程加多了，程序反而变慢。修法是按 cache line 填充，或者让每个线程写私有副本，最后再归约。

coherence 只管单个地址。不同地址之间的读写以什么顺序被其他核看到，由内存一致性模型（consistency）规定。顺序一致性（SC）[31] 要求所有核看到同一个全局交错顺序；x86 的 TSO 允许「写后读」重排，因为写操作会先进 store buffer；ARM 和 GPU 采用更弱的模型 [32]。程序员需要用 fence 或 acquire/release 原子操作来建立 happens-before 关系。GPU 上一个典型的坑是：用一个普通标志位在 block 之间传递「数据已就绪」，如果没有 `__threadfence()`，另一个 SM 可能先看到标志、后看到数据。

无锁数据结构依赖 CAS（compare-and-swap），它有一个经典问题叫 ABA。线程读到栈顶是 A 之后被挂起；其他线程弹出 A、弹出 B、又把 A 压回去。等这个线程恢复执行，CAS 发现栈顶仍是 A，于是成功了，但 A 的 next 指针早已失效。解法有带版本号的指针、hazard pointer 和 epoch 回收。

面试追问 6：什么是 false sharing？怎么发现，怎么修？

答：不同的核写同一条 cache line 上的不同变量，coherence 协议会让这条 line 反复失效、来回迁移。症状是线程数增加而吞吐下降，用 perf 看会发现 HITM 事件（命中了其他核修改过的 line）很高。修法是按 64 B 对齐填充每个线程的计数器，或者让每个线程写私有副本再归约。

常见错答：「没有共享变量就没有竞争」。竞争的单位是 cache line，不是变量。

面试追问 7：x86 上一个线程写 `data=1; flag=1`，另一个线程 `while(!flag); print(data)`，一定打印 1 吗？在 ARM 或 GPU 上呢？

答：在 x86 上，硬件层面会打印 1，因为 TSO 不重排「写–写」和「读–读」。但编译器仍可能重排这两条语句，或者把 `flag` 提到寄存器里，所以必须用 `std::atomic` 的 release/acquire。ARM 和 GPU 的弱内存模型允许写–写重排，没有 fence 时可能打印 0。

常见错答：「有 cache coherence 就一定能看到最新值」。coherence 不约束不同地址之间的顺序。

同步原语怎么选也是高频题。**本文配套实验**（CS149 A2）在 Python 中复刻了 CS149 的任务系统，测量 8 个空任务一次 launch 的开销：基于条件变量（sleep）的线程池是 38.7 µs，每次新建线程是 3169 µs，纯自旋的线程池却要 31–50 ms。在 C++ 里，自旋线程池本应最快。它在 CPython 里最慢，是因为自旋的线程一直持有 GIL，要等 5 ms 的切换间隔才让出，空闲时还烧满一个核。这个实验的价值在于说明：「spin 还是 sleep」取决于自旋的线程会不会挡住真正干活的线程，并没有一个与平台无关的标准答案。同一个实验里，异步依赖图让 48 个 9 任务的 launch 从 259 ms 降到 152 ms，实测 1.71 倍，接近理想比值 $2/(9/8) = 1.78$。这说明加速来自填满了每次 launch 尾部的空闲，CUDA stream 和 CUDA Graph 的动机也是这个。

C. GPU 的 SIMT 编程模型

有了 CPU 上的这些概念，GPU 就好理解了。CUDA 把计算组织成 grid、block、thread 三级：同一个 block 内的线程可以通过 shared memory 通信，并用 `__syncthreads()` 同步；不同 block 之间默认相互独立 [29]。硬件上，block 被调度到 SM 上，SM 以 32 个线程组成的 warp 为单位发射指令。H100 的存储层次见表 II。寄存器和 shared memory 的带宽比 HBM 高一个数量级以上，延迟低一个数量级，所以几乎所有 GPU 优化都在做同一件事：把数据搬到离计算近的地方，尽可能多地复用，再写回去 [33], [34]。

TABLE II
H100 SXM 存储层次的数量级（附录表 XVII 的子集）

层级	容量	带宽量级	延迟与说明
寄存器	256 KB/SM	—	1 周期；寄存器分块是 GEMM 复用的最内层
Shared/L1	≤228 KB/SM	∼30 TB/s（全卡）	20–30 周期；软件管理，32 个 bank
L2	50 MB	∼10 TB/s	∼200 周期；K/V 重读多命中于此
HBM3	80 GB	3.35 TB/s	∼500 周期；roofline 的默认 B
主机内存	数百 GB	32–64 GB/s	微秒级；经 PCIe，offload 的瓶颈

在这个层次结构上，有三个概念最常被问到：访存合并、bank conflict 和 occupancy。

访存合并（coalescing）说的是 HBM 的访问方式。一个 warp 的 32 个线程如果访问连续且对齐的地址，硬件会把它们合并成少量 32 B 或 128 B 的事务。如果按列访问一个行主序矩阵，相邻线程的地址相差一整行，每个线程各占一个事务，有效带宽会下降一个数量级。常见的修法是借 shared memory 做一次转置中转。

bank conflict 说的是 shared memory。shared memory 分成 32 个 bank，每个 bank 4 字节宽；同一个 warp 里的线程如果访问同一个 bank 的不同地址，这些访问会被串行化。典型例子是按列读取一个 32×32 的 float 矩阵：所有线程都落在同一个 bank 上。把它声明成 32×33，多填充一列，就能错开。所有线程读同一个地址则不算冲突，硬件会广播。

面试追问 8：什么是 memory coalescing？给一个不合并的例子。

答：合并以 warp 为单位，看的是 32 个线程地址的排列：地址连续且对齐时，一个 128 B 事务就能服务整个 warp。不合并的例子有：按列读行主序矩阵、在结构体数组（AoS）里只读一个字段、随机 gather。修法是改数据布局（改成 SoA）、借 shared memory 转置，或者让相邻线程处理相邻元素。

常见错答：「同一个 block 内的访问会自动合并」；或者把 coalescing 和 shared memory 的 bank conflict 混为一谈。

面试追问 9：Shared memory bank conflict 怎么产生、怎么避免？

答：同一个 warp 中多个线程访问同一个 bank 的不同地址时，访问会被串行化。按列访问 32×32 的 float 矩阵就是 32 路冲突。填充成 32×33 后，第 i 行第 j 列落在 bank $(33i + j) \bmod 32$，冲突就消失了。Hopper 的 TMA 和 swizzle 布局也是为了解决这件事。

常见错答：「bank conflict 发生在 global memory」；「所有线程读同一个地址也冲突」（那是广播，不冲突）。

最后是 occupancy。它是隐藏延迟的一种手段，本身并不是优化目标。Volkov 的经典报告 [33] 指出，靠指令级并行（ILP）和寄存器分块，低 occupancy 的 kernel 可以跑得更快。高性能 GEMM 常让每个线程算 8×8 以上的输出块，占用大量寄存器，occupancy 只有 25% 左右，但数据复用率高得多。CUTLASS [34] 的层次化分块把这个思路系统化了：先切成 threadblock tile，再切成 warp tile，最后是 MMA tile。

面试追问 10：occupancy 越高越好吗？

答：不是。occupancy 只是隐藏延迟的一种方式，另一种方式是 ILP 和异步拷贝。寄存器分块很重的 GEMM 和 FlashAttention 都是故意降低 occupancy 来换取复用。判断标准是吞吐：Nsight Compute 显示 stall 主要是「等内存」时，提高 occupancy 有用；主要是「等 tensor core」时，提高 occupancy 没用。

常见错答：「把 occupancy 拉到 100%，性能就最好」。

D. *Tensor core* 与 *Hopper* 的异步执行

GPU 的矩阵算力主要来自 tensor core，它的演进方向值得了解。Volta 引入 tensor core，每个时钟完成一个小矩阵乘加 [35]；Ampere 增加了 BF16、TF32 和 2:4 结构化稀疏 [14], [36]；Hopper 增加了 FP8、TMA（Tensor Memory Accelerator，异步地批量搬运张量块）、warpgroup 级的异步 MMA 指令（wgmma），以及 thread block cluster 和分布式 shared memory [15]。这些特性指向同一个方向：让搬运和计算由不同的硬件单元异步完成。这样软件就能组织成流水线，一部分 warp（producer）负责搬数据，另一部分（consumer）负责算。FlashAttention-3 的 warp specialization 就建立在这些特性之上 [11]。

面试要点

- 三种并行：多核对应任务并行，SIMD/SIMT 对应数据并行，多线程用来隐藏延迟；发散的代价按组内最慢的 lane 计算。
- coherence 管单个地址，consistency 管多个地址之间的顺序；false sharing 的单位是 cache line；无锁结构要注意 ABA。
- GPU 的三件事：访存合并看 warp 的地址模式，bank conflict 发生在 shared memory，occupancy 是手段而不是目标。
- Hopper 的方向是异步：TMA 负责搬，wgmma 负责算，软件用 warp specialization 把两者组成流水线。

研究视角

- 如何自动选择 warp specialization 中 producer 与 consumer 的划分以及流水线深度？这是一个带资源约束（寄存器、shared memory）的调度问题，目前多由专家手写（CUTLASS、FlashAttention-3）。
- 设计一个微基准，分别测量 H100 上 shared memory bank conflict、L2 命中和 TMA 异步拷贝对一个分块 GEMM 的贡献。关键是每次只改变一个因素，并给出与 roofline 上界之间差距的分解。

IV. 算子与内核：GEMM、FlashAttention 与融合

Transformer 的 FLOPs 绝大部分花在 GEMM 上，但端到端时间里有相当一部分，却花在访存受限的「小算子」和 attention 的中间矩阵上。这一节想讲清楚三件事：GEMM 怎样通过分块获得复用，它的上界在哪里；FlashAttention 三个版本各改了什么，以及一个常被误解的问题，即它到底把 HBM 流量降了多少；最后是算子融合和 Triton。

A. *GEMM* 的算术强度与分块

先看最基本的情形。$C_{M\times N}=A_{M\times K}B_{K\times N}$ 的运算量是 $F=2MNK$（一次乘加记 2 FLOP，厂商标称的峰值也是这么算的）。如果三个矩阵各从 HBM 过一次，强度就是

$$I_{\text{GEMM}}=\frac{2MNK}{s(MK+KN+MN)},\qquad I_{\text{square}}=\frac{2n}{3s},\qquad I_{\text{GEMV}}\approx\frac{2}{s},\tag{6}$$

其中 s 是每个元素的字节数。BF16 方阵的强度是 $n/3$，在 H100 上要 $n\gtrsim 900$ 才能越过拐点 295。GEMV（$M=1$）的强度只有 1 FLOP/B，最多只能用到峰值的 1/295。这一个式子就解释了 decode 为什么慢，也解释了为什么 batch 是推理吞吐的第一杠杆：batch 把 GEMV 变回 GEMM，强度随之上升。

上面的式子有一个前提：每个矩阵只过一次 HBM。这要求片上能放下足够大的块。设快存能放 M_s 个元素，采用 $b\times b$ 的方块，三个块同时驻留，就需要 $3b^2\le M_s$。每加载两个 $b\times b$ 块做 $2b^3$ 次运算，局部强度约为 b/s。总访存量是

$$Q\approx\frac{2n^3}{b}+n^2=\Theta\!\left(\frac{n^3}{\sqrt{M_s}}\right),\tag{7}$$

与 Hong–Kung 下界同阶 [25]。换句话说，块的边长受片上容量的平方根限制，局部强度也就受它限制。

白板估算 2：H100 一个 SM 的 shared memory 能支撑多大的 GEMM 块？

H100 每个 SM 最多有 228 KB shared memory。BF16 下 $3b^2\times 2\le 228$ KB，得到 $b\le 197$，工程上取 128。这时单个 block 的局部强度是 $b/s=64$ FLOP/B，还不到拐点 295。真实的 kernel 靠三件事越过拐点：多个 thread block 在 50 MB 的 L2 里共享 A 的同一行块和 B 的同一列块（L2 带宽约为 HBM 的 3 倍）；使用非方形的块（如 128×256）；在寄存器层再分一次块。所以面试中回答「块越大越好」并不完整。块的大小同时受 shared memory、寄存器、occupancy 和 SM 数量的约束，块数还必须多到能填满 132 个 SM。

同样的规律在加速器上也成立。**本文配套实验**（CS149 A4）用一个带 SBUF/PSUM 容量约束的 Trainium 式分块模型，计算了 4096^3 的 BF16 GEMM。tile 边长从 128 增加到 256、512 时，HBM 强度依次是 63.0、124.1、240.9 FLOP/B，大约等于边长的一半，和上面 b/s 的估计一致。SBUF 达到 1 MiB 以后，瓶颈换成了 2 MiB 的 PSUM（存放 FP32 累加器），它把强度封顶在 315.1；再增大 SBUF 只能加大 t_k，让 DMA 次数从 2080 降到 96，强度却不再提高。这些加速器参数是模型假设，不能当成 Trainium 的真实规格（见第 V 节）。

面试追问 11：矩阵乘分块时块大小怎么选？

答：块大小由四个约束共同决定。一是容量：$3b^2s\le M_s$，做双缓冲时还要减半。二是强度：局部强度约为 b/s，往往需要借助 L2 复用才能越过拐点。三是并行度：$(M/b_m)(N/b_n)$ 个块要足以填满所有 SM，矩阵小时常用 split-K。四是硬件形状：块要是 MMA 指令形状的整数倍，同时兼顾 occupancy 和寄

存器压力。最后由 autotuning 在候选集合里挑。

常见错答：「越大越好」；或者只考虑 shared memory 容量，忽略了 SM 数和波次量化（wave quantization，块数不是 SM 数整数倍时最后一波吃不满）。

知道了单个 GEMM 的上界，训练整体的效率怎么衡量？标准口径是 MFU（model FLOPs utilization）[37]：$\text{MFU} = 6N\cdot(\text{tokens/s})/(n_{\text{GPU}}P_{\text{peak}})$。例如 8 张 H100 训练 7B 模型，达到每秒 10^5 token，$\text{MFU} = 6\times 7\times 10^9\times 10^5/(8\times 989\times 10^{12}) \approx 53\%$。HFU 会把重计算的 FLOPs 也算进去，所以总有 $\text{HFU} \geq \text{MFU}$。作为大规模训练的参考量级，MegaScale 报告在 12,288 张 GPU 上训练 175B 模型达到了 55.2% 的 MFU [38]。

面试追问 12：batch=1 的矩阵–向量乘为什么在任何 GPU 上都慢？

答：因为它的强度太低。BF16 GEMV 的强度约为 1 FLOP/B，而 H100 的拐点是 295，所以只能用到峰值的 0.3%。要提高强度只有两条路：增加 batch，把 GEMV 变成 GEMM；或者减少每个权重的字节数，也就是量化。换更快的卡帮助有限，H100 的带宽只比 A100 多 1.6 倍。

常见错答：「decode 慢是因为计算量大」。

B. Attention 的代价

GEMM 的问题是复用不够，attention 的问题则是中间矩阵太大。单头 attention 中 $Q,K,V\in\mathbb{R}^{T\times d}$，$S=QK^\top$ 和 $O=PV$ 各需 $2T^2d$ FLOPs，多头合起来每层前向是 $4T^2d_{\text{model}}$。朴素实现要把 $T\times T$ 的 S 和 P 物化出来，每层每个样本要占 hT^2s 字节显存。

白板估算 3：8K 上下文的 attention 占多少计算和显存？

取 $T=8192$、$d_{\text{model}}=4096$、32 头、BF16。attention 前向 $4T^2d_{\text{model}} = 4\times 8192^2\times 4096 \approx 1.1$ TFLOP/层；同一层的投影和 MLP 约为 $24Td_{\text{model}}^2 \approx 3.3$ TFLOP（按 4 倍 MLP 估），所以 attention 约占 1/4。分数矩阵却有 $32\times 8192^2\times 2$ B ≈ 4.3 GB/层/样本，反传时还要保存。可见在中等长度下，attention 首先是*显存与 IO* 的问题，其次才是 FLOPs 的问题。这正是 FlashAttention 的出发点。

C. Online softmax 与 *FlashAttention*

不把 S 写回 HBM 的难点在 softmax：它看起来需要先知道整行的最大值和总和。Milakov 和 Gimelshein 的 online normalizer [39] 解决了这个问题。他们注意到 $e^{x-m_{\text{new}}} = e^{x-m_{\text{old}}}\cdot e^{m_{\text{old}}-m_{\text{new}}}$，所以只要每段维护统计量 (m_i,ℓ_i,o_i)，两段就可以这样合并：

$$\begin{aligned} m &= \max(m_1,m_2), \quad \ell = e^{m_1-m}\ell_1 + e^{m_2-m}\ell_2, \\ o &= e^{m_1-m}o_1 + e^{m_2-m}o_2, \qquad O = o/\ell. \end{aligned} \tag{8}$$

Rabe 和 Staats 据此证明，self-attention 只需要 $O(\sqrt{n})$ 乃至 $O(\log n)$ 的额外显存 [40]。FlashAttention [26] 把它和 GPU 的存储层次结合起来：把 Q 切成每块 B_r 行，把 K,V 切成每块 B_c 行，S 和 P 的块只存在于 SRAM 中；反向传播时保存每行的 logsumexp $L_i = m_i+\log\ell_i$，再按块重算 $P_{ij}=\exp(S_{ij}-L_i)$。要注意，它是*精确*算法，并不减少 FLOPs，反向还要多做一次 $QK^\top$ 的重算。它省下的是 HBM 读写和显存，额外显存从 $O(T^2)$ 降到了 $O(T)$。

那么 HBM 流量降了多少？这要看 IO 复杂度。设 SRAM 能放 M 个元素，要同时放下 K_j,V_j 和 Q_i,O_i，论文取 $B_c=\lceil M/4d\rceil$、$B_r=\min(\lceil M/4d\rceil,d)$。外层每遍历一个 K/V 块，就要把全部 Q,O 扫一遍，所以总访问量是

$$Q_{\text{flash}} = \Theta\left(\frac{T^2d^2}{M}\right), \tag{9}$$

论文还证明，没有哪个精确算法能在所有 $M\in[d,Td]$ 上做到 $o(T^2d^2/M)$ [26]。问题出在常数上：$d=64$–128、$M\approx 10^5$ 时，d^2/M 只是一个约 0.04–0.16 的常数，*所以流量在 T 上仍是二次的*，只是常数比朴素实现小。

图 4 是这个结论的直接证据。配套实验用 numpy 实现了按 FA2 顺序的分块前向，以及基于 logsumexp 的反向。在 float64 下，O、dQ、dK、dV 与 `torch` 自动微分的最大误差 $\leq 4.9\times 10^{-15}$，说明实现是对的；IO 计数器则记录每一次块读写。$d=64$、$T=65536$ 时，朴素实现是 34,393 MB，Flash 是 17,197 MB，比值 2.00；$d=128$ 时比值是 1.56。这和白板公式吻合：Flash 的流量 $\approx 2Td\cdot(T/B_r)+4Td$，朴素实现 $\approx 4T^2$，比值 $\approx 2B_r/d$，$B_r=d$ 时正好是 2。

这个结果容易被误读，所以要说清它不能推出什么。它不能推出「FlashAttention 在 GPU 上只快 2 倍」。真实的 eager 实现里，scale、mask、softmax、dropout 各自要把 T^2 的张量读写一遍，S 还常以 FP32 存储；而 K/V 的重读在 GPU 上大多命中 L2。FlashAttention 最确定的收益是显存从 $O(T^2)$ 降到 $O(T)$，以及消除了这些中间 kernel。同理，numpy 版的 Flash 比朴素版还慢（$T=1024, d=128$ 时 44.6 对 22.9 ms），那是 Python 循环的开销，和 GPU 无关。

面试追问 13：FlashAttention 快在哪？它减少了 FLOPs 吗？HBM 访问降到线性了吗？

答：它快在不把 $T\times T$ 的分数矩阵写回 HBM，FLOPs 并没有减少，反向还多一次重算，结果也是精确的。做法是分块计算，用 online softmax 维护每行的最大值和分母，把 matmul、softmax、matmul 融合成一个 kernel。显存从 $O(T^2)$ 降到 $O(T)$；HBM 访问是 $\Theta(T^2d^2/M)$，对常见的 d 来说在 T 上仍是二次，只是常数小了很多。

常见错答：「它是近似 attention」；「访存从二次降到了线性」。降到线性的只有显存。

既然 IO 的阶没法再降，后两个版本的改进就都落在工程上。FlashAttention-2 [17] 的改动见表 III。它把外层循环改为遍历 Q 块，每个 thread block 负责一段 Q，输出累加器留在寄存器里，最后只写一次。它推迟 rescale，只保存 logsumexp，以减少非矩阵乘的 FLOPs，因为 A100 上非 tensor-core 的 FLOP/s 比 tensor core 低约 16 倍。它除了在 batch 和 head 上并行，还沿序列长度并行，所以长序列、小 batch 时也能填满 SM。在 block 内部，它把 Q 切给 4 个 warp，避免了 v1 中 warp 之间经 shared memory 做归约。论文报告在 A100 上达到了理论峰值的 50–73%。

FlashAttention-3 [11] 面向 Hopper，核心是让搬运、矩阵乘和 softmax 互相重叠。producer warp 只负责发 TMA 加载，consumer warpgroup 做 wgmma 和 softmax，这叫 warp specialization；两个 consumer warpgroup 轮流交替（ping-pong），一个做 softmax 时，另一个做 GEMM；在同一个 warpgroup 内部，softmax 也和下一块的 GEMM 重叠。FP8 路径用分块量化和 Hadamard 变换（incoherent processing）

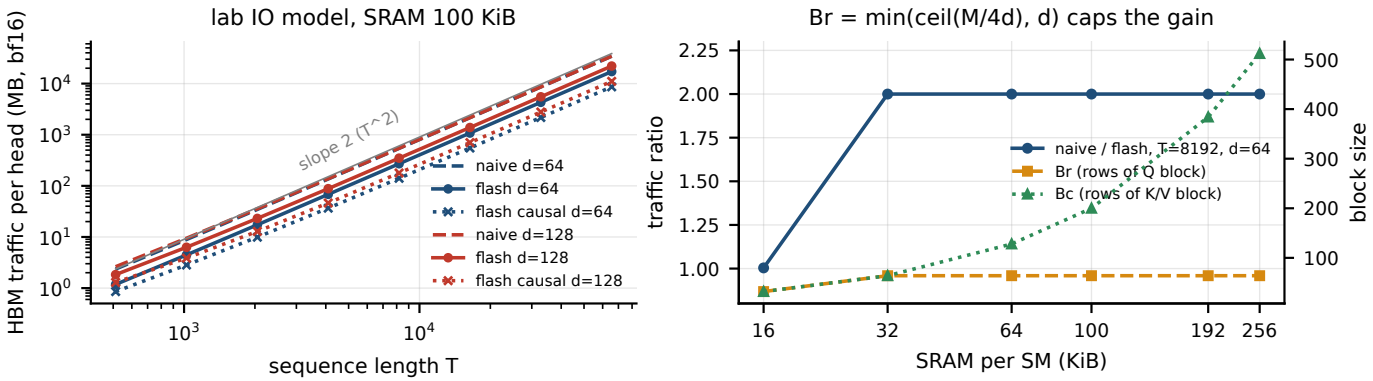


Fig. 4. 本文配套实验（CS229S P1）中 FlashAttention IO 计数器的结果。左：每个头的 HBM 流量随 T 的变化（BF16，SRAM 100 KiB），朴素实现只计 S、P 各读写一次。所有曲线的斜率都是 2，也就是 $\Theta(T^2)$。非因果时 Flash 只比朴素少 2.00 倍（$d=64$）或 1.56 倍（$d=128$）；因果时由于跳过了上三角的块，约少 4 倍。右：$T=8192, d=64$ 时流量比随 SRAM 大小的变化。B_r 在 32 KiB 处就被 d 截住了，更大的 SRAM 只能增大 B_c，不再减少 K/V 的重读次数，所以曲线变平。IO 计数器和解析模型在 8 组配置上逐元素一致。

TABLE III
FLASHATTENTION 三个版本的对比

	FA1 (2022)	FA2 (2023)	FA3 (2024)
目标硬件	A100 等	A100	H100 (Hopper)
外层循环	K/V 块	Q 块	Q 块
O 的写回	每个 K/V 块读写一次	只写一次	只写一次
并行维度	batch×head	+ 序列长度	+ warp specialization
warp 分工	按 K/V 切，需归约	按 Q 切，无归约	producer/consumer、ping-pong
非 matmul FLOPs	每步 rescale	推迟到末尾	与 GEMM 重叠
低精度	FP16/BF16	FP16/BF16	+ FP8（分块量化、Hadamard）

压低离群值带来的误差。论文报告在 H100 上 BF16 约 740 TFLOP/s，约为 75% 的利用率。

面试追问 14：FlashAttention-2 相对 v1 改了什么？为什么不是「换了算法」?

答：数学上它和 v1 是同一个 online softmax。改的是工程实现：循环顺序（外层遍历 Q 块，O 只写一次）、非矩阵乘运算的数量、沿序列长度并行，以及 warp 之间的分工。这些改动提高了 tensor core 的占用率，但没有改变 IO 的阶。

常见错答：「v2 用了新的近似」，或者「v2 降低了 IO 复杂度的阶」。

到这里讨论的都是 prefill 或训练。decode 阶段的 attention 是另一回事。每个头的 Q 只有一行，计算量是 $4Td$ FLOPs，却要读 $2Td$ 个 KV 元素，MHA 时强度只有 $2/s = 1$。GQA 让 H/H_{kv} 个查询头共享一组 KV，强度变成 $2(H/H_{kv})/s$，GQA-8 是 4 FLOP/B（配套实验 `decode_attention_intensity.csv`）。这时的瓶颈是把 KV 读一遍，根本没有 T^2 的中间矩阵可省，FlashAttention 的那部分收益也就不存在了。这时需要的是沿 KV 长度切分来并行（Flash-Decoding，也就是 split-K [41]），让单条长序列也能用满所有 SM。

面试追问 15：为什么 decode 时 FlashAttention 帮不上大忙？

答：因为 decode 每步的 Q 只有一个 token，没有 $T\times T$ 的矩阵可省，瓶颈在读 KV。batch 小、序列长的时候还会并行度不足：按 batch×head 划分只有几十个 block，填不满 132 个 SM。Flash-Decoding 沿 KV 长度切分，每段各自算出局部的 (m,ℓ,o)，再用式 (8) 合并。

常见错答：「FlashAttention 在任何场景都能加速 attention」。

D. 算子融合

FlashAttention 是一种特殊的融合。更一般的融合省下的是中间张量的 HBM 往返和 kernel 发射：k 个逐元素算子融合后，流量从 $2kS$ 降到 $2S$。Ivanov 等人的分析指出，Transformer 训练中有大量时间花在受数据搬运限制的非 GEMM 算子上 [42]。

但流量省了多少，不等于时间省了多少，端到端收益要用 roofline 来看。**本文配套实验**（15-849 编译与压缩）在自写的图 IR 上，对 Transformer 的 MLP 块（$T=4096, d=4096, d_{ff}=16384$，BF16，H100 模型）先做逐元素融合，再做 GEMM epilogue 融合。HBM 流量从 1.376 GB 降到 1.040 GB，再降到 0.705 GB（−49%），roofline 时间却只从 1342 µs 降到 1132 µs（−16%），见图 14 右。原因是两个 GEMM 各要 556 µs，受算力限制，占了总时间的 83%，融合只消掉了访存受限的那一小部分。同一个实验里，TASO 式的 QKV 合并 [43] 在 $T=4096$ 时把 GEMM 流量从 201.3 降到 134.2 MB，时间却不变（208 µs），因为它本来就受算力限制；$T=1$（decode）时，它只省下两次 kernel 启动。

面试追问 16：算子融合省的是什么？什么样的算子不能随便融合？

答：省的是中间结果的 HBM 读写和 kernel 发射开销，收益最大的是逐元素链，比如 bias、GeLU、dropout 加残差。带归约的算子（LayerNorm、softmax）后面如果接需要全局结果的操作，融合就得改写算法，FlashAttention 正是把 softmax 的归约改写成了 online 形式。融合还会增加寄存器压力、降低 occupancy。收益要按 roofline 判断：被融合的部分如果本来就不在关键路径上，端到端几乎不变。

常见错答：「融合减少了 FLOPs」；或者把 49% 的流量下降当成 49% 的加速。

E. Triton 与 kernel 编程的抽象层次

最后一个问题是：这些 kernel 用什么写？Triton [44] 让程序员写 block 级的程序：一个 program instance 处理一个 tile，用张量化的 `tl.load` 和 `tl.dot` 表达计算；block 内部的线程映射、访存合并、shared memory 分配和 tensor core 指令选择都交给编译器。它和 CUDA 的区别在抽象粒度，编程的单位是 tile 而不是线程；和 TVM 的区别在于不做自动调度搜索，只做有限的 autotune。PyTorch 2 的 TorchInductor 会把融合后的图降为 Triton kernel [45]，所以掌握 Triton 已经成了算子岗的基础要求。CUTLASS [34] 则在 C++ 模板层暴露同样的分块层次，适合需要控制每一级流水的场景。

面试要点

- GEMM 强度为 $2MNK/s(MK+KN+MN)$：BF16 方阵是 $n/3$，GEMV 约为 1。分块后的总访存是 $\Theta(n^3/\sqrt{M})$。
- MFU $= 6N\cdot\text{tokens/s}/(nP)$；不要把 HFU 当成 MFU。
- FlashAttention 是精确算法，不减 FLOPs，显存 $O(T)$，IO $\Theta(T^2d^2/M)$ 仍是二次；FA2 改循环和分工，FA3 利用 Hopper 的异步。
- decode attention 的强度是 $2(H/H_{kv})/s$，要用 split-K（Flash-Decoding），而不是 FlashAttention 的思路。
- 融合收益要按 roofline 看：本文实验中流量 -49%，时间只有 -16%。

研究视角

- 消融实验设计：在真实 GPU 上把 FlashAttention 的收益分解为三部分：（a）省去中间 kernel 的 launch，（b）省去 FP32 的 S 读写，（c）K/V 重读命中 L2 的比例。可以人为关闭 L2 持久化（access policy window）或改变 B_r 来单独隔离（c）。
- 对 $d = 256$ 甚至更大的 head 维度（部分新模型这样用），d^2/M 变大，Flash 和朴素实现的流量比会怎样变化？是否存在一个转折点，到那里就需要改变分块的维度（例如沿 d 切分）？

V. 专用加速器：systolic array、TPU 与 Trainium

既然 GPU 已经很快，为什么 Google、AWS 还要自己做芯片？答案在能耗上。通用处理器把大部分能耗花在取指、译码、调度和数据搬运上，真正做运算的部分只占一小份。Horowitz 在 45 nm 工艺下的测量表明，一次 DRAM 访问的能耗比一次 32 位浮点乘法高两个数量级以上，而 8 位整数乘法又比 FP32 乘法便宜一个数量级以上 [46]。所以专用加速器只有两条路可走 [47], [48]：一是把控制开销摊到大量运算上（宽 SIMD、矩阵单元），二是把数据留在离运算最近的地方反复使用。这两条路可以用同一个量来衡量：每从 DRAM 取一个字，能做多少次运算。这正是 roofline 里的算术强度。

A. Systolic array 与数据流

Kung 提出的 systolic 架构 [49] 让数据像心跳一样，在规则排列的处理单元（PE）之间一拍一拍地流动，每个 PE 只和邻居通信。用 $P\times P$ 的阵列算矩阵乘时，A 的一个元素从左侧进入，向右流过 P 个 PE，被复用了 P 次。只有阵列的边界需要读存储，所以片上带宽需求从 $O(P^2)$ 降到了 $O(P)$。按照哪个操作数常驻在 PE 里，可以分出三种数据流 [48], [50]：weight-stationary 让权重常驻，激活横向流动、部分和纵向流动，代表是 TPU v1；output-stationary 让每个 PE 固定累加一个输出；row-stationary 是 Eyeriss 的做法，把卷积的一行权重和对应的输入一起留在 PE 中。

阵列的代价是填充和排空。对 weight-stationary 阵列来说，一个 $P\times P$ 的权重块处理 M 行输入需要 $M+2P-2$ 个周期；如果权重加载不和计算重叠，还要再加 P 个周期，利用率约为 $M/(M+3P-2)$。也就是说，矩阵必须比阵列大得多，开销才摊得薄。**本文配套实验**（CS149 A4）用寄存器级仿真按周期精确地验证了这个公式（图 5 中）。$P=16$ 时，$n=16$ 的矩阵利用率只有 0.258；$n=64$ 时，串行加载权重为 0.582，权重做双缓冲后升到 0.957。反过来固定 $n=128$，$P=4$ 的阵列利用率是 0.928，$P=128$ 只有 0.251：矩阵只有一个块时，双缓冲也救不回来。

面试追问 17：Systolic array 为什么适合矩阵乘？缺点是什么？

答：因为矩阵乘里每个操作数都要参与 $O(n)$ 次运算，systolic 阵列让这种复用发生在相邻 PE 的寄存器之间：从 SRAM 读一次的数被用 P 次，控制逻辑又极少，所以能效很高。缺点有两个。矩阵比阵列小或形状不匹配时，$3P-2$ 的填充排空开销摊不薄，利用率低；对 softmax、归约、gather 这类非 GEMM 运算也不友好，需要额外的向量和标量单元。

常见错答：「TPU 快是因为频率高」。TPU v1 的频率只有 700 MHz。

面试追问 18：weight-stationary 和 output-stationary 怎么选？

答：看哪个操作数复用最多、哪个最大。推理时同一份权重要乘 batch 里的每个样本，让权重常驻就能把权重读取摊到整个 batch 上，适合 batch（M）大的场景。output-stationary 让部分和一直留在 PE 里，省掉部分和的往返，适合收缩维 K 很长、输出较小的场景。

常见错答：「两者只是实现细节，性能一样」。

B. TPU：从推理 ASIC 到光互联超级计算机

TPU v1 [51] 是一颗推理芯片：$256\times256=65536$ 个 8 位 MAC，频率 700 MHz，峰值约 92 TOPS；片上有 28 MiB 软件管理的存储（24 MiB unified buffer 加 4 MiB 累加器），权重来自 8 GiB 的 DDR3，带宽 34 GB/s。这篇论文最有价值的部分不是峰值，而是它的 roofline 分析。线上负载以 MLP 和 LSTM 为主，它们的权重复用程度取决于 batch；而 P99 时延约束（MLP0 为 7 ms）限制了 batch 不能太大。结果大多数应用都落在 roofline 的斜线段上，受权重带宽限制，只有 CNN 接近计算屋顶。

白板估算 4：TPU v1 的拐点与所需 batch

峰值是 $65536\times2\times700\,\text{MHz}\approx92$ TOPS，权重带宽 34 GB/s，拐点就是 $92\times10^{12}/34\times10^{9}\approx2700$ ops/B。INT8 的 MLP 在 batch 为 B 时，每个权重字节做 $2B$ 次运算，所以要 $B\geq1350$ 才受算力限制。线上的 batch 远小于这个数。这和今天 LLM decode 受 HBM 带宽限制是同一个现象，只是拐点从 2700 换成了 H100 的 295。

之后的 TPU 走向了大规模训练。TPU v4 [52] 用光路交换机（OCS）把 4096 颗芯片连成可重构的 3D torus，并加入了面向 embedding 的 SparseCore；每颗芯片的 BF16 峰值是 275 TFLOP/s，配 32 GiB HBM2，带宽约 1.2 TB/s。它的主要编程接口是 XLA 和 GSPMD [53]：程序员给张量加上 sharding 注解，由编译器插入集合通信。

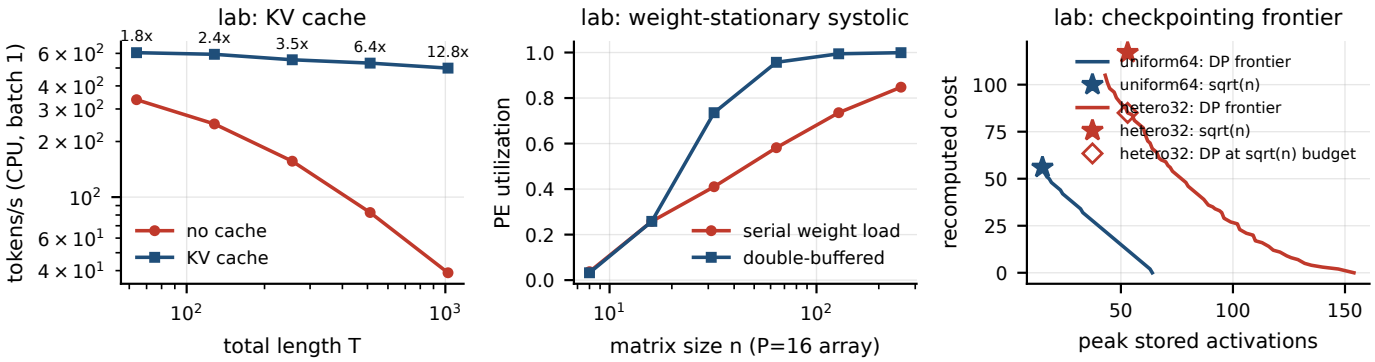


Fig. 5. 本文配套实验（CPU 测量与寄存器级仿真）。左：本地训练的 0.8M 参数 nanoGPT（4 层、宽 128）在 CPU、batch 1 下有无 KV cache 的生成吞吐。加速比从 $T = 64$ 的 1.8 倍增长到 $T = 1024$ 的 12.8 倍，因为没有 cache 时每一步都要重算整个前缀，总代价是 $O(T^2)$。中：16×16 weight-stationary systolic 阵列的 PE 利用率随矩阵尺寸的变化；权重双缓冲在 $n \geq 64$ 时把利用率推到 0.95 以上，因为加载权重的周期被藏到了计算后面。右：激活重计算的「峰值存储–重算代价」前沿，星号是 $\sqrt{n}$ 均匀分段，菱形是同一预算下动态规划的解。

C. *AWS Trainium* 与 *NKI*

Trainium 代表了另一种思路：把片上存储完全交给软件管理。Trainium（Trn1）的 NeuronCore-v2 包含 Tensor Engine（128×128 的 systolic array）、Vector Engine、Scalar Engine 和 GpSimd Engine。片上存储分两级：SBUF（24 MiB，分 128 个 partition）存放输入和输出 tile，PSUM（2 MiB）是矩阵乘的累加缓冲 [54]。NKI（Neuron Kernel Interface）是一种嵌在 Python 里的 kernel 语言。在 NKI 中，tile 的第 0 维是 partition 维，最大 128，对应阵列的一条边；`nc_matmul` 沿 partition 维做收缩，结果写入 PSUM；HBM 和 SBUF 之间的搬运全部是显式 DMA。CS149 Fall 2024 的作业 4 就是在这套硬件上手工分块卷积和矩阵乘，体会 tile 形状对利用率的影响。和 GPU 的 shared memory 不同，这里没有 L1/L2 兜底，所有复用都必须显式写出来，写错了就直接撞上 DRAM 带宽墙。

显式化是有代价的。**本文配套实验**（CS149 A4）测量了 im2col：它把卷积展开成 GEMM，数据膨胀倍数约为 $K_H K_W / \text{stride}^2$，ResNet conv1（$7 \times 7$、stride 2）是 12.25 倍。在 numpy 中，im2col 加 GEMM 仍比逐 tap 的直接卷积快，但 3×3 层有 73% 的时间花在拷贝 patch 矩阵上；1×1 卷积没有膨胀，直接卷积反而更快。GPU 上的 implicit GEMM 和 Winograd 就是为了避开这种显式膨胀。

表 IV 把这些加速器放在一起比较。面试中常见的追问是「那为什么 GPU 仍然是主流」。GPU 用大量线程和寄存器文件做通用调度，tensor core 只是嵌在 SM 里的小矩阵单元，面积效率不如大阵列；但它对 softmax、归约、gather 和动态形状的容忍度高得多。ASIC 把大部分面积给了阵列和 SRAM，对编译器和张量形状的要求也就更苛刻。这是通用性和能效之间的取舍，谈不上谁先进、谁落后。

面试要点

- 专用化的核心是提高「每次取数的复用次数」：按能耗排序，DRAM 访问 $\gg$ SRAM 访问 $\gg$ 乘加。
- systolic 阵列的利用率 $\approx M/(M+3P-2)$（串行加载权重时）；大阵列需要大矩阵和权重双缓冲。
- TPU v1 的教训：拐点约 2700，P99 约束下的线上推理多受权重带宽限制，与今天的 LLM decode 是同一个现象。
- Trainium/NKI：SBUF 和 PSUM 两级软件管理存储，partition 维 128，显式 DMA；tile 形状受 PSUM 容量约束。

研究视角

- 对 LLM decode 这类低强度负载，专用加速器应该怎样设计？一个方向是加大片上 SRAM，把权重留在片上（用面积换带宽）；另一个方向是近存计算。可以用本文的 roofline 计算器估算：要让 8B 模型在 batch=1 时 decode 达到 1000 token/s，需要多大的有效带宽？
- 设计一个跨加速器的「可移植性」实验：同一个 attention kernel 在 GPU（Triton）和 Trainium（NKI）上的 tile 选择为什么不同？把差异分别归因到片上容量、partition 约束和异步引擎的数量。

VI. 编译器与框架

上一节的融合和分块，如果每个模型都要人手写，成本是不可接受的。框架决定了用户怎么写模型，编译器决定了硬件怎么跑它，两者之间的那一层就是本节的主题。我们从计算图和自动微分讲起，接着讨论图级优化（融合、代数替换）、算子级优化（调度空间与搜索），以及 PyTorch 2 在 Python 字节码层面捕获计算图的路线，最后专门讨论面试的高频难点：动态形状。

A. 从 *define-and-run* 到 *define-by-run*

框架的第一个设计选择是：先构图再执行，还是边执行边构图。DistBelief [55] 和 TensorFlow [56] 采用前者，也就是静态图（define-and-run），好处是整张图可以统一优化和做分布式放置。PyTorch [57] 采用后者，即时执行（define-by-run），直接用 Python 控制流表达模型，易用得多，代价是失去了全局优化的机会。JAX [58] 走的是函数变换的路线（`jit`、`grad`、`vmap`、`pmap`），在 tracing 时得到纯函数的中间表示，再交给 XLA 编译 [59]。PyTorch 2 [45] 则想两者兼得：保留 eager 语义，在运行时把图捕获下来再编译。

B. 自动微分

不管用哪种框架，训练都离不开自动微分。深度学习用的是反向模式自动微分 [60]：对标量损失 $\mathcal{L}$ 和 n 个参数，一次反传就能得到全部梯度，代价只是前向的常数倍；前向模式则需要 n 次。对 GEMM 主导的网络，这个常数约为 2，因为反向要对输入和对权重各做一次同样大小的 GEMM。

TABLE IV
代表性 ML 加速器对比（数字取自论文与官方文档；拐点 = 峰值 / 片外带宽）

加速器	计算单元	片上存储	片外存储与峰值	编程模型	面试要点
NVIDIA H100 SXM [15]	132 SM，第四代 tensor core	每 SM 256 KB 寄存器、228 KB shared；50 MB L2	80 GB HBM3，3.35 TB/s；989 TF/s BF16 稠密，拐点 295	CUDA/Triton，SIMT + 异步 TMA/wgmma	通用性最好；硬件 cache 兜底；拐点右移使更多算子访存受限
Google TPU v1 [51]	256 × 256 INT8 systolic，700 MHz	28 MiB 软件管理	8 GiB DDR3，34 GB/s；92 TOPS，拐点约 2700	CISC 式大指令，主机经 PCIe 下发	推理负载多受权重带宽限制；P99 约束限制 batch
Google TPU v4 [52]	MXU systolic 阵列 + SparseCore	软件管理 SRAM	32 GiB HBM2，约 1.2 TB/s；275 TF/s BF16	XLA/GSPMD，sharding 注解	4096 芯片 OCS 可重构 3D torus；SparseCore 服务 embedding
AWS Trainium [54]	128 × 128 Tensor Engine + Vector/Scalar/GpSimd	SBUF 24 MiB（128 partition），PSUM 2 MiB	HBM（见官方文档）	NKI：显式 tile、DMA、partition 维	无 cache 兜底；PSUM 容量限制输出 tile，进而限制强度
MIT Eyeriss [50]	168 个 PE 的空间阵列	全局缓冲 + PE 本地存储	DRAM	配置化数据流	row-stationary 数据流最小化总能耗

训练 FLOPs 的经验公式 $C \approx 6ND$ 就是这么来的：前向 $2ND$，反向 $4ND$。反向模式的代价是必须保存前向的中间激活，训练显存里「激活」这一项就来源于此（第 VII 节）。

本文配套实验（15-849 编译与压缩）用 numpy 实现了一个反向模式自动微分，包括迭代式的拓扑排序、广播求和以及 softmax-交叉熵。在两层 MLP 上，梯度与 `torch` 的最大差 $\le 5.6\times10^{-17}$，与中心差分的最大差 $\le 2.3\times10^{-10}$。它还测试了 5000 层的链式乘法：因为拓扑排序是迭代而不是递归实现的，反向不会爆栈。

面试追问 19：为什么深度学习用反向模式而不是前向模式自动微分？代价是什么？

答：因为输出是一个标量，输入却是 n 维参数。反向模式一次反传就拿到全部梯度，代价约为前向的 2–3 倍；前向模式要跑 n 次（或用 n 维的对偶数）。代价是要保存前向激活，显存随层数和序列长度线性增长，这才引出了重计算和 offload。反过来，如果输出维度远大于输入，比如要求雅可比矩阵的列，前向模式更好。

常见错答：「反向模式更精确」。两者算出的都是精确导数，差别只在复杂度。

C. 图级优化：融合与代数替换

有了计算图，编译器就可以在图上做优化：常量折叠、公共子表达式消除、布局变换和融合。XLA 的 HLO 融合把逐元素算子和归约算子合成一个 kernel；TVM [61] 的 Relay 层按算子类别（injective、reduction、complex-out-fusable）定下融合规则。TASO [43] 更进一步，自动生成并验证图替换规则，比如把共享同一输入的两个矩阵乘合成一个更宽的矩阵乘，再用代价驱动的回溯搜索选出替换序列。融合并不总是划算。**本文配套实验**（15-849 编译与压缩）的结果是，MLP 块的流量降了 49%，时间却只降了 16%（第 IV 节）；而且融合后寄存器压力变大，还可能降低 occupancy。

D. 算子级优化：调度空间与搜索

图切好之后，每个算子内部的循环怎么排，是另一个问题。Halide [62] 提出了影响深远的「算法与调度分离」：算法说明算什么，是一个纯函数定义；调度说明按什么顺序、在哪里、用几个核来算，用的是 `split`、`reorder`、`tile`、`vectorize`、`compute_at` 这些原语。改调度不会改变结果。TVM 把这个思想推广到张量程序，并用学习出来的代价模型指导搜索 [61]。Ansor [63] 连调度模板也不需要人写了：它自动生成层次化的 sketch，再随机注解、做进化搜索，并用代价模型筛选候选。

本文配套实验（15-849 编译与压缩）在 2048^3 的 GEMM 上搭了一个调度空间，由循环顺序和 t_m, t_n, t_k 组成；代价模型是解析的，参数取单个 H100 SM 分到的算力和带宽（7.49 TFLOP/s、25.4 GB/s、228 KB）。在 3072 个调度里，只有 8 个落在最优值的 5% 以内。最优调度是 5.99 ms，也只达到峰值的 38%，原因是 C 块的强度 $t_m t_n/(t_m+t_n) = 128$ 低于拐点 295。搜索到 5% 以内需要评估多少次？随机搜索的中位数是 244、P90 是 649；进化搜索的中位数是 147.5、P90 是 496，大约快 1.65 倍；按枚举顺序的网格搜索是 434，不过这个数只取决于枚举顺序，参考意义不大。

这个实验还给了我一个方法论上的教训。第一版代价模型误用了整颗 CPU 的参数，结果 1080 个调度都落在 5% 以内，搜索问题被模型弄得过于简单。可见代价模型决定了搜索问题有多难，这也是 Ansor 要依赖学习型代价模型和真实测量的原因。

面试追问 20：TVM/Ansor 和 Triton 的定位有什么不同？

答：两者解决的是不同的问题。TVM 把算法和调度分开，靠搜索为每个算子、每种硬件找调度（Ansor 自动生成 sketch、做进化搜索、用学习的代价模型），目标是「不用人写 kernel」。Triton 让人写 block 级的程序，编译器负责 block 内的线程映射、访存合并和 shared memory，目标是「降低手写 kernel 的门槛」。前者是自动调优框架，后者是一门编程语言加编译器。

常见错答：「Triton 是 TVM 的前端」；「Ansor 比 Triton 快」。两者解决的问题不同，没法直接比快慢。

E. PyTorch 2：TorchDynamo、AOTAutograd 与 TorchInductor

回到「既要 eager 的易用，又要编译的性能」这个目标。PyTorch 2 [45] 的 `torch.compile` 由三部分组成。

TorchDynamo 借助 CPython 的帧求值钩子（PEP 523），在字节码层面捕获 FX 图，并为图生成 guard，也就是记录张量形状、dtype 和 Python 对象需要满足的条件。AOTAutograd 在编译期追踪出前向和反向合在一起的联合图，这样反向也能被优化，并在这里决定哪些激活保存、哪些重算。TorchInductor 把图降为 GPU 上的 Triton kernel，或者 CPU 上的 C++/OpenMP 代码。遇到追踪不了的 Python 代码，比如数据相关的控制流、`.item()`、打印、不支持的第三方库，Dynamo 会发生 graph break：断点前后变成两张独立的图，中间回到 Python 解释执行，跨越断点的融合机会也就没了。

面试追问 21：`torch.compile` 什么时候会 graph break？代价是什么？怎么排查？

答：会切图的情况有：数据相关的控制流（`if x.sum()>0`）、把张量转成 Python 标量（`.item()`、`.tolist()`）、打印和日志、不支持的 C 扩展或第三方库。代价是每个断点都要回一次 Python，图被切碎以后跨断点的融合消失，还可能引起更多重编译。排查可以用 `torch._dynamo.explain` 或 `TORCH_LOGS=graph_breaks`；修法是把控制流改写成 `torch.where` 或 `torch.cond`，或者把追踪不了的部分移出编译区域。

常见错答：「compile 之后整个模型就是一张图」。

F. 动态形状

LLM 的序列长度和 batch 每一步都在变，编译器却偏爱静态形状，因为形状决定了分块方式、循环边界和内存规划。主流的处理策略有三种。第一种是 padding 加分桶：把长度向上取整到若干个桶，用少量静态形状覆盖全部输入，这是 XLA/TPU 的传统做法，代价是 padding 上浪费的计算。第二种是符号形状：PyTorch 2 把尺寸表示成符号（SymInt），在 guard 中记录形状之间的约束，只有约束被违反时才重编译 [45]。第三种是运行时专用化：推理引擎为常见的 batch 大小预先捕获 CUDA Graph，其余情况走 eager 路径。面试中要能讲清楚三者在编译次数、浪费的计算和 kernel 质量这三方面各自的取舍。

面试追问 22：动态形状为什么让编译器难做？你会怎么处理变长 batch？

答：因为形状决定了分块、循环边界、内存规划和 kernel 选择，形状一变，静态编译的产物要么失效需要重编译，要么变得次优。训练时，我会按长度分桶并做打包（packing），减少 padding。推理时，decode 的 batch 维分桶并预先捕获 CUDA Graph；prefill 用 varlen attention，按 cu_seqlens 把序列拼接起来，避免 padding。在编译器一侧，用符号形状，并只把真正动态的维度标成动态（`mark_dynamic`），免得每来一个新长度就重编译一次。

常见错答：「把所有输入 pad 到最大长度就行」。这会把 T^2 的 attention 计算浪费在 padding 上。

把这些系统按时间放在一起看（表 V），能看到一条清楚的演化线：最早是人写调度（Halide、TVM 模板），然后是机器搜索调度（Ansor），再到人写 tile 级算法、由编译器负责 tile 内部的映射（Triton），最后是框架自动捕获计算图，并把融合后的子图交给 Triton（PyTorch 2）。GSPMD [53] 和 Alpa [64] 把同样的编译思想用到了分布式并行上，第 VII 节会讲到。

面试要点

- 反向模式 AD：一次反传得到全部梯度，代价约为前向的 2 倍（$6ND$ 的来源），但需要保存激活。
- 图级优化有融合、CSE 和代数替换（TASO）；算子级优化靠算法与调度分离（Halide/TVM）和搜索（Ansor）。
- `torch.compile` 由 Dynamo（字节码捕获 + guard）、AOTAutograd（联合前反向图）和 Inductor（生成 Triton）组成；graph break 会切断融合。
- 动态形状的三种策略：分桶 padding、符号形状、运行时专用化（CUDA Graph）。

研究视角

- 代价模型的可迁移性：在一类 GPU 上训练的代价模型，能不能迁移到新架构上？可以把本文的解析模型当作「先验」，只用少量真实测量做校准，衡量搜索到 5% 以内需要多少次测量。
- 融合决策应该基于 roofline，而不是简单地「能融就融」。一个可做的研究是把每个候选融合组的收益估计为 $\Delta t = \max(F/P, B/\mathrm{BW})$ 的变化量，再与 Inductor 的启发式规则对比。

VII. 分布式训练

模型大到一张卡放不下、或者一张卡要训几年时，就必须把训练拆到很多卡上。拆法有很多种，但每一种都可以看成在式 (1) 的三项之间做交换：ZeRO 和 FSDP 用通信换显存，重计算用计算换显存，梯度分桶和更大的 micro-batch 用显存换通信，流水线则用气泡换显存。看懂了这一点，各种并行策略就不再是一堆要背的名词。本节先建立集合通信的代价模型，再依次讨论数据并行与 ZeRO、张量并行、序列与上下文并行、流水线并行，最后给出 3D 并行的规划方法和几种显存优化手段。

A. 数据并行与参数服务器

最简单的拆法是数据并行（DP）：每张卡都持有完整的模型，各自处理不同的数据，每一步对梯度求平均。早期的 DistBelief [55] 和参数服务器 [65] 把参数分片放在专门的 server 上，worker 推送梯度、拉取参数，可以异步执行，并允许有界的陈旧（bounded staleness）。这种架构至今仍用于推荐模型的稀疏 embedding。稠密模型则转向了 all-reduce。Horovod [66] 把百度的 ring all-reduce 引入 TensorFlow；PyTorch DDP [67] 按反向传播的顺序把梯度分桶（默认约 25 MB），一个桶的梯度一就绪就异步发起 all-reduce，和后面层的反向计算重叠。

面试追问 23：DDP 怎么让通信与反向重叠？bucket 大小怎么取？

答：梯度按参数注册的逆序分桶。反向时，一个桶里的梯度全部就绪就异步发起 all-reduce，前面的层继续算反向，通信就藏在了计算后面。桶太小，延迟项 α 和 kernel 数会变多；桶太大，最后一个桶的通信会暴露在反向结束之后。默认是 25 MB 量级，可以按网络带宽和模型每层的大小调整。另外，`find_unused_parameters=True` 会破坏这种重叠。

常见错答：「等反向全部结束再 all-reduce」；或者以为 DDP 对每个参数单独做一次 all-reduce。

B. 集合通信的 α–β 模型

要判断通信要花多久，需要一个代价模型。最常用的是 α–β 模型：α 是每条消息的固定延迟，β 是每字节的传输

TABLE V
ML 编译器与编程系统对比

系统	输入/抽象	核心思想	优化手段	典型局限
Halide [62]	图像流水线 DSL	算法与调度分离	手写或自动调度（tile、vectorize、compute_at）	领域限于规则 stencil/张量
TVM [61]	Relay 图 + 张量表达式	端到端张量编译，调度原语	图级融合 + 模板化调度 + 学习的代价模型	依赖调度模板；调优耗时
Ansor [63]	张量表达式	无模板的层次化搜索空间	sketch 生成、随机注解、进化搜索、代价模型	搜索时间长；对新硬件需要测量
TASO [43]	算子图	自动生成并形式验证图替换	代价驱动的回溯搜索替换序列	主要是图级，不生成 kernel
XLA [59]	HLO（JAX/TF）	整图编译，静态形状	融合、布局、buffer 复用；GSPMD 自动分片	动态形状需 padding/分桶
PyTorch 2 [45]	Python 字节码	保留 eager 语义的图捕获	Dynamo guard、AOTAutograd、Inductor 生成 Triton	graph break；重编译
Triton [44]	block 级 Python kernel	以 tile 为单位编程	编译器负责线程映射、合并访存、shared memory	需人写算法；跨 block 协作受限

TABLE VI
集合通信算法的 α–β 代价（全互联、单端口全双工）

算法	步数	时间
ring reduce-scatter / all-gather	$p-1$	$(p-1)\alpha+\frac{p-1}{p}S\beta$
ring all-reduce	$2(p-1)$	$2(p-1)\alpha+\frac{2(p-1)}{p}S\beta$
二项树 all-reduce	$2\lceil\log_2 p\rceil$	$2\lceil\log_2 p\rceil(\alpha+S\beta)$
递归减半–倍增	$2\lceil\log_2 p\rceil$	$2\lceil\log_2 p\rceil\alpha+\frac{2(p-1)}{p}S\beta$
naive（root 汇聚 + 广播）	$2(p-1)$	$2(p-1)(\alpha+S\beta)$
all-to-all（成对交换）	$p-1$	$(p-1)\alpha+\frac{p-1}{p}S\beta$

时间（$1/\beta$ 就是单向带宽），消息大小为 S，参与者为 p 个。表 VI 列出了常用算法的代价 [68], [69]。其中最重要的是 ring all-reduce。它分成 reduce-scatter 和 all-gather 两个阶段，各走 $p-1$ 步，每一步每张卡收发 S/p 的数据：

$$t_{\text{ring}} = 2(p-1)\alpha + \frac{2(p-1)}{p}S\beta. \tag{10}$$

这个式子有两个要点。带宽项趋近于 $2S\beta$，几乎与卡数无关，而且已经是带宽最优的 [69]；但延迟项随 p 线性增长。二项树（先 reduce 再 broadcast）正好反过来：延迟项只有 $2\lceil\log_2 p\rceil\alpha$，可每一步都要传完整的消息，带宽项变成 $2\lceil\log_2 p\rceil S\beta$。递归减半–倍增（Rabenseifner 算法）同时做到了 $\log p$ 的步数和最优的带宽项，但对物理拓扑很敏感 [68]。

图 6 把这些公式画了出来。**本文配套实验**（15-849 分布式训练）的模拟器在 $P=8$ 时给出：1 KB 的消息，ring 要 70 µs，树只要 30 µs；消息到约 4.27 MB 时 ring 反超；到 1 GB 时，ring 是 4.25 ms，树是 14.3 ms。递归减半–倍增在交换机拓扑上和 ring 相当（4.21 ms），但在物理环上，由于多条通信争用同一段链路，它慢到 12.56 ms，约为 3 倍。卡数越多，延迟项的差距越明显：$P=64$、1 KB 时，ring 要 630 µs，树只要 60 µs。这就是 NCCL 在大规模集群上引入 tree 和双二叉树算法的原因。实验中唯一的真实测量是 CPU 上的 gloo：world=2 时，1 KB 要 0.40 ms，16 MB 要 25.8 ms，拟合出 $\alpha\approx 181$ µs、等效带宽约 0.97 GB/s。这组数只用来说明数量级。

白板估算 5：7B 模型的梯度 all-reduce 要多久？

BF16 梯度 $S = 2\times 7\times 10^9 = 14$ GB。8 卡 H100 节点内走 NVLink，单向 450 GB/s。由式 (10)，带宽项是 $2\times\frac{7}{8}\times 14\times 10^9/450\times 10^9 \approx 54$ ms（`mlsys_calc.ring_allreduce_time` 给出 54.4 ms）。如果跨节点走每卡 400 Gb/s（50 GB/s）的 InfiniBand，就要约 0.49 s。7B 模型在 8 卡上的单步计算如果只有数百毫秒，那么跨节点时就必须让通信和反向重叠（分桶），或者改用分层 all-reduce：节点内先 reduce-scatter，节点间做 all-reduce，节点内再 all-gather。

面试追问 24：all-reduce 为什么可以拆成 reduce-scatter + all-gather？这有什么用？

答：因为先按分片把和算好（每张卡得到完整和的 $1/p$），再把各个分片广播给所有人，结果和直接 all-reduce 一样。拆开的好处是可以只做其中一半。ZeRO-2 只需要 reduce-scatter，因为每张卡只更新自己负责的那份参数；ZeRO-3 和 FSDP 在前向时用 all-gather 收集权重；序列并行用 RS 加 AG 替代 all-reduce，并在两者之间把激活按序列切开。

常见错答：「all-reduce 是原子操作，不能拆」。

面试追问 25：ring 和 tree all-reduce 怎么选？「环越大越慢」对吗？

答：按消息大小选。ring 的带宽项 $2(p-1)S\beta/p$ 是最优的，而且几乎与 p 无关，但延迟项 $2(p-1)\alpha$ 随 p 线性增长；树的延迟项是 $O(\log p)$，带宽项却差了 $\log p$ 倍。所以小消息（延迟主导）用树，大消息（带宽主导）用 ring。NCCL 会根据消息大小和拓扑自动选择，并对大消息做分块流水。

常见错答：「ring 总是最好」，或者「环越大越慢」。随环变大而变慢的只有延迟项，带宽项不变。

C. ZeRO 与 FSDP

数据并行的问题在显存。混合精度 Adam 的静态显存是每个参数 16 字节：BF16 权重 $2N$、BF16 梯度 $2N$、FP32 主权重 $4N$、Adam 一阶矩 $4N$ 和二阶矩 $4N$，后三项合称优化器状态，共 $12N$ [70]。数据并行让每张卡都存一份完整的 $16N$，冗余非常大。ZeRO [70] 分三级消除这种冗余

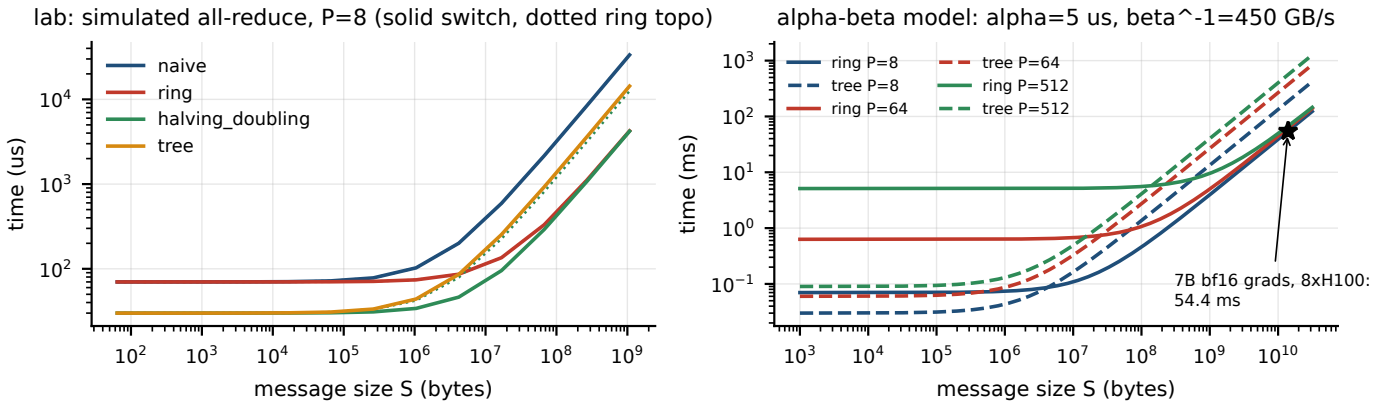


Fig. 6. all-reduce 时间随消息大小的变化。左：本文配套实验（15-849 分布式训练）的集合通信模拟器，$P=8$，$\alpha=5$ µs，$1/\beta=450$ GB/s；实线是交换机（全互联）拓扑，虚线是物理环拓扑。所有算法都真实地搬运 numpy 数组，并逐个 rank 统计字节数，模拟时间与闭式公式一致到 10^{-12}。右：闭式 α–β 模型下，ring 与二项树在 $P=8,64,512$ 时的对比；五角星是 7B 模型的 BF16 梯度（14 GB）在 8 卡 NVLink 上做 ring all-reduce 的时间（54.4 ms）。这张图说明，小消息由延迟项主导，所以树更快；大消息由带宽项主导，所以 ring 更快；两者的交叉点随 α/β 和 P 移动。

TABLE VII
训练静态显存（GB/卡，不含激活；
`MLSYS_CALC.TRAINING_MEMORY_BYTES`）

模型	DP	不切	ZeRO-1	ZeRO-2	ZeRO-3
7B（$N=7\times10^9$）	8	112.0	38.5	26.3	14.0
Llama-3-8B（8.03B）	8	128.5	44.2	30.1	16.1
Llama-3-8B	64	128.5	33.6	17.8	2.0
Llama-3-70B（70.6B）	8	1128.9	388.1	264.6	141.1
Llama-3-70B	64	1128.9	295.4	156.5	17.6

（d 为 DP 的度）：

$$M_{\text{ZeRO-1}}=4N+\frac{12N}{d},\qquad M_{\text{ZeRO-2}}=2N+\frac{14N}{d},\qquad M_{\text{ZeRO-3}}=\frac{16N}{d}. \tag{11}$$

ZeRO-1 只切优化器状态；ZeRO-2 再切梯度，用 reduce-scatter 代替 all-reduce；ZeRO-3 连权重也切，代价是前向和反向各要 all-gather 一次权重。通信量要仔细算。DP 的 all-reduce 等价于 RS 加 AG，每张卡约传 2Ψ（Ψ 是参数的字节数），ZeRO-1 和 ZeRO-2 与它相同；ZeRO-3 要做前向 AG、反向 AG 和梯度 RS，共 3Ψ，是 DP 的 1.5 倍。PyTorch FSDP [71] 是 ZeRO-3 的工程实现：它以模块为单位（FSDP unit）分片和 all-gather，预取下一个 unit 的参数来重叠通信，还支持节点内分片、节点间复制的混合分片（HSDP），以减少跨节点的流量。

白板估算 6：Llama-3-8B 与 70B 的训练显存

8B：$16N=16\times8.03\times10^9\approx128.5$ GB，单卡放不下。8 卡 ZeRO-1 是 $4N+12N/8=32.1+12.0=44.2$ GB，ZeRO-3 是 16.1 GB（表 VII）。70B：$16N\approx1.13$ TB。即使 64 卡做 ZeRO-2，每卡仍需 156.5 GB，因为每张卡都保留了完整的 BF16 权重（141 GB）；只有 ZeRO-3 或 TP/PP 能把权重本身切开。激活要另算。按 Korthikanti 等人的估计 [72]，用 FlashAttention 时每层约 $34sbh$ 字节（s 是序列长度，b 是 micro-batch，h 是隐藏维）。70B（$h=8192$，80 层）一条 4096 的序列约 91 GB；做全量重计算后每层只保留输入 $2sbh$，约 5.4 GB。完整的分解见图 7。

这些公式在配套实验里得到了验证。**本文配套实验**（15-849 分布式训练）在 4 个模拟 rank 上，用显式的 reduce-scatter 和 all-gather 实现了 ZeRO 0–3（FP64、分片 Adam）。训练 20 步后，参数和单进程训练的最大差是 2.9×10^{-16}，说明它们在数学上等价。每个 rank 的常驻字节分别是 DP 的 1、0.625、0.4375、0.25。这里是 FP64，params:grads:m:v=1:1:1:1，理论比例是 1、$(2+2/P)/4$、$(1+3/P)/4$、$1/P$，与实测吻合；ZeRO-3 的通信量也恰好是 DP 的 1.5 倍。换成论文的混合精度口径，$\Psi=7.5$B、$N_d=64$ 时计算器给出 120 / 31.4 / 16.6 / 1.88 GB，与 ZeRO 论文的图 1 一致 [70]。

面试追问 26：ZeRO 三个 stage 分别切了什么？通信量变不变？

答：stage 1 切优化器状态，stage 2 再切梯度，stage 3 再切权重。stage 1 和 2 的通信量与 DP 相同，因为 all-reduce 本来就等于 RS 加 AG，stage 2 只是把 AG 挪到了参数更新之后；stage 3 在前向和反向各多一次权重 all-gather，是 DP 的 1.5 倍。以 7B、8 卡为例，静态显存依次是 112、38.5、26.3、14 GB。

常见错答：「ZeRO-3 的通信是 DP 的 3 倍」；或者「ZeRO 改变了数学，会影响收敛」。它和 DP 在数学上等价。

D. 张量并行（*Megatron-LM*）

ZeRO-3 切的是存储，计算时还要把权重收集回来。另一种思路是连计算一起切：Megatron-LM [73] 把 Transformer 的每一层沿隐藏维切到 t 张卡上。以 MLP $Y=\mathrm{GeLU}(XW_1)W_2$ 为例：W_1 按列切，每张卡得到 GeLU 输入的一部分列；GeLU 是逐元素的，可以就地算；W_2 按行切，正好消费这部分列，得到部分和，最后做一次 all-reduce 求和。attention 按头切，W_O 按行切，同样只需一次 all-reduce。用一对共轭算子来描述：f 前向是恒等、反向是 all-reduce，g 前向是 all-reduce、反向是恒等。这样每层前向 2 次、反向 2 次 all-reduce，每次的消息是 $b\cdot s\cdot h$ 个元素，与 t 无关。

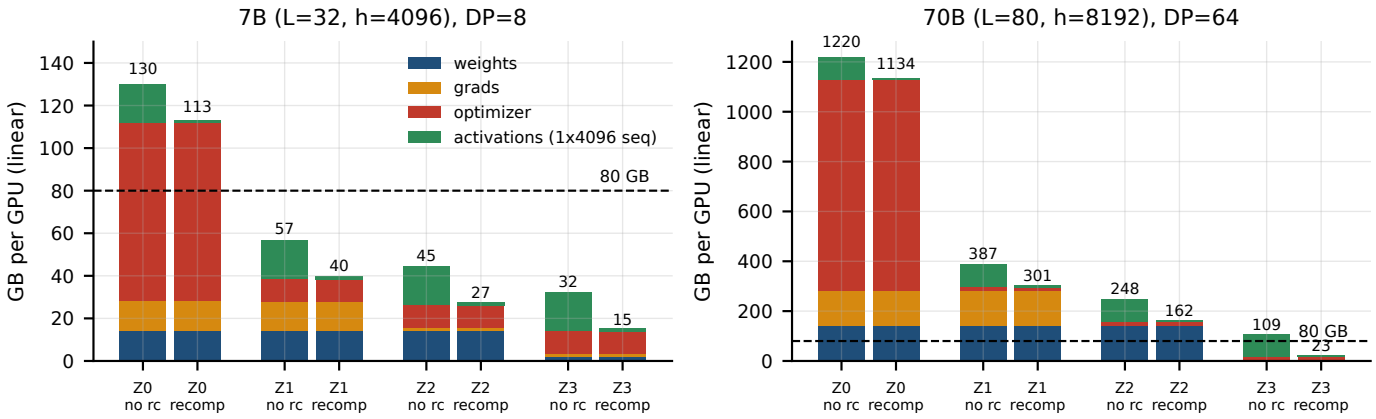


Fig. 7. 每卡训练显存的分解：权重、梯度、优化器状态和激活（micro-batch 为 1 条 4096 token 的序列，按 Korthikanti 等人的 $34sbh$/层估计；recomp 表示全量重计算，每层只保留 $2sbh$）。左：7B（Llama-2 形状），DP=8；右：70B（Llama-3 形状），DP=64。虚线是 80 GB。7B 做 ZeRO-1 加重计算后就能放进单卡（40 GB）；70B 只有 ZeRO-3 才放得下，而且这时激活反而成了最大的一项（不重计算时 109 GB），下一步要优化的是激活。激活公式针对 GPT 式的 $4h$ MLP，对 SwiGLU 只是近似。

白板估算 7：张量并行每步要传多少？

以 70B 为例：$h = 8192$，每张卡一步处理 $b \cdot s = 8192$ 个 token，BF16 下每次 all-reduce 的消息是 $8192 \times 8192 \times 2 \approx 134$ MB；每层 4 次，80 层合计约 43 GB/卡/步。ring 因子 $2(t-1)/t = 1.75$，走 450 GB/s 的 NVLink 约 0.17 s，走 50 GB/s 的 IB 约 1.5 s。这些通信都在关键路径上，很难和计算重叠，所以 TP 要留在节点内。**本文配套实验**（15-849 分布式训练）对 Llama-3-8B（batch 1、$s = 8192$、$t = 8$）的模型值是每次 67.1 MB、0.331 ms，32 层每步 42.4 ms；TP 切分前后的前向输出、输入梯度和参数梯度的最大差 $\leq 8.9 \times 10^{-16}$。

面试追问 27：Megatron 的 MLP 为什么第一层按列切、第二层按行切？

答：因为这样中间不需要通信。列切以后每张卡得到 XW_1 的一部分列，GeLU 是逐元素的，可以就地算；行切的 W_2 正好消费这部分列，输出部分和，最后一次 all-reduce 就够了，整个 MLP 前向只通信一次。如果两层都按列切，第二层需要完整的输入，中间就得先做一次 all-gather，通信更多。

常见错答：「两层都按列切更省通信」。

面试追问 28：为什么张量并行不出节点，而流水线并行可以跨节点？

答：因为两者的通信量和能否重叠差别很大。TP 每层前反向共 4 次 $b \cdot s \cdot h$ 的 all-reduce，都在关键路径上，很难重叠，必须走 NVLink（450 GB/s）。PP 只在 stage 边界做点对点传输，每个 micro-batch 传一次 $b \cdot s \cdot h$ 的激活，量小，而且可以和计算重叠，走 IB（50 GB/s，约为 NVLink 的 1/9）也能接受。

常见错答：「随便放」，或者「TP 通信量小，所以可以跨节点」。

E. 序列并行与上下文并行

TP 切开了权重和大部分激活，但 LayerNorm 和 dropout 的激活没有被切，序列长了以后这部分很可观。Korthikanti 等人的序列并行（SP） [72] 在这些区域沿序列维切分，把 TP 的 all-reduce 换成 reduce-scatter 加 all-gather：通信量不变，激活却按 t 切开了。序列再长下去，单卡连 attention 的输入都放不下，就需要上下文并行（CP）。Ring Attention [74] 把序列切到 c 张卡上，K/V 块沿环传递，每张卡用 online softmax（式 (8)）累加结果，通信和块计算重叠。DeepSpeed-Ulysses [75] 换了一种做法：在 attention 前后各做一次 all-to-all，把「按序列切」转换成「按头切」。它的通信量与 c 无关，但并行度受头数限制。

F. 流水线并行

TP 被限制在节点内，模型更大时就需要跨节点切分，流水线并行（PP）正好适合。它把层切成 p 个 stage，把一个 mini-batch 切成 m 个 micro-batch。GPipe [76] 先做完全部前向，再做全部反向；PipeDream [77] 的 1F1B 调度在稳态中交替做一个前向和一个反向。两种调度的填充和排空都各需要 $p-1$ 个时隙，所以气泡比例是一样的：

$$\text{bubble} = \frac{p-1}{m+p-1}, \qquad \text{bubble}_{\text{interleaved}} = \frac{(p-1)/v}{m+(p-1)/v}, \tag{12}$$

其中交错调度（interleaved 1F1B）让每张卡负责 v 个不相邻的虚拟 stage [78]，气泡缩小到 $1/v$，代价是点对点通信变成 v 倍。1F1B 真正的收益在*激活显存*上：同时在途的 micro-batch 从 m 个降到最多 p 个。

白板估算 8：8 级流水线、32 个 micro-batch 的气泡

$p = 8$、$m = 32$ 时气泡是 $7/39 \approx 17.9\%$；$m = 8$ 时升到 46.7%，所以经验上取 $m \geq 4p$。交错 $v = 2$ 时降到 9.9%，$v = 4$ 时为 5.2%（`mlsys_calc.pipeline_bubble`）。配套模拟中，$p = 4, m = 8$ 时 GPipe 和 1F1B 的气泡都是 0.273，交错 $v = 2$ 是 0.158；但 0 号卡的在途激活，GPipe、1F1B、交错分别是 8、4、5.5 份。可见气泡和激活显存之间确实存在权衡（图 8）。

面试追问 29：1F1B 比 GPipe 好在哪？它减少了气泡吗？

答：没有减少气泡。stage 均匀时两者的气泡完全相同，都是 $(p-1)/(m+p-1)$。1F1B 的好处在显存：它在稳态中每做一个前向就做一个反向，一个 micro-batch 的反向一做完就释放它的激活，在途激活从 m 份降到 p 份。显存省下来，就可以用更大的 m，从而间接降低气泡。真正直接减小气泡的是交错调度（除以 v），代价是 v 倍的点对点通信和更多的在途激活。

常见错答：「1F1B 的气泡比 GPipe 小」。

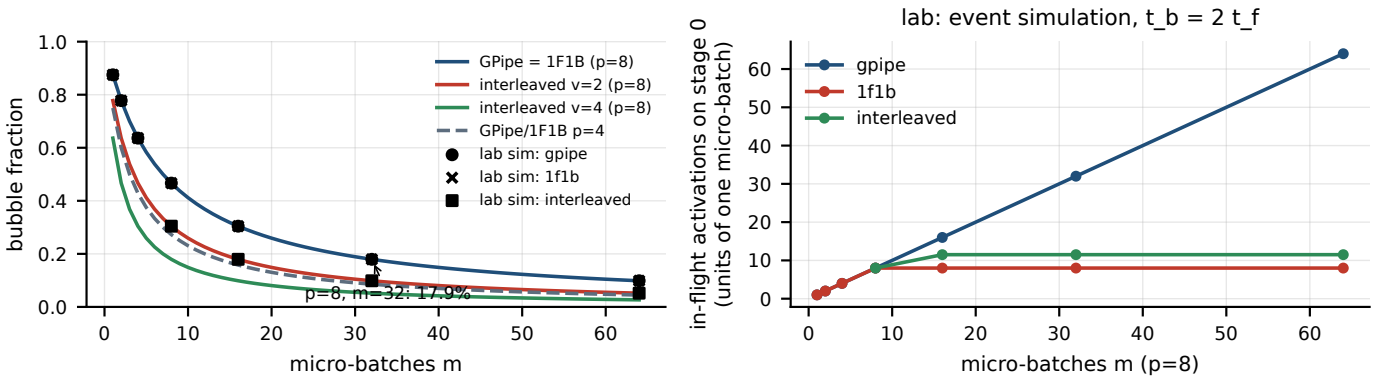


Fig. 8. 流水线气泡与在途激活。左：式 (12) 的气泡比例随 micro-batch 数 m 的变化（$p=8$，$v=1,2,4$；虚线为 $p=4$），黑色标记是配套实验离散事件模拟的结果（$t_b=2t_f$），与公式一致到 10^{-12}。$p=8, m=32$ 时气泡为 17.9%。右：0 号 stage 的峰值在途激活。GPipe 随 m 线性增长，1F1B 封顶于 $p=8$；交错调度（$v=2$）让气泡减半，在途激活却升到 11.5 份，比 1F1B 还多。两张图放在一起看，就能看出气泡和激活显存是一对此消彼长的量。

G. 3D 并行的规划

各种并行方式要组合起来用，这就是 3D 并行。表 VIII 比较了各类策略切分的对象、通信模式和适用范围。Narayanan 等人 [78] 总结的经验可以整理成一个能复述的流程：

1) TP 的度 t 取节点内的 GPU 数（通常是 8），因为 TP 的通信量最大，而且在关键路径上；
2) PP 的度 p 取能放下模型的最小值，因为 stage 越多，气泡越大；
3) 剩下的维度给 DP，配上 ZeRO-1 和梯度分桶重叠；
4) 取 $m \geq 4p$ 来控制气泡，同时检查 global batch 有没有超过收敛允许的临界 batch；
5) 序列很长时再加 SP 或 CP，MoE 模型再加专家并行（第 IX 节）。

也有工作尝试让机器来做这个规划。Alpa [64] 把算子内并行（TP/DP）和算子间并行（PP）分层求解；GSPMD [53] 通过 sharding 注解让编译器自动插入通信。它们是「自动并行」的代表。

白板估算 9：70B 模型预训练 15T token 需要多少卡、多少天？

总算力 $C = 6ND = 6 \times 70.6 \times 10^9 \times 15 \times 10^{12} \approx 6.35 \times 10^{24}$ FLOP。用 H100、MFU 40%，1024 张卡需要 $6.35\times10^{24}/(1024\times989\times10^{12}\times0.4) \approx 1.57\times10^7$ s，约 181 天；换成 8192 张卡约 23 天（图 15 右）。再看显存：TP=8（节点内）乘 PP=4，每卡的静态显存是 $16N/32 \approx 35$ GB；再加上 DP=256 的 ZeRO-1，降到 $4N/32+12N/(32\times256) \approx 8.9$ GB，给激活留出了空间。最后是通信：DP 的梯度每卡分片 $2N/32 \approx 4.4$ GB，在 IB 上做 ring all-reduce 约 $2\times4.4/50 \approx 0.18$ s，可以藏在反向计算里。

面试追问 30：70B 训练为什么不直接用 FSDP/ZeRO-3，而用 TP+PP+DP？

答：两种方案都可行，取舍在于通信发生在哪里。ZeRO-3 每层的前向和反向都要 all-gather 权重，跨 IB 的总通信是 DP 的 1.5 倍；TP+PP 把每层的大通信留在 NVLink 内，跨节点的只有 PP 的点对点传输和 DP 的梯度分片。所以规模越大、网络越慢，TP+PP 越有优势。FSDP 的优点是不用改模型代码、调度简单，在中等规模（数百卡）下常常就够用了。

常见错答：「ZeRO-3 最省显存，所以总是最好」。

H. 激活重计算

并行策略解决了权重和优化器状态，剩下的大头是激活。重计算的思路是：前向时不存全部激活，反向需要时再算一遍。Chen 等人 [81] 提出每 k 层存一个检查点，显存约为 $n/k+k$，取 $k=\sqrt{n}$ 时最小，约 $2\sqrt{n}$；额外计算约为一次前向，总 FLOPs 从 $3F$ 变成 $4F$，即 +33%。Checkmate [82] 把重计算建模成混合整数线性规划，在给定显存预算下求最优的保存与重算方案。Korthikanti 等人 [72] 提出了选择性重计算：只重算 attention 中占显存大、但计算便宜的部分（softmax、dropout mask），开销只有几个百分点。

本文配套实验（15-849 分布式训练）实现了链状网络上的 $\sqrt{n}$ 方案和带预算约束的动态规划（图 5 右）。对均匀的 64 层，$\sqrt{n}$ 方案（每 8 层一段）峰值是 15、重算是 56，已经在 Pareto 前沿上。但层不均匀时情况就不同了：异构的 32 层（每层大小和代价在 $[1,8]$ 内），在同样为 53 的预算下，$\sqrt{n}$ 要重算 117，动态规划只要 85，少了 27%。在 torch 版的 16 层实验中，4 层一段（$=\sqrt{16}$）的峰值最低，梯度与不做重计算时逐位相等。CPU 上的时间开销是 1.52 倍，远高于理论上的 1.33 倍，多出来的部分是 Python 钩子的开销，不能外推到 GPU。

面试追问 31：激活重计算多花多少算力？

答：全量重计算多花三分之一，不是翻倍。它多做一次前向，而前向和反向的比例是 1 : 2，所以总计算从 $3F$ 变成 $4F$，即 +33%。按 $\sqrt{n}$ 设检查点时，激活从 $O(n)$ 降到 $O(\sqrt{n})$。选择性重计算只重算 softmax、dropout 等，开销通常 $<5\%$。另外，报告 MFU 时不计重算的 FLOPs，HFU 才计入。

常见错答：「重计算让训练慢一倍」。

I. 混合精度、*offload* 与 *FP8*

降低精度既省显存又提速，但要守住数值稳定。Micikevicius 等人 [83] 的混合精度训练确立了三条做法：用 FP16 计算，保留 FP32 主权重，做动态 loss scaling。BF16 [84] 有 8 位指数，动态范围和 FP32 相同，通常不需要 loss scaling，代价是只有 7 位尾数。无论用哪种格式，求和、softmax、LayerNorm 的统计量和 loss 都应该在 FP32 中累加。

显存还不够时，可以把一部分状态挪到 CPU。ZeRO-Offload [85] 把优化器状态和 Adam 更新放到 CPU 上，GPU

TABLE VIII
并行策略对比（N 为参数量，s 为每元素字节数，$b \cdot s_{\text{SEQ}} \cdot h$ 为一个 MICRO-BATCH 的激活张量）

策略	切分对象	每步通信	显存收益	适用与代价
DP [67]	数据	梯度 all-reduce，$\approx 2Ns$/卡	无（每卡完整 $16N$）	最简单；受 global batch 与显存限制
ZeRO-1/2 [70]	优化器状态/梯度	同 DP（RS+AG）	$4N + 12N/d$ / $2N + 14N/d$	几乎免费的显存收益，默认开启
ZeRO-3 / FSDP [70], [71]	参数、梯度、状态	1.5× DP（AG+AG+RS）	$16N/d$	模型放不下且不愿改模型代码；跨 IB 时通信重
TP [73]	层内权重（隐藏维/头）	每层 4 次 $bs_{\text{seq}}h$ all-reduce	权重、梯度、部分激活 $/t$	节点内（NVLink）；t 受头数与效率限制
SP [72]	LayerNorm/dropout 激活（序列维）	与 TP 相同（RS+AG 替代 AR）	剩余激活 $/t$	与 TP 配合，几乎无额外代价
CP [74], [75]	序列（attention）	ring 传 K/V 或 2 次 all-to-all	激活与 attention 输入 $/c$	长上下文训练/prefill
PP [76], [77]	层（stage）	stage 边界点对点 $bs_{\text{seq}}h$	权重与状态 $/p$	可跨节点；气泡 $(p-1)/(m+p-1)$
EP [79], [80]	MoE 专家	每层 2 次 all-to-all（前向）	专家参数 $/e$	MoE；负载均衡与 all-to-all 带宽

只做前反向；ZeRO-Infinity [86] 进一步用上了 NVMe。它们的瓶颈是 PCIe：7B 模型每一步要向主机传 $2N = 14$ GB 梯度，再传回 14 GB 权重，在 32 GB/s 的 PCIe 4.0 上各要约 0.44 s。所以 offload 只有在单步计算足够长（batch 很大）时才划算。

FP8 是更激进的一步。它有 E4M3 和 E5M2 两种格式 [87]：E4M3 精度更高，用于前向的权重和激活；E5M2 范围更大，用于梯度。H100 上 FP8 的算力是 BF16 的 2 倍。由于动态范围窄，FP8 需要按张量（或更细的粒度）缩放，常用的是 delayed scaling，即用前若干步的 amax 历史来估计当前的 scale。DeepSeek-V3 报告了用细粒度（分块）缩放做 FP8 训练 [88]。主要风险在离群值和累加精度上，主权重和优化器状态仍然保持高精度。

面试追问 32：BF16 和 FP16 训练的区别？FP8 训练要注意什么？

答：BF16 范围大、精度低，FP16 正好相反。BF16 有 8 位指数、7 位尾数，范围与 FP32 相同，不需要 loss scaling；FP16 有 5 位指数、10 位尾数，精度更高，但梯度容易下溢，必须做动态 loss scaling。两者都需要 FP32 主权重和 FP32 累加。FP8 需要 per-tensor 或 per-block 的 scale（delayed scaling 用 amax 历史），前向用 E4M3，梯度用 E5M2；要注意激活的离群值、GEMM 的累加精度，以及哪些算子（LayerNorm、softmax、loss）必须留在高精度。

常见错答：「BF16 精度比 FP16 高」；「FP8 训练就是把所有张量都转成 8 位」。

面试要点

- ring all-reduce 的时间是 $2(p-1)\alpha + 2(p-1)S\beta/p$；小消息用树，大消息用 ring；7B 梯度在 8 卡 NVLink 上约 54 ms。
- $16N$：7B 单卡要 112 GB；ZeRO-1/2/3 的公式见式 (11)；ZeRO-3 的通信是 DP 的 1.5 倍。
- Megatron：先列切再行切，每层前反向共 4 次 bsh 的 all-reduce，所以留在节点内。
- 气泡是 $(p-1)/(m+p-1)$；1F1B 省激活，不省气泡；交错调度把气泡除以 v。
- 全量重计算 +33% FLOPs，选择性重计算 $< 5\%$；BF16 不需要 loss scaling；FP8 需要管理 scale。

研究视角

- 3D 并行的自动规划（Alpa、GSPMD）通常基于静态代价模型，而真实集群的带宽会随拓扑和拥塞变化。可以做的实验是：在 α–β 模型中加入拓扑项（交换机与物理环；本文实验显示 halving-doubling 在物理环上慢 3 倍），看最优并行配置会不会改变。
- 流水线调度的「零气泡」方向 [89] 把反向拆成对输入求导和对权重求导两部分，分别调度。它和激活显存的关系值得系统测量：在本文的事件模拟器中把 t_b 拆成 $t_{b,x} + t_{b,w}$，比较气泡和峰值激活的 Pareto 前沿。

VIII. LLM 推理与服务

2023 年以来，LLM 推理是面试中权重上升最快的方向。它和训练最大的区别在于自回归的 decode：每生成一个 token，都要把全部权重和这条序列的全部 KV cache 读一遍，计算量却只有 $2N$ FLOPs。所以推理的很多问题，归根到底是「每生成一个 token 要读多少字节」的问题。本节先用 roofline 区分 prefill 和 decode，再推导 KV cache 的大小，然后讨论批处理与调度（continuous batching、PagedAttention、chunked prefill、PD 分离、前缀缓存），最后是推测解码和量化。

A. *Prefill* 与 *decode* 的 *roofline*

一次请求分两个阶段。prefill 一次处理整段 prompt，线性层是 $[B \cdot T_p, d] \times [d, d]$ 的 GEMM，强度很高，受算力限制，它决定了首 token 时延（TTFT）。decode 每步每条序列只处理一个 token，线性层退化成 GEMV（batch 为 B 时是 $M = B$ 的瘦 GEMM），受带宽限制，它决定了每 token 时延（TPOT）。Pope 等人对此做过系统的分析 [90]。忽略非矩阵运算，一个 decode 步的 roofline 下界是

$$t_{\text{dec}} \approx \max\left(\frac{2NB + 4LdTB}{P_{\text{peak}}},\ \frac{sN + \text{KV}(T,B)}{\text{BW}}\right), \quad (13)$$

其中 $4LdTB$ 是 attention 对长度为 T 的上下文做点积和加权求和的运算量，$\text{KV}(T,B)$ 是 KV cache 的字节数（式 (14)）。如果只看权重，强度 $I \approx 2NB/(sN) = B$（BF16），也就是说临界 *batch* 约等于拐点，H100 上 $B^* \approx 295$。但 KV 的读取不能被 batch 分摊，每条序列

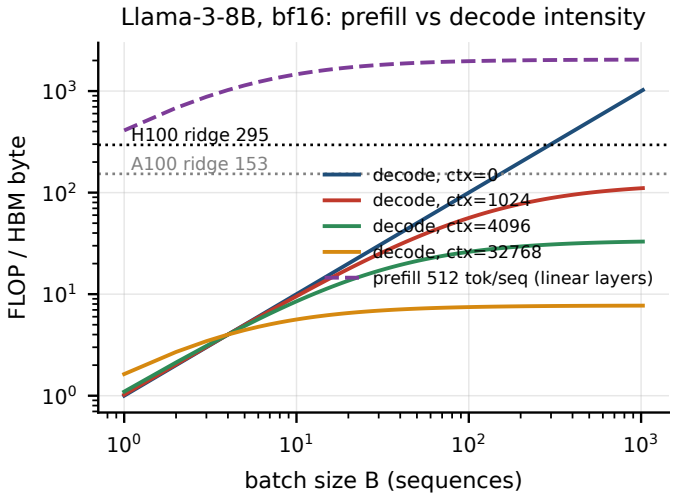


Fig. 9. Llama-3-8B 在 BF16 下 prefill 与 decode 的算术强度随 batch 的变化（`mlsys_calc.decode_step_time`）。不计 KV（ctx=0）时，decode 强度等于 B，在 $B \approx 295$ 处越过 H100 的拐点；计入 KV 后，强度会饱和在一个由上下文长度决定的常数上，因为 KV 字节随 batch 同步增长。prefill 的线性层在任何 batch 下都受算力限制。

都要读自己的 KV，所以长上下文下它会先成为瓶颈。图 9 显示了这一点：上下文为 4096 时，Llama-3-8B 的 decode 强度在大 batch 下趋于约 30；上下文到 32K 时只有约 7，永远到不了拐点。相比之下，prefill（每条序列 512 token）的线性层强度在 batch 1 时就已经超过 400。

强度饱和意味着吞吐也会饱和。图 10 把强度换算成了 decode 吞吐的上界。

白板估算 10：H100 上 Llama-3-8B 的 decode 上限

权重 $8.03 \times 10^9 \times 2 \approx 16.06$ GB。batch=1、上下文 4096 时，KV 是 0.54 GB，每步读 16.6 GB，$t \geq 16.6/3.35 \approx 4.95$ ms，约 202 token/s；而计算只有 $2N \approx 16$ GFLOP，只用到峰值的 0.3%。batch=64 时，KV 是 34.4 GB，每步 15.05 ms，约 4252 token/s，仍然受访存限制。batch=256 时，KV 达到 137 GB，单卡放不下（`inference_math.csv` 与配套实验 CS229S P2 一致）。作为对比，看短上下文的 7B 模型在 batch=512 时的情况：读权重 14 GB 需要 4.2 ms，计算 $2\times7\times10^9\times512/989\times10^{12} \approx 7.2$ ms，已经受算力限制了。RL rollout 这类大 batch 场景里推测解码收益消失，原因就在这里。

面试追问 33：prefill 和 decode 的瓶颈分别是什么？这对系统设计意味着什么？

答： prefill 是 GEMM，受算力限制，决定 TTFT 和算力需求；decode 是 GEMV 或瘦 GEMM，受带宽限制，因为要读全部权重和 KV，它决定 TPOT 以及对显存带宽和容量的需求。两者混在一起跑会互相干扰：一个长 prefill 会让同批 decode 的 TPOT 抖动。由此引出了两种设计：chunked prefill 把 prefill 切块混进 decode 步；PD 分离把两者部署到不同的池子里，各自按 TTFT 和 TPOT 选择并行度。

常见错答：「decode 慢是因为计算量大」。

B. KV cache 的大小

既然 KV 会成为瓶颈，就得会算它有多大。每一层、每个 token 都要存一个 K 向量和一个 V 向量，长度都是 $n_{kv} \cdot d_{\text{head}}$，所以

$$\text{KV}(T, B) = 2 \cdot L \cdot n_{kv} \cdot d_{\text{head}} \cdot T \cdot B \cdot s. \tag{14}$$

TABLE IX
KV CACHE 大小（BF16，BATCH=1，单位 GIB；`MLSYS_CALC.KV_CACHE_BYTES`）

模型	n_{kv}	KiB/tok	8K	32K	128K
Llama-2-7B（MHA）	32	512	4.0	16.0	64.0
Llama-3-8B（GQA-8）	8	128	1.0	4.0	16.0
Llama-3-70B（GQA-8）	8	320	2.5	10.0	40.0
70B 若为 MHA（反事实）	64	2560	20.0	80.0	320.0

代入几个常见模型：Llama-3-8B（$L = 32$、$n_{kv} = 8$、$d_{\text{head}} = 128$）是 128 KiB/token；Llama-3-70B（$L = 80$、$n_{kv} = 8$）是 320 KiB/token；没有用 GQA 的 Llama-2-7B（$n_{kv} = 32$）是 512 KiB/token [2], [91]。不同上下文长度下的数值见表 IX 和图 11。

白板估算 11：8K 与 128K 上下文的 KV cache

Llama-3-8B：$2 \times 32 \times 8 \times 128 \times 2 = 131072$ B=128 KiB/token。8K 的一条序列是 1 GiB，batch 32 是 32 GiB（34 GB）；128K 的一条序列是 16 GiB。Llama-3-70B：320 KiB/token，8K 是 2.5 GiB，128K 是 40 GiB，一条 128K 的序列就要占半张 H100。同样条件下，Llama-2-7B 在 8K、batch 32 时需要 128 GiB。到了 128K，decode 的瓶颈已经从「读权重」变成「读 KV」。KV 量化、GQA/MLA 和上下文并行都是从这里出发的。

面试追问 34：KV cache 为什么能省计算？它本身的代价是什么？

答： 它省掉的是对前缀的重复计算。不缓存时，每一步都要为全部前缀重算 K、V 投影，生成 T 个 token 总共要 $O(T^2)$ 次线性层；缓存之后每步只算新 token，总共 $O(T)$。配套实验（CS229S P2，CPU）中，$T = 1024$ 时 KV cache 把生成速度从 38.9 提到 499 token/s，快了 12.8 倍，总耗时也从 $\sim T^2$ 变成了 $\sim T$。代价是显存 $2Ln_{kv}d_{\text{head}}TBs$，它决定了最大并发和最长上下文；而且每一步仍要读全部 KV 做点积，attention 的读取量随 T 线性增长。

常见错答：「KV cache 减少了 attention 的计算」；或者计算 KV 大小时用了 n_{heads} 而不是 n_{kv}（GQA 下会差 4–8 倍）。

C. MQA、GQA 与 MLA

既然 KV 的大小正比于 n_{kv}，最直接的办法就是减少 KV 头。MQA [92] 让所有查询头共享一组 K/V，KV 缩小 n_{heads} 倍，但有质量损失。GQA [93] 是折中：每组查询头共享一组 K/V，在质量和 KV 大小之间取平衡，而且可以从 MHA 的 checkpoint 经过少量 uptraining 得到。二者都不改变 attention 的 FLOPs，因为每个查询头仍要和 T 个 key 做点积；它们减少的是 KV 的存储和读取，所以 decode attention 的强度提高了 n_{heads}/n_{kv} 倍。DeepSeek-V2 的 MLA [94] 走得更远：它缓存一个低秩压缩的 latent（512 维）加一个解耦的 RoPE key（64 维），推理时把上投影矩阵吸收进查询和输出投影。论文报告其 KV cache 比 DeepSeek 67B 减少了 93.3%。

面试追问 35：GQA、MLA、KV INT8 量化各自省了什么、代价是什么？

答： 三者都在减少 KV 的字节，方式不同，而且可以叠加。GQA 减少 KV 头数，KV 和 decode attention 的读取量都减少 n_{heads}/n_{kv} 倍；它改变了模型结构，需要从训练开始用，或

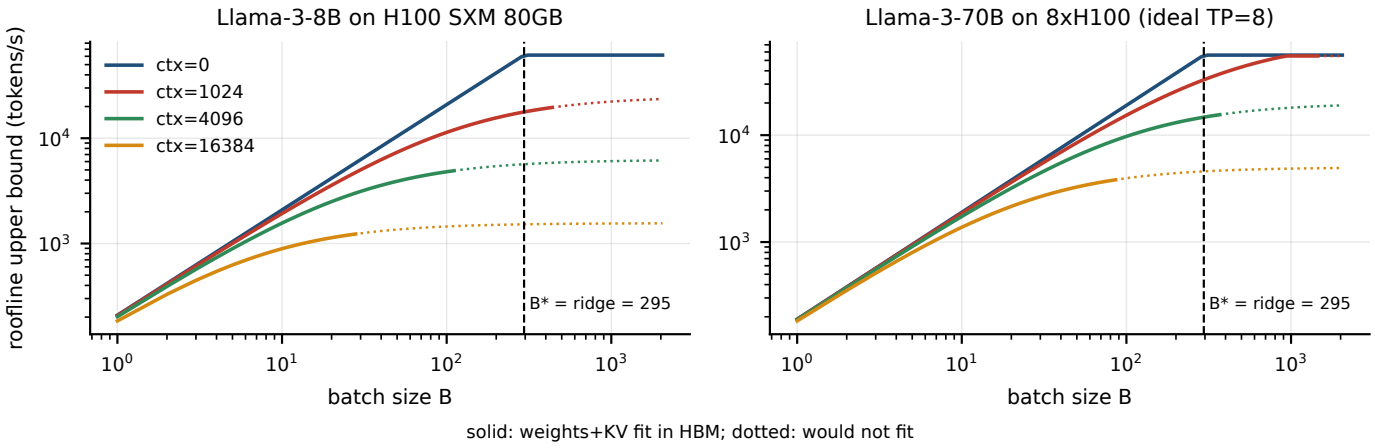


Fig. 10. decode 吞吐的 roofline 上界随 batch 的变化。左：Llama-3-8B，单张 H100；右：Llama-3-70B，理想 TP=8 的 8 卡 H100（聚合算力与带宽，忽略 all-reduce）。实线表示权重和 KV 能放进 HBM，虚线表示放不下。ctx=0 的曲线在 $B^* = 295$ 处出现从访存受限到算力受限的拐点；上下文越长，曲线越早饱和，而且还没到拐点就因为 KV 放不下而终止。也就是说，长上下文下限制吞吐的是 KV 的带宽和容量，而不是算力。

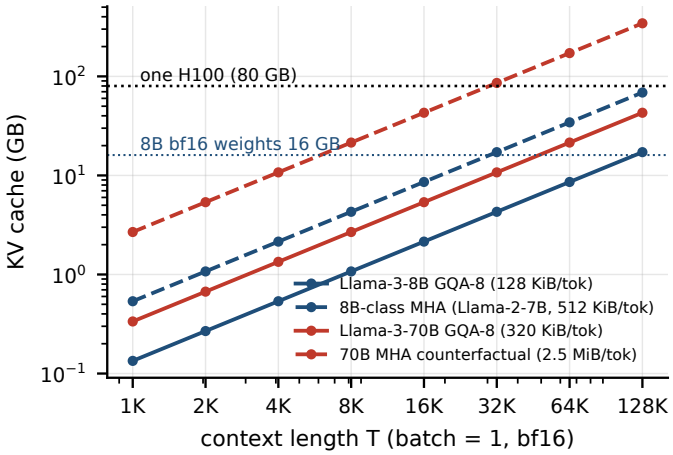


Fig. 11. KV cache 随上下文长度的增长（batch=1，BF16）。实线是采用 GQA-8 的真实模型，虚线是同规模的 MHA。Llama-3-8B 在 128K 时 KV 为 16 GiB，已经和它 16 GB 的权重相当；70B 如果是 MHA，32K 上下文的一条序列就要 80 GiB，占满一张 H100。GQA 把 KV 缩小了 n_{heads}/n_{kv} 倍（8B 是 4 倍，70B 是 8 倍），同样的显存因此能放下长得多的上下文。

者做 uptraining。MLA 缓存低秩 latent，压缩比更高，但实现复杂：RoPE 要解耦，还需要能吸收投影矩阵的 kernel。KV INT8/FP8 量化不需要重训，KV 直接减半，代价是轻微的精度损失和反量化开销。

常见错答：「GQA 减少了 attention 的 FLOPs」。

D. 批处理与调度

decode 受带宽限制，提高吞吐最有效的办法就是把更多请求拼进同一个 batch，让一次权重读取服务更多 token。下面五项技术都在回答同一个问题：怎样在显存和时延约束下把 batch 做大、做稳。

第一项是 continuous batching。传统的静态批处理要等一批里最长的请求生成完，才能换下一批。Orca [4] 提出了 iteration-level 调度：每个 decode 步都可以让完成的请求离开、让新请求加入；同时用 selective batching，只对线性层拼批，attention 则按请求分别计算。

第二项是 PagedAttention。如果按最大长度给每条序列连续预分配 KV，会产生大量内部碎片和预留浪费。vLLM [3] 借鉴了操作系统的分页机制：KV 按固定大小的块分配（默认 16 个 token 一块），用块表把逻辑块映射到物理块，浪费只发生在最后一块；并行采样和 beam search 还可以通过引用计数和 copy-on-write 共享前缀块。论文报告的吞吐比当时的系统高 2–4 倍。

第三项是 chunked prefill。Sarathi-Serve [5], [95] 把长 prompt 切成固定预算的块，和 decode 拼在同一步里执行（stall-free batching）。decode 本来就受访存限制，捎带进去的 prefill 计算几乎是免费的；同时，长 prefill 再也不会让 decode「卡顿」。

第四项是 PD 分离。DistServe [6] 和 Splitwise [12] 把 prefill 和 decode 放到不同的 GPU 池里，各自选择并行度和 batch，目标是优化 goodput，即满足 TTFT 和 TPOT 两个 SLO 的请求率。代价是 KV 需要跨卡甚至跨机传输。DistServe 报告，在相同 SLO 下可服务的请求率最高提升了 7.4 倍。

第五项是前缀缓存。SGLang [10] 的 RadixAttention 用基数树管理 KV，自动复用不同请求之间共享的前缀，比如系统提示词、few-shot 示例和多轮对话历史；它还配合缓存感知的调度，优先调度命中前缀较长的请求。

这些机制的效果有多大？**本文配套实验**（15-849 服务与显存）实现了块表、引用计数、copy-on-write fork、swap/recompute 两种抢占方式，以及按块表 gather 的 attention（结果与连续存储的实现逐元素相等），再用离散事件模拟比较了三种调度（图 12）。在饱和负载（$\lambda = 160$ req/s）下，吞吐从 static 的 891 token/s 提到 continuous 的 14,050，再到 chunked 的 16,156。最长的单次迭代从 5448 ms 降到 121 ms，再降到 9.2 ms；$\lambda = 100$ 时，三者的 P99 TTFT 分别是 583 s、4.58 s 和 0.595 s。显存紧张时差别更明显：把 KV 预算压到 6.55 GB、$\lambda = 60$ 时，continuous 抢占了 73 次，重算了 26,480 个 prefill token，goodput 跌到 6.9 req/s；chunked 只抢占了 39 次，goodput 保住了 51.1。分页的浪费也可以在白板上估出来：每条序列最多浪费 block-1 个 token，平均约 block/2，$8/590 \approx 1.4\%$，和实测的 1.25% 吻合。

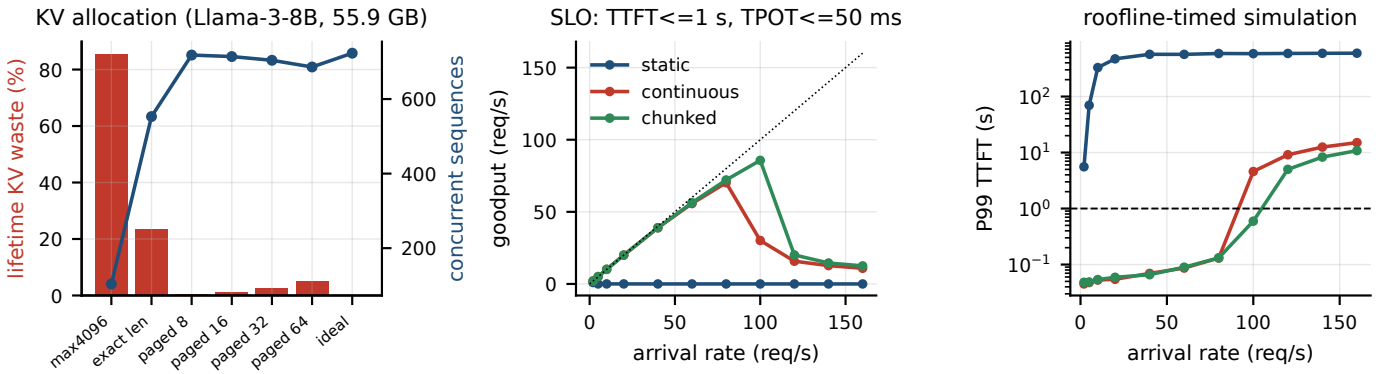


Fig. 12. 本文配套实验（15-849 服务与显存）中 KV 分配与调度的模拟结果（Llama-3-8B、H100，每步耗时取自 roofline 模型）。左：10k 个请求（长度服从对数正态分布）在生命周期内的平均 KV 浪费和并发序列数。按 4096 连续预留时浪费 85.6%、并发 104；即使预先知道最终长度也浪费 23.5%；分页（block=16）只浪费 1.25%，并发升到 714。中：goodput 随到达率的变化（SLO 为 TTFT≤1 s 且 TPOT≤50 ms），static 的峰值是 0.97 req/s，continuous 是 70.2，chunked 是 85.7。右：P99 TTFT，虚线是 1 s 的 SLO。所有时延都是模型输出，没有计入 kernel launch 和 CPU 调度开销，绝对值偏乐观；可以引用的是三者的相对关系和拐点位置。

面试追问 36：PagedAttention 解决的是什么问题？代价呢？block 大小怎么取？

答：它解决的是 KV 的内部碎片和预留浪费，并没有让 KV 变小。按最大长度连续预留时，平均浪费可以超过 80%；分页之后浪费只出现在最后一块，还能在并行采样、beam search 和共享前缀之间共用块。代价是 attention kernel 要按块表间接寻址：块越小，访存越零碎，块表也越大；但块越小，浪费也越少。vLLM 默认取 16，是在浪费和 kernel 效率之间的折中。

常见错答：「PagedAttention 减少了 KV 的大小」。它减少的是浪费，不是 KV 本身。

面试追问 37：continuous batching 与 static batching 的区别？chunked prefill 又解决了什么？

答：区别在于组批的时机。static batching 下，一批请求要等最长的那条结束，短请求空等，新请求排队；continuous batching 在每个 decode 步重新组批，吞吐和平均时延都更好。但它留下了一个问题：新请求的整段 prefill 混进某一步时，这一步会变得很长，同批 decode 的 TPOT 出现明显的尖峰。chunked prefill 给每一步设一个 token 预算，把 prefill 切成块，本文的模拟中最长的一步因此从 121 ms 降到了 9.2 ms。

常见错答：「continuous batching 就是 batch 更大」。

面试追问 38：chunked prefill 和 PD 分离怎么选？

答：看规模和 SLO。chunked prefill 在同一组 GPU 上平滑干扰，部署简单，也不用传 KV，但 prefill 和 decode 只能共用一套并行配置。PD 分离让两个池各自按 TTFT 和 TPOT 选择 TP 和 batch，规模大、SLO 严格时 goodput 更高；代价是 KV 传输（70B、2K token 的 KV 是 640 MB，在 50 GB/s 上约 13 ms），以及两池容量配比带来的运维复杂度。所以规模小或 prompt 短时选前者，规模大、prompt 长、SLO 严格时选后者。

常见错答：「PD 分离总是更好」。规模小的时候，两个池都吃不满。

把上面的知识串起来，就能回答一道典型的系统设计题。

白板估算 12：70B 模型在线推理 100 req/s 需要多少张 H100？

假设输入 2K、输出 500 token，要求 TTFT P99<2 s、TPOT P99<50 ms，使用 8×H100 节点、TP=8。权重：$70.6\times10^9\times2=141$ GB，每卡 17.6 GB。*KV* 预算：每卡 $80\times0.9-17.6-4\approx50$ GB；按 TP 切开 KV 头后，每卡每 token 是 40 KiB，整个节点约能放 123 万 token，大约是 490 条 2.5K 长度的并发序列。*decode*（batch 256、平均上下文 2.5K）：读权重要 $141/(8\times3.35)\approx5.3$ ms，读 KV 要 $256\times2500\times320\,\text{KiB}/26.8\,\text{TB/s}\approx7.8$ ms，计算要 4.6 ms，所以仍受访存限制，单步约 13 ms；按 50% 的效率算，每个节点约 1 万 token/s。需求是 $100\times500=5$ 万 token/s，需要约 5 个节点。*prefill*：每秒 $100\times2\text{K}$ 个 token，需要 $2\times70.6\times10^9\times2\times10^5\approx28$ PFLOP/s；单节点峰值 7.9 PFLOP/s，按 50% MFU 约为 4，需要约 7 个节点。合计约 12 个节点、96 张 H100，其中一半以上用于 prefill。最便宜的两项优化是前缀缓存和 FP8。

面试追问 39：什么时候前缀缓存（prefix caching）最值钱？它的代价是什么？

答：请求之间共享长前缀的时候最值钱，比如固定的系统提示词、few-shot 模板、多轮对话、agent 的工具描述、对同一份文档的多次提问。命中之后，这一段的 prefill 计算和 TTFT 都省掉了。代价有三点：缓存要占 KV 显存，和并发争抢；需要淘汰策略（LRU，或基于基数树的引用计数）；路由要有亲和性，同一前缀的请求要落到同一个实例上，否则命中率会很低。

常见错答：「前缀缓存会改变模型输出」。命中的是完全相同前缀的精确 KV，输出不变。

E. 推测解码

batch 小的时候，decode 读一遍权重只产出一个 token，带宽被浪费了。推测解码 [96], [97] 利用的正是这部分富余：先用便宜的草稿模型 q 连续提议 γ 个 token，再让目标模型 p 用一次前向并行地验证。第 i 个草稿 token 以概率 $\min(1,p(x)/q(x))$ 被接受；第一次被拒绝时，从残差分布 $\text{norm}(\max(0,p-q))$ 中重新采样，并截断后面的草稿；如果全部被接受，目标模型再多采一个 token。设每个 token 的接受率为 $\alpha=\sum_x\min(p(x),q(x))=1-\text{TV}(p,q)$（假设各位置独立同分布），那么每次目标前向产生的期望 token 数和加速比为

$$\mathbb{E}[\text{tokens}]=\sum_{i=0}^{\gamma}\alpha^i=\frac{1-\alpha^{\gamma+1}}{1-\alpha},\qquad S=\frac{\mathbb{E}[\text{tokens}]}{\gamma c+1},\tag{15}$$

其中 c 是一次草稿前向相对于一次目标前向的成本。分母里的「1」假设验证 $\gamma+1$ 个 token 和生成 1 个 token 一样贵，这在访存受限的 decode 上近似成立。

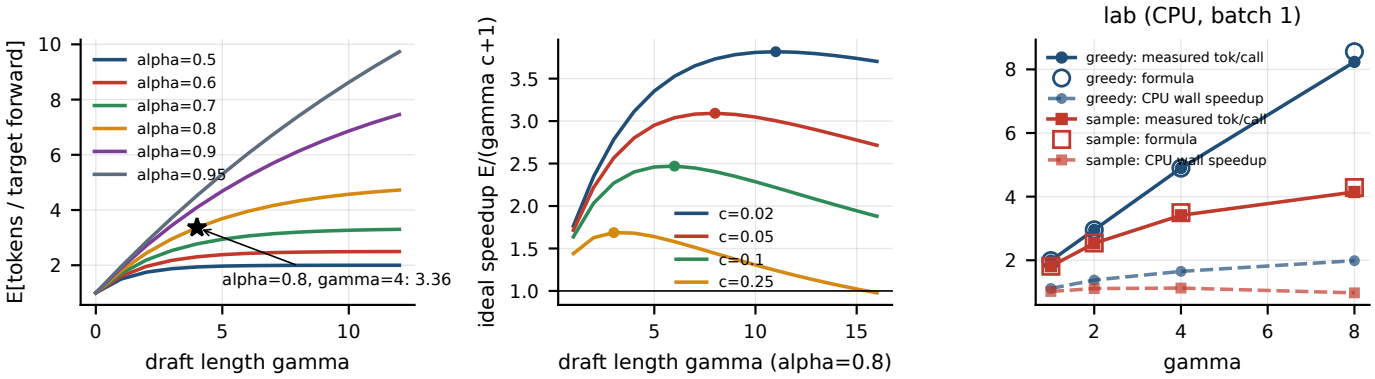


Fig. 13. 推测解码。左：式 (15) 的期望 token 数随草稿长度 γ 和接受率 α 的变化，五角星是 $\alpha=0.8,\gamma=4$ 时的 3.36；α 越高，加长草稿越有用。中：$\alpha=0.8$ 时理想加速比随 γ 和草稿成本 c 的变化，圆点是最优的 γ：草稿越贵，最优 γ 越短，收益也越低。右：本文配套实验（CS229S P2，CPU，batch 1）中每次目标调用实测得到的 token 数（实心）与公式（空心）相差不到 4%，但 CPU 上的墙钟加速只有 0.98–1.99 倍。

白板估算 13：$\alpha=0.8$、$\gamma=4$ 的推测解码

$\mathbb{E}=(1-0.8^5)/(1-0.8)=(1-0.32768)/0.2\approx 3.36$，即每次目标前向平均产出 3.36 个 token。如果草稿成本 $c=0.05$，$S=3.36/(4\times0.05+1)=3.36/1.2\approx 2.8$。从图 13 中图可以读出，$c=0.05$ 时最优的 γ 约为 8（$S\approx3.1$），$c=0.25$ 时最优 γ 降到 3，S 只有约 1.7。

本文配套实验（CS229S P2）在一个合成语法上训练了 4 层的目标模型（79.7 万参数）和 1 层的草稿模型，实现了采样版的接受–拒绝和贪心版的推测解码，并用显式给出的 p,q 做了 60,000 次试验来检验输出分布。其中有三个结论值得在面试里讲。第一，贪心版的输出和目标模型逐 token 一致（16/16）。第二，α 的估计方法很容易出错：必须用「接受数 /（接受数 + 拒绝数）」；如果用「接受数 / 提议数」，第一次拒绝之后那些从未被检验过的草稿也被算进了分母，$\gamma=8$ 时会把 0.80 算成 0.41。第三，CPU 上验证 $\gamma+1$ 个 token 的一次前向是单 token 前向的 1.41–1.61 倍，草稿的相对成本 $c=0.256$，这是被调用开销抬高的结果，而不是参数比 0.04，所以墙钟加速只有 0.98–1.99 倍。这个结果不能推出「GPU 上也只有 2 倍」：GPU 上 batch=1 的 decode 受带宽限制，多验证几个 token 几乎不增加时间。

也有方法不用独立的草稿模型。Medusa [98] 在目标模型上加几个解码头，并行预测后面的若干 token，再用树形 attention 一次验证多条候选；EAGLE [99] 在特征层（倒数第二层的隐状态）上自回归地生成草稿，接受率更高。但它们仍受同一条约束：只有当验证近乎免费，也就是 decode 受访存限制时，才划算。

面试追问 40：推测解码为什么不改变输出分布？

答：因为接受和重采样两条路径的概率加起来正好是 $p(x)$。对任意 token x，输出它的概率 $=q(x)\min(1,p(x)/q(x))+P(\text{拒绝})\cdot\frac{\max(0,p(x)-q(x))}{\sum_y\max(0,p(y)-q(y))}$。第一项等于 $\min(p,q)$；拒绝的概率是 $1-\sum\min(p,q)=\sum\max(0,p-q)$，所以第二项等于 $\max(0,p-q)$。两项之和恰好是 $p(x)$。贪心版则保证输出和目标模型的贪心解码完全相同。

常见错答：「草稿模型猜错的 token 会混进输出」；或者「推测解码是一种近似方法」。

面试追问 41：大 batch 下推测解码还划算吗？

答：通常不划算。batch 越大，decode 越接近算力受限（7B 在 batch 512 时已经受算力限制），多验证的 γ 个 token 就不再免费。式 (15) 的分母变成 $\gamma c+v$，而 v 会随 batch 增大逐渐接近 $\gamma+1$；同时，被拒绝的 token 白白浪费了算力。所以高吞吐的服务一般只在负载低时打开推测解码，或者动态调整 γ。

常见错答：「推测解码在任何负载下都能加速」。

F. 量化

推测解码是让每次读权重多产出几个 token，量化则是直接减少要读的字节。线性量化把张量映射到 b 位整数网格上。对称量化取 $\text{scale}=\max|x|/(2^{b-1}-1)$，$q=\text{round}(x/\text{scale})$，误差上界是 scale/2。粒度从粗到细分为 per-tensor、per-channel 和 per-group（$g=64/128$）。LLM 量化的核心难点是激活离群值：少数固定通道的幅值比其他通道大几十到上百倍 [100]。per-tensor 的 scale 被它撑大以后，其余通道几乎都被量化成了 0。表 X 汇总了主要的应对方法。

本文配套实验（15-849 编译与压缩）的激活量化结果见图 14。权重量化方面还有两点值得一提。其一，对重尾（Student-t）分布的权重，INT8 per-tensor 的误差是 20.6%，per-channel 是 1.97%，group128 是 1.11%，说明整张表的步长是由一个极大值决定的。其二，GPTQ 只在输入之间有相关性时才有用：输入谱衰减为 1.0 时，测试误差是 3.5%（RTN 是 25.3%）；输入各向同性时反而略差（28.2% 对 26.4%）。原因是 $H\approx cI$ 时没有可利用的相关性，补偿项拟合的只是校准集的采样噪声。

面试追问 42：为什么 LLM 的激活难量化，权重容易？

答：因为离群值集中在激活上。权重近似零均值，离群值少；激活却在少数固定通道上有几十到上百倍的离群值，per-tensor INT8 的 scale 被撑大，普通通道只剩下几个量化级。离群值是按通道出现的，而不是按 token，所以 per-token 量化只能缓解（配套实验中误差减半）。LLM.int8 把离群通道拆出去走 FP16；SmoothQuant 用逐通道缩放把激活的难度挪到权重上。

常见错答：「INT8 对所有张量都一样好做」。

TABLE X
LLM 量化方法对比（WxAy 表示权重 x 位、激活 y 位）

方法	典型配置	核心思想	加速来源	代价与局限
LLM.int8() [100]	W8A8 + FP16 离群	把幅值超阈值（6）的特征维拆出走 FP16，其余 INT8	显存减半	混合分解需额外 kernel，吞吐不一定提升
SmoothQuant [101]	W8A8	$XW = (X\mathrm{diag}(s)^{-1})(\mathrm{diag}(s)W)$，把激活难度按通道挪到权重	INT8 tensor core，prefill 与 decode 都受益	需要校准；权重也变难量化
GPTQ [102]	W4A16/W3A16	逐列量化，用 Hessian $H = X^\top X$ 的逆把舍入误差补偿到未量化的列	decode 读权重字节减少	依赖校准数据的协方差；prefill 不加速
AWQ [103]	W4A16	按激活幅值找出约 1% 的显著权重通道，用等价缩放保护	同上，硬件友好（无混合精度）	需要校准；仍是 weight-only
QLoRA [104]	NF4 权重 + BF16 LoRA	4 位 NormalFloat、双重量化、分页优化器	微调显存	训练吞吐低于 BF16
FP8 [87]	W8A8（E4M3）	浮点格式自带动态范围，per-tensor/per-block scale	H100 tensor core 2 倍算力	需要 scale 管理；仅新硬件支持

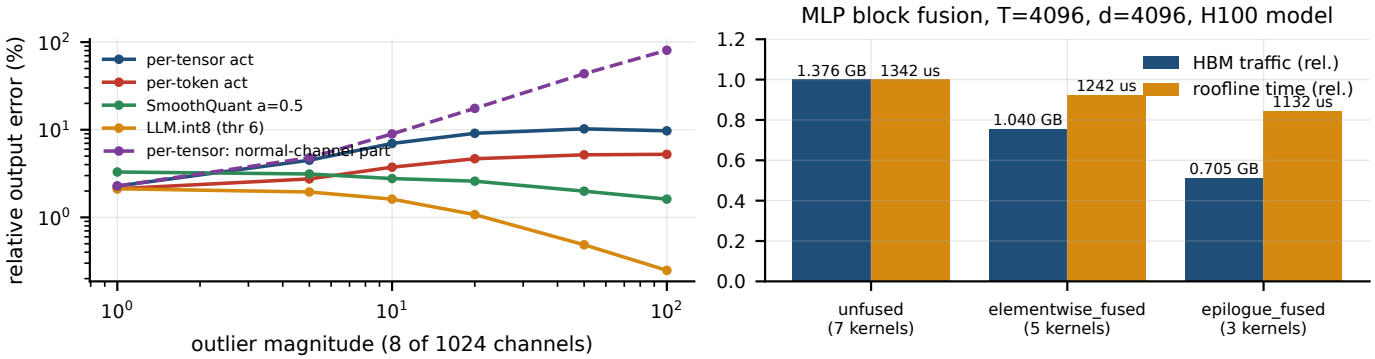


Fig. 14. 本文配套实验（15-849 编译与压缩）。左：在 1024 个通道中放入 8 个离群通道，W8A8（权重 per-channel）的相对输出误差随离群倍数的变化。50 倍离群时，per-tensor 激活量化的误差是 10.2%，per-token 是 5.17%，SmoothQuant（$\alpha = 0.5$）是 1.99%，LLM.int8 是 0.49%。虚线显示，per-tensor 下普通通道部分的误差高达 43.6%（100 倍时 80.8%）；整体误差「只有」10%，是因为离群通道同时主导了输出的范数，掩盖了普通通道的损坏。右：Transformer MLP 块的融合，HBM 流量 −49%，roofline 时间却只有 −16%（第 IV 节）。数据是合成的，用于说明机制。

面试追问 43：weight-only 量化（W4A16）为什么能加速 decode，却不一定加速 prefill？W4A16 与 W8A8 怎么选？

答：因为两个阶段的瓶颈不同。decode 受带宽限制，读 INT4 权重比 BF16 少 4 倍字节，反量化在寄存器里做，计算本来就不是瓶颈；prefill 受算力限制，权重仍要反量化成 BF16 才能上 tensor core，算力一点没省，反而多了反量化的开销。所以小 batch、低时延的 decode 选 W4A16；大 batch、prefill 占比高、受算力限制的服务选 W8A8 或 FP8，用上 INT8/FP8 的 tensor core。

常见错答：「量化总是更快」；「4 位一定比 8 位快」。

表 XI 把本节讨论过的服务技术放在一起。它们几乎都在应对同一个约束：decode 受带宽和 KV 容量限制，prefill 受算力限制，而在线服务又要同时满足 TTFT 和 TPOT。面试中的系统设计题（比如上面的 70B 估算）考的就是能否按收益给这些技术排序。一个合理的顺序是：先上 continuous batching 和 PagedAttention，再做量化和前缀缓存，然后是 chunked prefill 或 PD 分离，最后才是推测解码。

面试要点

- decode 步时 $\approx \max(2NB/P,\ (sN + \mathrm{KV})/\mathrm{BW})$：临界 batch 约等于拐点（H100 约 295），但 KV 不能被 batch 分摊。
- KV $= 2Ln_{kv}d_{\text{head}}TBs$：8B 是 128 KiB/token，70B 是 320 KiB/token；8B 在 8K 时 1 GiB，128K 时 16 GiB。
- 调度的演进：continuous batching、PagedAttention、chunked prefill，再到 PD 分离和前缀缓存。
- 推测解码 $\mathbb{E} = (1-\alpha^{\gamma+1})/(1-\alpha)$，$\alpha = 0.8, \gamma = 4$ 时为 3.36；输出分布无偏；大 batch 下不划算。
- 激活离群值按通道出现：W4A16 加速 decode，W8A8/FP8 加速 prefill。

研究视角

- 本文的模拟显示，chunked prefill 的吞吐优势有一部分来自 roofline 假设下「混合步完美重叠」。真实 GPU 上混合步的效率是多少？可以在真实引擎中固定 decode batch，改变捎带的 prefill token 数，测量步长时间与 $\max(F/P, B/\mathrm{BW})$ 的偏差。
- 推测解码的接受率 α 并不是独立同分布的，它随位置、温度和领域变化。一个可做的实验是按位置统计接受率，检验式 (15) 的偏差，并据此设计自适应 γ 的策略。评估指标必须是同一负载下的 goodput，而不是 batch=1 时的加速比。

TABLE XI
LLM 服务系统的关键技术对比

系统/技术	解决的问题	机制	代价与适用条件
Orca [4]	静态批处理的空等与排队	iteration-level 调度、selective batching	需要 KV 管理支持动态加入/离开
vLLM [3]	KV 碎片与预留浪费	PagedAttention：块表、引用计数、CoW、抢占（swap/recompute）	kernel 间接寻址；块大小折中
Sarathi-Serve [5]	prefill 造成 decode 卡顿（TPOT 尖峰）	chunked prefill、stall-free batching、token 预算	prefill 被切块后 TTFT 略增；预算需调
DistServe [6] / Splitwise [12]	prefill 与 decode 干扰、并行配置耦合	PD 分离、按 goodput 规划两池并行度	KV 跨机传输；容量配比与运维
SGLang [10]	请求间前缀重复计算；结构化程序	RadixAttention 前缀缓存、缓存感知调度、约束解码	缓存占用显存；需要亲和路由
推测解码 [96], [98], [99]	batch 小时 decode 带宽受限	草稿 + 并行验证，分布无偏	大 batch 下收益消失；需要草稿模型或额外头
S-LoRA [105] / Punica [106]	多 LoRA 适配器服务	统一分页、分段 gather-matmul（SGMV）	异构 rank 的 kernel 效率

IX. 模型效率与架构

前几节都把模型当作给定的，只改系统。这一节反过来，讨论通过「改模型」来改变系统代价的方法：参数高效微调、剪枝与蒸馏、混合专家（MoE）、attention-free 架构，以及决定「训多大的模型、用多少数据」的 scaling law。面试中这部分考的不是算法细节，而是每种方法把式 (1) 的哪一项换成了哪一项。

A. 参数高效微调：*LoRA* 与 *QLoRA*

全量微调一个大模型，显存主要花在梯度和优化器状态上。LoRA [107] 的做法是冻结预训练权重 $W \in \mathbb{R}^{d\times k}$，只训练一个低秩的增量：

$$W' = W + \frac{\alpha}{r} BA, \quad A \in \mathbb{R}^{r\times k},\ B \in \mathbb{R}^{d\times r},\ B_0 = 0, \tag{16}$$

可训练参数只有 $r(d+k)$ 个；推理时可以把 BA 合并回 W，不增加时延。$d = k = 4096$、$r = 16$ 时，可训练参数是 131K，只占全量 16.8M 的 0.78%。QLoRA [104] 再把冻结的基座量化成 4 位 NormalFloat（NF4），加上双重量化和分页优化器，使 65B 模型能在单张 48 GB 的 GPU 上微调。

本文配套实验（15-849 编译与压缩）验证了 LoRA 的两个恒等式：初始化时输出与基座的最大差为 0，合并后的最大差是 3.3×10^{-15}。它还验证了低秩逼近的下界：$d = 256$、真实 ΔW 的秩为 8 时，$r = 4$ 的误差 0.434 已经接近 Eckart–Young 下界 0.424，$r = 8$ 时降到 0.044。按 Llama-3-8B 的形状算，$r = 8$ 只挂在 q、v 上是 3.41M 参数（0.042%），$r = 16$ 挂在全部线性层上是 41.9M（0.52%）。

面试追问 44：LoRA 省的是什么显存？不省什么？

答：省的是梯度和优化器状态。只有 $r(d+k)$ 个参数需要 FP32 主权重和 Adam 的 m, v，7B 模型的优化器状态可以从 84 GB 降到 0.1 GB 的量级。不省的是基座权重和激活：BF16 的基座仍要 $2N$（QLoRA 用 4 位把它降到约 $0.5N$），反向传播也仍要流经全部层。

常见错答：「LoRA 省了前向显存」，或者「LoRA 让训练快很多」。它的前反向计算量和全量微调相近。

B. 剪枝、稀疏与蒸馏

剪枝想直接减少参数，难点在于稀疏能不能在硬件上兑现成加速。Deep Compression [108] 用剪枝、量化（权重共享）和 Huffman 编码，把 CNN 压缩了一到两个数量级；但非结构化的稀疏在 GPU 上很难变成实际加速。NVIDIA 的 2:4 结构化稀疏（每 4 个元素保留 2 个）由 Ampere 之后的稀疏 tensor core 直接支持，理论算力翻倍 [36]。SparseGPT [109] 把 GPTQ 式的二阶补偿用到了一次性剪枝上。知识蒸馏 [110] 则用大模型的软标签训练小模型，是部署侧最常用的「改模型」手段之一。

结构约束是有代价的。**本文配套实验**（15-849 编译与压缩）中，对一个 1024^2 的重尾矩阵剪掉 50%，非结构化剪枝的输出误差是 20.4%，2:4 是 29.0%，4:8 是 25.1%。在一个小 MLP 上，2:4 稀疏不微调时准确率是 62.9%，微调 3 轮后回到 67.2%（稠密模型是 66.2%）。这说明结构约束越强，就越依赖恢复训练。

面试追问 45：2:4 稀疏和非结构化稀疏怎么选？

答：在精度和能否加速之间取舍。非结构化稀疏在同样稀疏度下精度更好，但访存不规则、索引开销大，GPU 很难加速。2:4 有硬件支持（稀疏 tensor core，每 4 个元素用 2 位元数据），能兑现约 2 倍的算力上限，但约束更强、精度损失更大，通常要微调来恢复。实际加速还要看 GEMM 是否受算力限制：decode 受带宽限制时，2:4 省下的是权重字节，而不是 FLOPs。

常见错答：「剪掉 50% 就快 2 倍」。非结构化的 50% 在 GPU 上几乎不会变快。

C. 混合专家（*MoE*）与专家并行

能不能让参数变多，而每个 token 的计算不变？这就是 MoE 的想法。Sparsely-gated MoE [111] 用一个门控网络为每个 token 选出 top-k 个专家（每个专家是一个 FFN），于是参数量随专家数 E 线性增长，每 token 的 FLOPs 却只随 k 增长。GShard [79] 用 top-2 门控和自动分片把 MoE 扩展到了 600B 参数；Switch Transformer [80] 进一步简化为 top-1，并引入了容量因子和辅助负载均衡损失。容量因子规定每个专家最多接收 capacity = factor × tokens/E 个 token，溢出的 token 被丢弃或走残差连接。

专家并行（EP）把专家分到不同的卡上，每层前向需要两次 all-to-all，一次把 token 分发给专家（dispatch），一次把结果收回来（combine），反向还要再两次。DeepSpeed-MoE [112] 从推理系统的角度优化 MoE；MegaBlocks [113] 用块稀疏 kernel，避免了容量因子带来的丢 token 和 padding。DeepSeek-V2/V3 [88], [94] 采用了细粒度专家加共享专家的结构。

白板估算 14：MoE 的 all-to-all 每次要传多少？

每张卡 8192 个 token、$d = 4096$、top-2、BF16、EP=8：每张卡每次发送 $\approx 8192 \times 2 \times 4096 \times 2 \times \frac{7}{8} \approx 117$ MB。每层前向 2 次、反向 2 次；如果跨 IB（50 GB/s），每次约 2.3 ms，32 层每步约 0.3 s，和 TP 的通信是同一个量级。所以 EP 通常放在节点内，或者和 DP 交织，以利用节点内的带宽。

面试追问 46：MoE 为什么「参数多但不贵」？它贵在哪？

答：不贵的是 FLOPs：每个 token 只经过 top-k 个专家，计算量随 k 增长，而不是随 E。贵的地方有三处。一是显存，全部专家都要放下，部署时要按总参数而不是激活参数来配卡。二是通信，EP 每层前反向共 4 次 all-to-all，对网络拓扑很敏感。三是负载不均，热门专家所在的卡会拖慢整步，需要负载均衡损失、容量因子，或者无丢弃的块稀疏实现。推理时还有一点：decode 读的是被激活专家的权重，batch 一大，几乎所有专家都会被激活，访存上的优势也就随之减少。

常见错答：「MoE 省算力，所以训练和部署都便宜」。

D. *Attention-free* 与状态空间模型

第 VIII 节看到，长上下文下 KV cache 是 decode 的主要负担。Transformer [114] 的 attention 让 KV cache 随长度线性增长，而循环形式的模型每一步的状态大小是固定的，这正是 attention-free 架构吸引人的地方。线性 attention [115] 把 $\mathrm{softmax}(QK^\top)V$ 换成 $\phi(Q)(\phi(K)^\top V)$，复杂度降到 $O(Td^2)$。S4 [116] 用结构化状态空间模型，训练时写成长卷积的形式来并行。Hyena [117] 用隐式参数化的长卷积加门控来替代 attention。Mamba [118] 引入了随输入变化的选择性状态空间，用硬件感知的并行扫描来训练；推理时每个 token 的代价是 $O(1)$，没有 KV cache。

本文配套实验（CS229S P1）有两个与此相关的测量。第一个是 depthwise 因果卷积的三种写法（$C = 16$，float64）。FFT 的耗时只取决于 N，$N = 16384$ 时约 14–15 ms；$O(NK)$ 的直接写法随 K 线性增长，其中移位相加在 $K \approx 16$ 处、滑窗在 $K \approx 64$ 处与 FFT 交叉。所以短卷积（Mamba/H3 的 $K = 4$）直接写快 5.2–8.5 倍，而长卷积（Hyena 的 $K = N$）在所有 N 上都是 FFT 更快。第二个是 associative recall。一个手工设定权重、没有经过训练的两层 attention-only 模型（一个「上一 token」头加一个匹配头，也就是 induction head）就能精确地解这个任务：$\beta \geq 8$ 时，32 个键值对的准确率是 1.0；$\beta = 1$ 时只有 0.227。这说明 attention 天然擅长精确检索；而状态大小固定的 SSM 必须把历史压缩进状态里，这正是它在 recall 类任务上的弱点。

面试追问 47：Mamba/SSM 相对 Transformer 的推理优势是什么？代价呢？

答：优势是状态大小固定。写成递归形式时，decode 不需要随长度增长的 KV cache，每个 token 的代价是 $O(1)$，长序列、大 batch 下吞吐和显存都更好；训练时则用并行扫描或卷积形式来并行。代价是固定大小的状态是对历史的有损压缩，精确检索（长距离复制、associative recall）不如 attention。所以出现了把 attention 和 SSM 混合起来的架构。

常见错答：「SSM 在所有任务上都比 attention 好」，或者「SSM 训练时也是每 token $O(1)$，没法并行」。

E. *Scaling law* 与计算最优

最后一个问题是：给定一笔算力，应该训多大的模型、用多少数据？Kaplan 等人 [119] 发现，损失随参数量 N、数据量 D 和算力 C 呈幂律下降。Chinchilla [120] 在固定算力下重新做了拟合，结论是参数和数据应该大致同比例增长，约为每个参数 20 个 token：

$$C \approx 6ND, \quad D^* \approx 20N^* \;\Rightarrow\; N^* \approx \sqrt{C/120}. \tag{17}$$

要注意，这是训练算力最优，而不是推理成本最优。要大规模部署的模型常常远超 20 token/参数：Llama-3-8B 训了 15T token，约 1868 token/参数 [2]，相当于用更多的训练算力换一个更小、推理更便宜的模型。图 15 左把若干公开模型放在 Chinchilla 曲线旁边：GPT-3 175B 只训了 300B token（约 2 token/参数），明显「欠训」；Chinchilla 70B 落在曲线上；Llama 系列则大幅「过训」。

白板估算 15：给定 6×10^{23} FLOP，计算最优的模型多大？

$N^* = \sqrt{6 \times 10^{23}/120} \approx 7.1 \times 10^{10}$，也就是约 70B；$D^* = 20N^* \approx 1.4$T token。这和 Chinchilla 70B/1.4T 一致（$6 \times 70 \times 10^9 \times 1.4 \times 10^{12} \approx 5.9 \times 10^{23}$）。如果改训 8B 模型，同样的算力可以训约 12T token，推理成本约为 70B 的 1/9。用 1 万张 H100、MFU 40%，6×10^{23} FLOP 约需 1.5×10^5 s，不到 2 天。

面试追问 48：Chinchilla 的结论是什么？「每个模型都该训 20 token/参数」对吗？

答：不对，因为 Chinchilla 只考虑了训练算力。它的结论是：固定训练算力时，参数和数据应同比例扩展，最优约为 20 token/参数，$C \approx 6ND$。但如果模型要服务大量请求，总成本是训练加推理，而推理成本随模型大小线性增长，所以应该训练更小的模型、喂更多的 token（Llama-3-8B 约 1868 token/参数）。可用的高质量数据有限，也是一个现实约束。

常见错答：「Chinchilla 说模型训 20 token/参数最好」。这忽略了推理成本和数据约束。

面试要点

- LoRA 的参数量是 $r(d+k)$，省梯度和优化器状态，不省基座权重和激活；QLoRA 再把基座压到 4 位。
- 2:4 稀疏有硬件支持（约 2 倍的算力上限）；非结构化稀疏难以兑现加速。
- MoE：FLOPs 随 k 增长，显存随 E 增长；EP 每层前反向共 4 次 all-to-all（8192 token、$d = 4096$、top-2、EP=8 时约 117 MB/次）。
- SSM：decode 每 token $O(1)$，没有 KV，但精确检索弱；短卷积直接算，长卷积用 FFT。
- Chinchilla：$N^* = \sqrt{C/120}$、$D^* = 20N^*$；要部署的模型常常故意过训。

研究视角

- 推理感知的 scaling law：把部署时的 token 总量 D_{inf} 纳入目标函数 $\min_{N,D} 6ND + 2ND_{\text{inf}}$（在损失约束下），最优的 token/参数比怎样随 D_{inf} 变化？这是一个可以用公

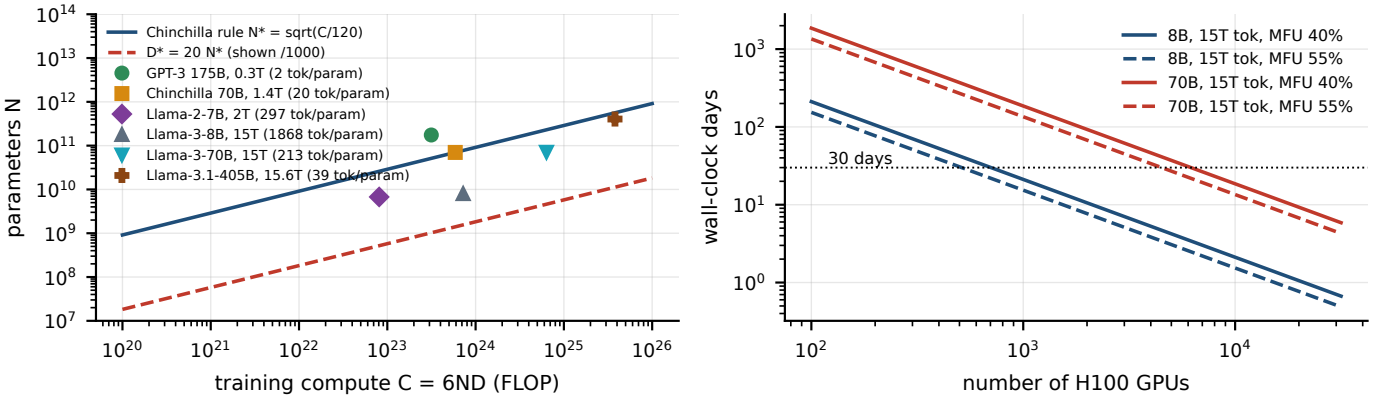


Fig. 15. 左：Chinchilla 计算最优规则 $N^* = \sqrt{C/120}$（实线）与 $D^* = 20N^*$（虚线，缩小 1000 倍显示），以及公开模型的 (C, N) 和 token/参数比；离曲线越远，说明这个模型的训练越偏离「训练算力最优」。右：在 H100 上用 15T token 训练 8B 和 70B 模型所需的墙钟天数随 GPU 数的变化（MFU 取 40% 和 55%）。70B 用 8192 卡、MFU 40% 时约需 23 天。

开拟合系数来做的解析练习。

- MoE 的通信与计算之比随专家粒度变化：细粒度专家增加了 all-to-all 的消息数（α 项），但改善了负载均衡。可以在本文的 α–β 模拟器上扫描专家数和 top-k，找出 all-to-all 从带宽主导转为延迟主导的拐点。

X. 集群、可靠性与数据

把单个作业的并行策略调好以后，大模型训练的真实效率还受三件事影响：多个作业怎样共享集群（调度），万卡规模下的故障怎样才不会吞掉训练进度（可靠性），以及 GPU 是不是在等数据（数据管线）。这一层是训练 infra 岗位高级面试题的主要来源。

A. 深度学习集群调度

深度学习作业和传统的批处理作业很不一样。Jeon 等人分析微软 Philly 集群 [121] 后发现：分布式训练需要 gang scheduling，即所有 worker 必须同时启动，否则先启动的只能空等；作业对拓扑局部性很敏感，最好放在同一节点或同一交换机下，而这种局部性约束又让排队时间变长；还有相当比例的作业因为用户错误或故障而失败。针对这些特点，Gandiva [122] 利用 DL 作业的迭代性做时间片轮转和迁移，并根据运行时的观察调整放置。Tiresias [123] 在不知道作业时长的前提下，按「已获得的服务量 = GPU 数 × 时间」做二维 LAS（least attained service）调度，降低平均完成时间。Pollux [124] 注意到吞吐和统计效率都会随 batch 变化，于是联合调整每个作业的 GPU 数和 batch size，以 goodput 为优化目标。

面试追问 49：深度学习集群调度和传统批处理调度有什么不同？

答：主要有四点不同。第一是 gang scheduling：分布式作业的所有 worker 必须同时拿到资源。第二是拓扑敏感：放在同一节点或同一交换机下，会直接影响 all-reduce 的时间。第三是可抢占和弹性：作业可以依靠 checkpoint 暂停、迁移和伸缩。第四是吞吐与资源不成线性关系：卡数加倍不一定吞吐加倍，batch 变大还会改变统计效率（Pollux）。

常见错答：「按 GPU 数量排队就行」。

B. 大规模训练的故障与恢复

到了万卡规模，故障就是常态。Llama 3 报告，在约 1.6 万张 H100 上的一段 54 天预训练中，作业共中断了 466 次，其中 419 次是非计划中断；约 78% 归因于已确认的硬件问题（GPU 和主机组件），GPU 相关问题是最大的一类。靠自动化恢复，有效训练时间仍然超过了 90% [2]。MegaScale [38] 在 12,288 张 GPU 上，把算法、通信重叠、算子和数据管线的优化，与「深度可观测性」（细粒度监控、心跳、自动诊断和慢节点剔除）结合起来，达到了 55.2% 的 MFU。Gemini 技术报告 [125] 也讨论了大规模训练中的静默数据损坏（SDC）和快速恢复。

白板估算 16：万卡训练多久坏一次？checkpoint 应该多久存一次？

设单卡的年故障率是 2%，1 万张卡每年约坏 200 次，平均约 44 小时一次；再算上网络、主机和软件故障，实际往往是十几小时一次。70B 模型的完整 checkpoint（BF16 权重加 FP32 主权重和 Adam 状态）约为 $16 \times 70.6 \times 10^9 \approx 1.1$ TB。Young 公式 [126] 给出最优的保存间隔 $\tau \approx \sqrt{2\delta M}$，其中 δ 是一次保存的开销，M 是平均故障间隔。取 $\delta = 30$ s、$M = 12$ h，$\tau \approx \sqrt{2 \times 30 \times 43200} \approx 1610$ s，也就是约 27 分钟存一次；每次故障平均损失约 $\tau/2$，再加上重启的时间。

从公式可以看出，工程上的关键是把 δ 压小。异步 checkpoint 先把分片拷到主机内存，每张卡几 GB，几秒就能完成，训练随即继续，再由后台写入分布式存储；每个 DP 组只需要保存一份；CheckFreq [127] 则把快照和持久化组成流水线，并自适应地调整保存频率。δ 越小，最优间隔就越短，故障造成的平均损失也越小。

面试追问 50：checkpoint 间隔怎么定？为什么要异步？

答：间隔要在保存开销和故障损失之间平衡：间隔太短，保存开销占比高；太长，故障时丢的进度多。最小化期望损失，就得到 Young 公式 $\tau \approx \sqrt{2\delta M}$。异步保存把 δ 从「写分布式存储的时间」降成「拷贝到主机内存的时间」，既降低了开销，又允许存得更频繁。恢复时间也要考虑：从内存副本（或相邻节点上的冗余副本）恢复，比从远端存储读取快得多。

常见错答：「每 N 步存一次就行」，不考虑故障率和保存开销；或者同步写远端存储，把训练阻塞住。

TABLE XII
时间线症状、最可能的原因与区分性观测

时间线上看到的	最可能的原因	区分它的观测
kernel 间大量小空洞	CPU 发射开销、小 kernel 过多	空洞宽度与 kernel 数成正比；CUDA Graph 或 `torch.compile` 后消失
每步开头一段空白	dataloader 跟不上	加倍 `num_workers` 后变短；GPU 空闲时 CPU 满载
反向后一大段 NCCL	梯度通信未与反向重叠	bucket 过大或开启了 `find_unused_parameters`
周期性长尖峰	分配器碎片整理、checkpoint	`num_alloc_retries` 增长
利用率 100% 但慢	访存受限 kernel	DRAM 吞吐近峰值、SM 吞吐低
某卡总是最后到	负载不均或慢卡	每 rank 步时分布；换卡后是否跟随

面试追问 51：分布式训练挂住了，你怎么查？

答：挂住几乎总是因为集合通信对不上：某个 rank 少调或多调了一次 collective，或者各 rank 的调用顺序不同。我会这样查。先打开 `NCCL_DEBUG=INFO` 和异步错误处理，并设一个合理的超时，让「挂住」变成「报错」。再用 `py-spy dump` 看每个 rank 停在哪一行，如果停在不同的 collective 上，就是顺序不一致。常见的根因有：只在 rank 0 执行的分支里调用了 collective；输入长度不同，导致某个 rank 多跑了一轮；部分参数在某些 rank 上没有梯度。如果只是变慢而不是挂住，就先用 `nccl-tests` 测 bus bandwidth，与式 (10) 的理论值比较，再检查拓扑（是不是走了 PCIe）和 IB 的配置。

常见错答：「重启一下」或「把超时调大」，却不去定位是哪个 rank 不一致。

C. 性能排查的顺序

训练或推理变慢时，面试官想听的是排查的顺序，以及能区分不同假设的那个观测。一个可以复述的顺序是：先用 roofline 算出上界，实测如果已经在上界的 1.2 倍以内，就别再优化 kernel 了；再看时间线上有没有空洞；然后看单个 kernel 的 DRAM 和 SM 吞吐；最后才调超参，而且每次只动一个变量。表 XII 把常见的时间线症状和对应的原因、区分方法列在一起。

面试追问 52：MFU 只有 25%，你怎么查？

答：先算账，再看 profile。算账是指用 $6N$ 加上 attention 项估计模型 FLOPs，并确认 MFU 的分母用的是稠密峰值，不含稀疏。然后按占比从大到小逐项排查：通信有没有暴露（没有和计算重叠的 all-reduce 或 all-gather）；流水线气泡是不是太大（$m < 4p$）；dataloader 有没有造成每步开头的空白；逐元素 kernel 有没有融合；micro-batch 是不是太小，GEMM 吃不满；是不是开了全量重计算（这时 HFU 高而 MFU 低）。每次只改一个因素，改完重新测。

常见错答：一上来就去手写 kernel，或者只报 HFU。

D. 数据管线

最后一个容易被忽视的问题是：GPU 会不会在等数据？Mohan 等人 [128] 发现，很多训练作业有相当一部分时间耗在「数据停顿」（data stall）上，也就是数据读取和预处理跟不上 GPU。tf.data [129] 用流水化、并行 map 和自动调优来缓解这个问题。LLM 预训练的数据管线以离线预分词、内存映射的二进制分片和按长度打包（packing）为主，在线开销较小；但多模态（要解码图像和视频）和 RL（要生成 rollout）又把数据这一侧推回了关键路径。

这里还有两个经典系统值得一提。Spark 的 RDD [130] 靠 lineage 重算实现容错，是大规模离线数据处理的基础；参数服务器 [65] 则在推荐场景中继续承担稀疏 embedding 的训练。把这两者和 all-reduce 对照着讲，是 CS149 和 15-849 之间最自然的衔接：它们分别代表了数据并行原语在批处理、稀疏训练和稠密训练中的三种形态。另一个方向是用学习的方法来优化系统本身，比如用强化学习做设备放置 [131]，这对研究岗是加分项。

面试要点

- DL 调度的特点：gang、拓扑敏感、可抢占与弹性、吞吐与资源非线性（Gandiva、Tiresias、Pollux）。
- 万卡训练的故障按小时计；Llama 3 的 54 天里有 419 次非计划中断，约 78% 是硬件问题。
- Young 公式 $\tau \approx \sqrt{2\delta M}$；用异步、分片、先写内存再落盘来压小 δ。
- 排查顺序：先算 roofline 上界，再看时间线空洞，然后看单 kernel 吞吐，最后调超参；训练挂住时先查 collective 的调用顺序。

研究视角

- 静默数据损坏（SDC）在万卡规模下怎样检测？一个可做的设计是周期性地让不同的 rank 重复计算同一个小批次，并比较梯度的哈希；需要量化它的开销和检出率之间的权衡。
- 用本文的事件模拟器研究「慢节点」对同步训练的影响：一个 rank 慢 10% 时，DP、PP、TP 各自的整体吞吐下降多少？哪种并行方式对尾部延迟最敏感？

XI. 面试能力框架

前十节讲的是知识。可是面试官看不到你脑子里的知识，只能看到你怎样回答问题。本节把前面的内容组织成面试中能被观察到的能力。MLSys 面试通常同时考两件事。一是*工程能力*：能不能把一个系统问题拆成可以测量、可以估算的部分，并按正确的顺序定位瓶颈。二是*研究能力*：能不能读懂一篇论文真正的贡献和前提，并提出能区分不同假设的实验。两者的底座是同一个，即式 (1) 的三项分解和第 II 节的 roofline。

A. 按岗位的能力矩阵

不同岗位的侧重点不同。表 XIII 按五类常见岗位，列出了工程和研究两条线上的核心要求、必须当场算出的数，以及面试官最常深挖的方向。它和图 1 的课程分层是对应的：推理岗主要对应第 VIII 节，训练 infra 对应第 VII、X 节，算子和编译对应第 IV、VI 节，芯片岗对应第 III、V 节。

B. 回答模板：结论、估算、瓶颈、权衡、证据

开放式系统题最常见的失败是「列名词」：把知道的技术全报一遍，却不说当前场景下哪一个真正起作用。下面这个五步模板逼着回答落到数字上：

TABLE XIII
岗位 × 能力矩阵：工程与研究两条线的核心要求

岗位	工程能力（必须能做）	研究能力（加分项）	必须当场算出	常被深挖
LLM 推理系统	KV 管理、batching 调度、量化部署、推测解码的工程取舍；读 profile 定位 decode 瓶颈	调度策略的理论分析；prefill/decode 分离的建模；新注意力变体的系统代价	decode 步时、KV 容量、并发上限、TTFT/TPOT	vLLM 分页细节、chunked prefill、SLO 与 goodput
训练 infra	DDP/FSDP/3D 并行配置、显存预算、通信重叠、checkpoint 与故障恢复	并行策略自动搜索、通信压缩、故障与慢节点建模	$16N$ 显存、all-reduce 时间、气泡、$6ND$ 天数	ZeRO 各阶段、TP 为何限在节点内、MFU 排查
算子 / 编译器	CUDA/Triton kernel、tiling、融合、profile（Nsight）	调度空间搜索、代价模型、动态形状	算术强度、tile 大小、HBM 字节、MFU	FlashAttention IO、bank conflict、occupancy
AI 芯片 / 加速器	数据流映射、片上存储规划、性能建模与仿真	新数据流与稀疏/低精度支持、软硬协同	PE 利用率、片上带宽、能耗/字节	systolic 填充开销、TPU 与 GPU 的取舍
MLSys 研究	可复现实验、基线公平、消融设计	找到被忽视的前提、提出可证伪的假设	任一上界估算	论文的局限、你会做的下一个实验

1) 结论：第一句话就给判断，比如「这是带宽受限，先上 GQA 和 INT8 权重」；
2) 估算：用第 II 节的 roofline 和附录的公式给出数量级；
3) 瓶颈：说明估算显示的主导项是计算、访存还是通信，以及它在什么条件下会切换；
4) 权衡：讲清每个方案的代价，包括精度、时延、复杂度和显存；
5) 证据：说出怎样测量来验证判断，指出能区分两个假设的那个观测。

面试追问 53：「把 Llama-3-70B 在线服务的成本降一半」，你怎么答？

答：结论：decode 受带宽限制，所以优先减少每个 token 要读的字节数，其次提高 batch。估算：BF16 权重是 141 GB，8 卡 TP 时每卡每步读约 17.6 GB，在 3.35 TB/s 的 H100 上约 5.3 ms/步；GQA 下每个 token 的 KV 是 320 KiB。瓶颈：batch 小时权重占主导，batch 大、上下文长时 KV 占主导。权衡：INT8/FP8 权重把权重字节减半，精度损失要在具体任务上验证；FP8 KV 让并发翻倍；推测解码能降低时延，但在大 batch 下收益递减；前缀缓存在共享系统提示词的场景下收益最大。证据：先测当前的 batch 分布和 KV 占用。如果平均 batch 已经很大、KV 已经占满，就优先做 KV 量化和前缀缓存；如果 batch 很小，就优先做权重量化。

常见错答：不做任何估算，直接列出「量化、剪枝、蒸馏、FlashAttention、vLLM」。

C. 常见错答

前面各节的追问框里，有些错答出现得特别频繁，表 XIV 把它们汇总在一起。这些错答几乎都来自同一个错误：把一个只在特定区间成立的结论当成了普遍规律，却没有说明区间的边界在哪里，比如 batch、序列长度、拓扑或精度。

TABLE XIV
高频错答与纠正

错答	纠正
FlashAttention 把 attention 变成线性	FLOPs 不变；HBM 流量 $\Theta(T^2d^2/M)$ 仍二次；显存从 $O(T^2)$ 到 $O(T)$
decode 慢是因为算力不够	小 batch 下带宽受限；大 batch、短上下文才转为计算受限
ZeRO-3 通信和 DP 一样	约为 DP 的 1.5 倍（多一次前向 all-gather）
ring all-reduce 时间随卡数线性增长	带宽项 $2(p-1)/p \cdot S/B$ 几乎与 p 无关，延迟项线性
7B 模型训练需要 14 GB	混合精度 Adam 约 $16N = 112$ GB，另加激活
GQA 减少计算	主要减少 KV 字节，比例 n_{heads}/n_{kv}
推测解码总能加速	α 低或大 batch 时可能变慢
MoE 省算力所以便宜	显存按总参数，且有 all-to-all 与负载不均
利用率 100% 说明 GPU 吃满	只表示有 kernel 在跑，看 SM 与 DRAM 吞吐

D. 把配套实验变成 *STAR* 故事

面试里最可控的部分是项目经历，因为题目是你自己出的。本文配套仓库的每个实验，都按「20 秒结论、STAR、白板推导、不能推出的结论、追问」的顺序整理过，表 XV 列出了十个故事的结论和考察点。所有数字都来自 CPU 上的测量，或者解析模型、离散事件模型的输出，都不是 GPU 实测。讲故事时必须主动交代这一点，并说出这个实验不能推出什么。面试官判断你是否真正理解自己的实验，这往往是最强的信号。

怎么挑故事？只准备两个，但要讲透。推理岗用 1 和 3，训练岗用 4 和 6（或 4 和 10），算子和芯片岗用 1 和 7。开头先报结果数字，讲到 90 秒左右就停下来，给对方留出打断追问的机会。

面试追问 54：讲一个你发现自己实验结论错了的经历。

答：（故事 10）我第一版多进程 Mandelbrot 测出 8 个 worker 有 21 倍加速，可机器只有 4 个物理核，这显然不可能。我没有直接报告这个数，而是先怀疑基线。块是按 worker 数来切的，块变小以后，numpy 的中间数组落进了缓存：同一张图串行整块算要 3.65 s，按小块串行算只要 1.29 s。也就是说，加速有一大半来自缓存行为的改变，而不是并行。把所有调度方式的块大小统一以后，加速回到了与物理核数相符的范围。我从中得到的教训是：比较两种实现时只能改变一个变量，看到超线性加速，首先要当成测量问题来查。

常见错答：只讲「我实现了什么」，却不讲「我怎么确认它是对的」。

TABLE XV
本文配套实验的十个 STAR 故事（数字为 CPU 测量或模型输出）

#	实验	20 秒结论	考察点
1	CS229S P1 FlashAttention	按论文块大小，HBM 流量只比朴素少 2.00 倍（因果 3.7–4.0 倍），在 T 上仍二次	IO 复杂度、读论文
2	CS229S P2 推测解码	每次目标前向的 token 数与 $(1-\alpha^{\gamma+1})/(1-\alpha)$ 差 4% 以内；CPU 上加速只有 0.98–1.99 倍	推理加速、口径
3	15-849 服务	连续预留 KV 浪费 85.6%，分页后 1.25%，并发从 104 升到 714；chunked prefill 把最差一步 121 ms 压到 9.2 ms	serving
4	15-849 分布式	ZeRO 0–3 与单进程差 2.9×10^{-16}；ZeRO-3 通信恰为 DP 的 1.5 倍	分布式训练
5	15-849 编译	MLP epilogue 融合省 49% HBM 流量，但只省 16% 时间	编译器、roofline
6	15-849 压缩	8/1024 个 50 倍离群通道让 per-tensor W8A8 误差从 2.3% 升到 10.2%，LLM.int8 拉回 0.49%	量化
7	CS149 A3 渲染	乱序画圆 93% 像素错误；分块 + scan 保序后误差为 0	并行正确性
8	CS149 A1 SIMD	每个 8 宽 gang 只要一条慢 lane，SIMD 利用率掉到 0.163	发散、负载均衡
9	CS149 A2 线程池	纯 spin 线程池在 CPython 中每次 launch 31–50 ms，sleep 版 38.7 µs	同步、GIL
10	CS149 A1 多进程	第一版测出 8 进程 21 倍「超线性」加速，根因是块大小改变了缓存行为	实验可信度

面试追问 55：你的实验都在 CPU 上，怎么说明它对 GPU 有意义？

答：我会把结论分成三类来说。第一类是正确性结论，比如 ZeRO 和单进程逐位一致、分页 attention 和连续实现逐元素相等，它们与硬件无关。第二类是计数结论，比如 HBM 字节数、KV 浪费率、通信量，它们由算法决定，也与硬件无关。第三类是时间结论，只有相对关系和拐点位置有意义，绝对值必须到 GPU 上重测。我会主动说明每个数属于哪一类。例如推测解码在 CPU 上只加速 0.98–1.99 倍，是因为 CPU 上验证 $\gamma+1$ 个 token 的代价是单 token 的 1.4–1.6 倍；而 GPU 上 batch 1 的 decode 受带宽限制，这个比值接近 1。

常见错答：把 CPU 上的墙钟加速直接当成 GPU 上的结论；或者反过来说「CPU 实验没有意义」。

E. 为面试读论文

面试官常会问「你读过哪篇论文，它做了什么」。为了能答得有条理，读系统论文时可以用五个问题做笔记，每个问题的答案不超过两句：

1) 瓶颈：论文声称解决的瓶颈是计算、访存、通信，还是调度和碎片？对应式 (1) 的哪一项？
2) 关键想法：用一句话说清它把什么换成了什么，例如 FlashAttention 用重算换 HBM 流量，vLLM 用间接寻址换连续性。
3) 前提：它在什么区间成立？batch、序列长度、硬件代际、拓扑变了以后还成立吗？
4) 证据：主结果的基线公平吗（同等调优、同一硬件）？加速来自哪一部分，有没有消融？
5) 下一个实验：如果你来做，会补哪个消融，或者扩展到哪个区间？

以 vLLM [3] 为例。它的瓶颈是 KV 的碎片，约束的是显存而不是算力；关键想法是把操作系统的分页搬到 KV 上；前提是请求长度的方差大，而且显存是并发的主要约束；证据是与 Orca 等系统的吞吐对比，2–4 倍的提升主要来自 batch 变大了；下一个实验可以是：在长度方差小的负载上（如固定长度的批处理），分页的收益会不会消失？

F. 研究练习：提出一个消融

研究岗的面试常以「如果你来改进这篇论文，你会做什么实验」收尾。好的回答包含三个要素：一个可以被证伪的假设，一个只改变一个变量的实验，一个能区分不同假设的观测。

面试追问 56：给 chunked prefill 设计一个消融实验。

答：假设：chunk 大小存在一个最优值。太小时，prefill 的 GEMM 吃不满，总吞吐下降；太大时，同批 decode 的步长变长，TPOT 的尾延迟上升。实验：固定负载（到达率和长度分布）和硬件，只扫描 chunk 大小（从 128 到 8192），记录吞吐、TTFT 和 TPOT 的 P99，以及 goodput。区分性观测：如果 goodput 曲线呈倒 U 形，而且峰值位置随到达率移动，就支持这个假设，并说明 chunk 大小应该自适应；如果曲线是单调的，说明在当前负载下只有一侧的约束在起作用。本文配套的离散事件模拟器（第 VIII 节）可以先在 CPU 上做这个扫描，再到 GPU 上验证峰值的位置。

常见错答：「把 chunk 调大调小看看效果」。这样既没有假设，也没说要看哪个指标。

面试追问 57：给你一块没见过的新加速器，第一周你怎么评估它？

答：先测三个上界，再测真实负载，最后看软件栈。三个上界是：峰值算力（用大方阵 GEMM 测）、内存带宽（用 stream 类的拷贝测）、互连带宽和延迟（用不同大小消息的 all-reduce 测），由此得到它的 roofline 和 α–β 参数。然后用代表性负载（一个 decode 步、一个训练步）测出实际达到上界的比例。最后看软件栈：编译器能不能自动融合，支不支持需要的精度和稀疏格式。报告里要写清硬件、软件版本、预热和重复次数，以及中位数和离散度。

常见错答：直接跑一个端到端模型看速度。这样分辨不出慢在硬件、编译器还是负载上。

G. 实验与汇报的工程规范

可信的数字有固定的产生方式。预热后多次测量，取中位数并报告离散度；写明硬件、软件版本和线程数；测量之前先用解析模型给出预期值，偏差超过 20% 时，先怀疑测量，再怀疑理论；比较两个实现时只改变一个变量；先保证正确，再谈性能，例如在 float64 下与参考实现比较

到 10^{-9}。本文所有配套实验的数字都是按这套规范产生的，原始 CSV 和生成脚本随仓库公开。

面试要点

- 回答模板：结论、估算、瓶颈、权衡、证据；第一句话就给判断。
- 错答大多源于忽略适用区间，要说清 batch、长度、拓扑、精度的边界。
- 项目故事要带上「不能推出什么」；CPU 实验中的计数结论和正确性结论与硬件无关。
- 读论文五问：瓶颈、关键想法、前提、证据、下一个实验。

研究视角

- 把本节的五步模板当作一个「可以打分的协议」：对同一道题，用模板回答和自由回答在信息密度（每分钟给出的可验证数字）上差多少？这可以作为自我训练的度量。
- 从表 XV 中任选一个实验，写出它在 GPU 上的复现计划：哪些数应该不变，哪些会变，预计变多少，为什么。

XII. 开放问题与研究方向

前面各节的「研究视角」框里散落着不少问题。本节把它们归并成六个方向，每个方向都说明它对应式 (1) 的哪一项、目前有哪些代表性工作，以及一个可以先在 CPU 上起步的切入点。

(1) 长上下文的访存墙。上下文从 8K 走向百万级以后，KV cache 成了 decode 中最主要的字节来源，Llama-3-8B 在 128K 时单条序列就要 16 GiB。现有的路线有结构压缩（GQA/MLA [93], [94]）、KV 量化、序列并行 [74], [75]，以及 attention-free 架构 [118]。开放的问题是：在精确检索能力（第 IX 节的 associative recall）和每 token 字节数之间，帕累托前沿在哪里？

(2) 推理调度的理论。continuous batching、chunked prefill 和 prefill/decode 分离 [4]–[6], [12] 都是在 TTFT、TPOT 和吞吐之间做取舍，但大多靠经验调参。把它们形式化成带 SLO 约束的排队问题，给出 goodput 的上界和可以证明的调度策略，这个方向的空白还很多。

(3) 自动并行与代价模型。Alpa [64] 和 GSPMD [53] 证明了自动搜索 3D 并行是可行的，但它们的代价模型对通信重叠、显存碎片和故障的刻画仍然很粗。第 VII 节的解析式可以作为一个可微或可搜索的代价模型的起点，关键问题是弄清它和实测之间的偏差从哪里来。

(4) 低精度的边界。FP8 训练 [87], [88] 和 4 位推理 [102], [103] 已经进入实用，更低的位宽（FP4、三值）则受限于离群值和累加精度。系统一侧的问题是：缩放因子的粒度（per-tensor、per-channel、per-block）该怎样在 kernel 效率和精度之间取舍？

(5) 规模化的可靠性。万卡训练的故障间隔以小时计 [2], [38]。异步 checkpoint、冗余副本、慢节点检测和静默数据损坏的发现，目前都是工程投入多、理论很少的领域。把 Young 公式推广到「故障率随规模变化，恢复时间随 checkpoint 位置变化」的情形，是一个干净的建模问题。

(6) 编译器与硬件的协同。Hopper 的 TMA 和 warp 特化 [11]、Trainium 的 NKI [54]，都要求程序员显式地管理异步数据搬运。编译器能不能自动生成 FlashAttention-3 这个水平的 kernel？Triton [44] 和 TVM/Ansor [61], [63] 给出了部分答案，但在异步流水线和跨 kernel 融合上仍然依赖人工。

面试要点

- 面试中谈研究方向时，先说它对应计算、访存、通信中的哪一项，再说代表性工作，最后说你会做的第一个实验。
- 好的开放问题都有清晰的度量，比如 goodput、每 token 字节数、MFU、有效训练时间。

研究视角

- 从上面任选一个方向，写一页研究提案：问题是什么，为什么现在重要，可证伪的假设，最小的实验，预期结果和风险。这是研究岗面试中最常见的「带回家」题型。

XIII. 结论

本文把 CS149、CS229S 和 CMU 15-849 三门课的内容放到同一个框架下：每个系统问题都拆成计算、访存、通信三项。沿着这条线，我们从 roofline 和 Amdahl 定律出发，经过 SIMD/SIMT 硬件、GEMM 与 FlashAttention 等 kernel、systolic 加速器、编译器与框架，一直讲到大模型的分布式训练、推理服务、模型效率和集群可靠性。

读完全文，最值得带走的是一个看问题的习惯：几乎每一项系统优化，都是「拿一种资源去换另一种资源」。FlashAttention 用重算换 HBM 流量，ZeRO 用通信换显存，激活重计算用 FLOPs 换显存，分页 KV 用间接寻址换连续性，推测解码用草稿模型的算力换目标模型的串行步数，MoE 用显存和通信换每 token 的 FLOPs。要判断一次交换划不划算，只需要估算交换前后的主导项。

为了面试，本文给出了可以当场推导的白板估算、按岗位组织的能力矩阵、五步回答模板和高频错答，并把配套实验整理成了可以复述的 STAR 故事。所有配套实验都是在 CPU 上完成的，它们支撑的是正确性、计数和相对关系方面的结论，不是 GPU 上的绝对性能；文中每一处都标明了这个区别。希望这份综述能帮助读者在面试和研究中，从「知道很多名词」走到「能算出哪个名词在当前场景下真正起作用」。

Appendix A
公式速查

考前复习时，最省时间的办法是把正文里反复用到的公式过一遍，并各配一个能当场算出的例子。表 XVI 汇总了正文中用到的 22 个公式，编号与配套仓库的公式卡一致。表中的记号如下：p 是处理器或 GPU 数，P 是峰值算力，BW 是带宽，b 是每个元素的字节数，N 是参数量，D 是训练 token 数，T 是序列长度，B 是 batch，L 是层数。正文的 GEMM 和 KV 公式里，每元素字节数记作 s，与这里的 b 是同一个量。

Appendix B
硬件与模型参数

白板估算的数字要前后一致，就得统一用一组硬件参数。表 XVII 就是正文所有估算使用的那一组。其中算力一律取稠密值，因为厂商标称的稀疏值是稠密值的 2 倍，直接拿来算 MFU 会把利用率低估一半；NVLink 取单向带宽。表 XVIII 给出常用模型的结构，以及由结构算出的参数量和每 token 的 KV（BF16、全部层）。

TABLE XVI
公式速查表（F1–F22）

#	名称	公式	白板用例
F1	Amdahl / Gustafson	$S=\frac{1}{(1-f)+f/p}$；$S_G=(1-f)+fp$	$f=0.95,p=8$：5.93；$f=0.99,p=1024$：91.2
F2	Roofline	$t=\max(F/P,\ B_{\text{bytes}}/BW)$，拐点 $I^*=P/BW$	H100 BF16：$989/3.35\approx295$
F3	GEMM 强度	$I=\frac{2MNK}{b(MK+KN+MN)}$，方阵 BF16 为 $n/3$	4096^3：1365；GEMV：≈1
F4	分块访存	$B_{\text{bytes}}\approx bMNK(\frac{1}{T_M}+\frac{1}{T_N})$，方块 $I\approx T/b$	$T=128$ 的 BF16 tile：$I\approx64$
F5	Work–span	$\max(W/p,S)\le T_p\le W/p+S$	并行度 W/S
F6	Scan	Blelloch：$W=O(n)$，$S=O(\log n)$	上扫 + 下扫各 $n-1$ 次加法
F7	Attention	FLOPs $\approx4T^2d$（每层，QK 与 PV）；Flash 的 HBM $\Theta(T^2d^2/M)$	显存从 $O(T^2)$ 降到 $O(T)$
F8	Online softmax	$m=\max(m_1,m_2)$，$\ell=\ell_1e^{m_1-m}+\ell_2e^{m_2-m}$	块间合并可结合
F9	参数与 FLOPs	$N\approx12Ld^2$（不含词表），$C\approx6ND$	8B×15T：7.2×10^{23}
F10	训练显存	DP：$16N$；ZeRO-1/2/3：$4N+\frac{12N}{p}$，$2N+\frac{14N}{p}$，$\frac{16N}{p}$	7B、$p=8$：112/38.5/26.25/14.0 GB
F11	Ring all-reduce	$t=2(p-1)\alpha+\frac{2(p-1)}{p}\cdot\frac{S}{BW}$	14 GB、8 卡、450 GB/s：54.4 ms
F12	TP 通信	每层前向 2 次 all-reduce，每次 $bBTd$ 字节，反向再 2 次	$BT=8192$、$d=4096$：每次 64 MiB
F13	流水线气泡	$\frac{p-1}{m+p-1}$；交错 $\frac{p-1}{vm+p-1}$	$p=8,m=32$：0.18；$v=2$：0.099
F14	$\sqrt{n}$ 重计算	存 $\sqrt{n}$ 个检查点，激活 $O(\sqrt{n})$，多一次前向	64 层：存 8 段
F15	KV cache	$2\,L\,n_{kv}\,d_h\,b\,T\,B$	Llama-3-8B：128 KiB/token
F16	Decode 步时	$t\approx\frac{2N+\text{KV}}{BW}$（小 batch），临界 $B^*\approx I^*$	8B、H100、$B=1$：4.95 ms
F17	推测解码	$E=\frac{1-\alpha^{\gamma+1}}{1-\alpha}$，$S=\frac{E}{\gamma c+1}$	$\alpha=0.8,\gamma=4$：3.36；$c=0.05$：2.80
F18	Little 定律	$L=\lambda W$	100 req/s×10 s：1000 并发
F19	线性量化	$q=\text{round}(x/s)+z$，$s=\frac{x_{\max}-x_{\min}}{2^k-1}$	离群值放大 s
F20	LoRA	可训练参数 $r(d+k)$	$d=k=4096,r=16$：131K
F21	MoE all-to-all	每卡 $n_{\text{tok}}\,k\,d\,b\,\frac{E_p-1}{E_p}$	8192 token、top-2、EP=8：117 MB
F22	MFU	$\frac{6N\cdot\text{tokens/s}}{p\cdot P}$（可加 attention 项）	与 HFU 区分：后者计入重计算

TABLE XVII
硬件参数（稠密算力、单向带宽）

型号	BF16 (TFLOP/s)	FP8 (TFLOP/s)	HBM (GB)	HBM 带宽 (TB/s)	拐点 (FLOP/B)	SM	互连（单向）
A100 SXM [14]	312	—	80	2.04	153	108	NVLink3 300 GB/s
H100 SXM [15]	989	1979	80	3.35	295	132	NVLink4 450 GB/s
H200 SXM	989	1979	141	4.8	206	132	NVLink4 450 GB/s
RTX 4090	≈165	—	24	1.01	164	128	PCIe4 x16 ≈32 GB/s
TPU v4 [52]	275	—	32	≈1.2	≈229	—	ICI + OCS，4096 芯片/pod

网络：PCIe 5.0 x16 ≈64 GB/s；InfiniBand HDR 25 GB/s/端口；NDR 50 GB/s/端口（节点内 NVLink 约快 9 倍）。
H100 存储层次：寄存器 256 KB/SM；共享内存最多 228 KB/SM；L2 50 MB；HBM 延迟约 500 周期。

TABLE XVIII
常用模型结构（参数量与 KV 由结构算出）

模型	层	d	头/KV 头	参数	KV/token
GPT-2 124M	12	768	12/12	0.124B	36 KiB
Qwen2.5-0.5B	24	896	14/2	0.49B	12 KiB
Llama-2-7B	32	4096	32/32	6.74B	512 KiB
Llama-3-8B	32	4096	32/8	8.03B	128 KiB
Llama-3-70B	80	8192	64/8	70.6B	320 KiB

Appendix C
两周学习计划

最后是一个以面试为目标的两周计划（表 XIX）。第一周打底，覆盖性能模型、硬件、kernel 和编译；第二周攻大模型，覆盖训练、推理、效率和集群，最后落到项目故事和模拟面试上。安排的顺序沿着图 1 的箭头自下而上。每天的产出有两样：一个能在白板上复述的估算，一个能在 90 秒内讲完的结论。

TABLE XIX
两周学习计划（每天约 4 小时）

天	主题（本文章节）	阅读与动手	当天必须能讲出的数
1	性能模型（§II）	Roofline [13]、Amdahl；用计算器算三种 GEMM 的强度	H100 拐点 295；4096^3 强度 1365
2	并行硬件（§III）	CS149 A1/A2：SIMD 发散、线程池；伪共享实验	慢 lane 使 SIMD 利用率降到 0.163
3	GPU 编程（§III）	CUDA 模型 [29]、coalescing、bank conflict、occupancy [33]	每 SM 228 KB 共享内存
4	GEMM 与 tiling（§IV）	CUTLASS 分块；推导 tile 大小	方块 tile 强度 T/b
5	FlashAttention（§IV）	FA1/2/3 [11], [17], [26]；online softmax 实现	HBM $\Theta(T^2d^2/M)$；实验中仅少 2 倍
6	加速器与编译（§V、§VI）	TPU [51]、TVM [61]、PyTorch 2 [45]；融合实验	融合省 49% 流量、16% 时间
7	复习与模拟	用第 XI 节模板答第 2–6 节的追问框	第一周所有估算
8	数据并行与 ZeRO（§VII）	ZeRO [70]、FSDP [71]；ring all-reduce 推导	$16N$；7B 需 112 GB
9	模型/流水线并行（§VII）	Megatron [73], [78]、GPipe [76]	气泡 0.18（$p=8, m=32$）
10	推理基础（§VIII）	KV cache、GQA [93]、decode roofline	8B 在 H100 上 $B=1$ 约 200 tok/s
11	推理服务（§VIII）	vLLM [3]、Sarathi [5]、DistServe [6]	KV 浪费 85.6% → 1.25%
12	推测解码与量化（§VIII）	[96], [100], [102], [103]	$E=3.36$（$\alpha=0.8, \gamma=4$）
13	效率与集群（§IX、§X）	LoRA、MoE、Chinchilla [120]；Llama 3 可靠性	$N^*=\sqrt{C/120}$；27 分钟 checkpoint
14	项目故事与模拟面试	准备两个 STAR 故事（表 XV）；完成一次系统设计	每个故事的「不能推出什么」

References

[1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] A. Dubey, A. Jauhri, A. Pandey, A. Kadian *et al.*, "The Llama 3 herd of models," arXiv:2407.21783, 2024.

[3] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[4] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for Transformer-based generative models," in *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022, pp. 521–538.

[5] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," in *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

[6] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

[7] K. Fatahalian and K. Olukotun, "CS149: Parallel computing," Stanford University, course materials, 2024, fall 2024. [Online]. Available: https://gfxcourses.stanford.edu/cs149/fall24

[8] A. Mirhoseini and S. Arora, "CS229S: Systems for machine learning," Stanford University, course materials, 2023, fall 2023. [Online]. Available: https://cs229s.stanford.edu/fall2023/

[9] Z. Jia, "15-849: Machine learning systems," Carnegie Mellon University, course materials, 2022, spring 2022. [Online]. Available: https://www.cs.cmu.edu/~zhihaoj2/15-849/

[10] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[11] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[12] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative LLM inference using phase splitting," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2024.

[13] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[14] NVIDIA, "NVIDIA A100 tensor core GPU architecture," Whitepaper, 2020.

[15] ——, "NVIDIA H100 tensor core GPU architecture," Whitepaper, 2022.

[16] A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer, "AI and memory wall," *IEEE Micro*, vol. 44, no. 3, pp. 33–39, 2024.

[17] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[18] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485.

[19] J. L. Gustafson, "Reevaluating Amdahl's law," *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.

[20] R. P. Brent, "The parallel evaluation of general arithmetic expressions," *Journal of the ACM*, vol. 21, no. 2, pp. 201–206, 1974.

[21] R. D. Blumofe and C. E. Leiserson, "Scheduling multithreaded computations by work stealing," *Journal of the ACM*, vol. 46, no. 5, pp. 720–748, 1999.

[22] G. E. Blelloch, "Prefix sums and their applications," Carnegie Mellon University, Tech. Rep. CMU-CS-90-190, 1990.

[23] W. D. Hillis and J. Guy L. Steele, "Data parallel algorithms," *Communications of the ACM*, vol. 29, no. 12, pp. 1170–1183, 1986.

[24] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.

[25] J.-W. Hong and H. T. Kung, "I/O complexity: The red-blue pebble game," in *Proc. ACM Symposium on Theory of Computing (STOC)*, 1981, pp. 326–333.

[26] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[27] J. L. Hennessy and D. A. Patterson, "A new golden age for computer architecture," *Communications of the ACM*, vol. 62, no. 2, pp. 48–60, 2019.

[28] E. Lindholm, J. Nickolls, S. Oberman, and J. Montrym, "NVIDIA Tesla: A unified graphics and computing architecture," *IEEE Micro*, vol. 28, no. 2, pp. 39–55, 2008.

[29] J. Nickolls, I. Buck, M. Garland, and K. Skadron, "Scalable parallel programming with CUDA," *ACM Queue*, vol. 6, no. 2, pp. 40–53, 2008.

[30] M. S. Papamarcos and J. H. Patel, "A low-overhead coherence solution for multiprocessors with private cache memories," in *Proc. International Symposium on Computer Architecture (ISCA)*, 1984, pp. 348–354.

[31] L. Lamport, "How to make a multiprocessor computer that correctly executes multiprocess programs," *IEEE Transactions on Computers*, vol. C-28, no. 9, pp. 690–691, 1979.

[32] S. V. Adve and K. Gharachorloo, "Shared memory consistency models: A tutorial," *IEEE Computer*, vol. 29, no. 12, pp. 66–76, 1996.

[33] V. Volkov, "Better performance at lower occupancy," GPU Technology Conference (GTC), 2010.

[34] NVIDIA, "CUTLASS: CUDA templates for linear algebra subroutines," https://github.com/NVIDIA/cutlass, 2017.

[35] ——, "NVIDIA Tesla V100 GPU architecture," Whitepaper, 2017.

[36] A. Mishra, J. A. Latorre, J. Pool, D. Stosic, D. Stosic, G. Venkatesh, C. Yu, and P. Micikevicius, "Accelerating sparse deep neural networks," arXiv:2104.08378, 2021.

[37] A. Chowdhery, S. Narang, J. Devlin *et al.*, "PaLM: Scaling language modeling with pathways," *Journal of Machine Learning Research*, vol. 24, no. 240, pp. 1–113, 2023.

[38] Z. Jiang, H. Lin, Y. Zhong, Q. Huang, Y. Chen *et al.*, "MegaScale: Scaling large language model training to more than 10,000 GPUs," in *Proc. USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2024.

[39] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," arXiv:1805.02867, 2018.

[40] M. N. Rabe and C. Staats, "Self-attention does not need $O(n^2)$ memory," arXiv:2112.05682, 2021.

[41] T. Dao, D. Haziza, F. Massa, and G. Sizov, "Flash-Decoding for long-context inference," Stanford CRFM blog, 2023.

[42] A. Ivanov, N. Dryden, T. Ben-Nun, S. Li, and T. Hoefler, "Data movement is all you need: A case study on optimizing transformers," in *Proc. Machine Learning and Systems (MLSys)*, 2021.

[43] Z. Jia, O. Padon, J. Thomas, T. Warszawski, M. Zaharia, and A. Aiken, "TASO: Optimizing deep learning computation with automatic generation of graph substitutions," in *Proc. ACM Symposium on Operating Systems Principles (SOSP)*, 2019, pp. 47–62.

[44] P. Tillet, H. T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proc. ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019, pp. 10–19.

[45] J. Ansel, E. Yang, H. He *et al.*, "PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation," in *Proc. ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.

[46] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *Proc. IEEE International Solid-State Circuits Conference (ISSCC)*, 2014, pp. 10–14.

[47] W. J. Dally, Y. Turakhia, and S. Han, "Domain-specific hardware accelerators," *Communications of the ACM*, vol. 63, no. 7, pp. 48–57, 2020.

[48] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.

[49] H. T. Kung, "Why systolic architectures?" *IEEE Computer*, vol. 15, no. 1, pp. 37–46, 1982.

[50] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2016, pp. 367–379.

[51] N. P. Jouppi, C. Young, N. Patil, D. Patterson *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2017, pp. 1–12.

[52] N. P. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles, C. Young, X. Zhou, Z. Zhou, and D. Patterson, "TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2023.

[53] Y. Xu, H. Lee, D. Chen, B. Hechtman, Y. Huang, R. Joshi, M. Krikun, D. Lepikhin, A. Ly, M. Maggioni, R. Pang, N. Shazeer, S. Wang, T. Wang, Y. Wu, and Z. Chen, "GSPMD: General and scalable parallelization for ML computation graphs," arXiv:2105.04663, 2021.

[54] Amazon Web Services, "AWS Neuron documentation: Trainium architecture and the neuron kernel interface (NKI)," https://awsdocs-neuron.readthedocs-hosted.com/, 2024.

[55] J. Dean, G. S. Corrado, R. Monga, K. Chen, M. Devin, Q. V. Le *et al.*, "Large scale distributed deep networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.

[56] M. Abadi, P. Barham, J. Chen *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265–283.

[57] A. Paszke, S. Gross, F. Massa *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[58] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: Composable transformations of Python+NumPy programs," https://github.com/jax-ml/jax, 2018.

[59] OpenXLA, "XLA: An optimizing compiler for machine learning," https://openxla.org/xla, 2017.

[60] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: A survey," *Journal of Machine Learning Research*, vol. 18, no. 153, pp. 1–43, 2018.

[61] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018, pp. 578–594.

[62] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," in *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2013, pp. 519–530.

[63] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen, J. E. Gonzalez, and I. Stoica, "Ansor: Generating high-performance tensor programs for deep learning," in *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020, pp. 863–879.

[64] L. Zheng, Z. Li, H. Zhang, Y. Zhuang, Z. Chen, Y. Huang, Y. Wang, Y. Xu, D. Zhuo, E. P. Xing, J. E. Gonzalez, and I. Stoica, "Alpa: Automating inter- and intra-operator parallelism for distributed deep learning," in *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022, pp. 559–578.

[65] M. Li, D. G. Andersen, J. W. Park, A. J. Smola, A. Ahmed, V. Josifovski, J. Long, E. J. Shekita, and B.-Y. Su, "Scaling distributed machine learning with the parameter server," in *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2014, pp. 583–598.

[66] A. Sergeev and M. D. Balso, "Horovod: Fast and easy distributed deep learning in TensorFlow," arXiv:1802.05799, 2018.

[67] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, and S. Chintala, "PyTorch distributed: Experiences on accelerating data parallel training," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3005–3018, 2020.

[68] R. Thakur, R. Rabenseifner, and W. Gropp, "Optimization of collective communication operations in MPICH," *International Journal of High Performance Computing Applications*, vol. 19, no. 1, pp. 49–66, 2005.

[69] P. Patarasuk and X. Yuan, "Bandwidth optimal all-reduce algorithms for clusters of workstations," *Journal of Parallel and Distributed Computing*, vol. 69, no. 2, pp. 117–124, 2009.

[70] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "ZeRO: Memory optimizations toward training trillion parameter models," in *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2020.

[71] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer, A. Desmaison, C. Balioglu, P. Damania, B. Nguyen, G. Chauhan, Y. Hao, A. Mathews, and S. Li, "PyTorch FSDP: Experiences on scaling fully sharded data parallel," *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 3848–3860, 2023.

[72] V. A. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoeybi, and B. Catanzaro, "Reducing activation recomputation in large transformer models," in *Proc. Machine Learning and Systems (MLSys)*, 2023.

[73] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training multi-billion parameter language models using model parallelism," arXiv:1909.08053, 2019.

[74] H. Liu, M. Zaharia, and P. Abbeel, "Ring attention with blockwise transformers for near-infinite context," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[75] S. A. Jacobs, M. Tanaka, C. Zhang, M. Zhang, S. L. Song, S. Rajbhandari, and Y. He, "DeepSpeed Ulysses: System optimizations for enabling training of extreme long sequence transformer models," arXiv:2309.14509, 2023.

[76] Y. Huang, Y. Cheng, A. Bapna, O. Firat, M. X. Chen, D. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, "GPipe: Efficient training of giant neural networks using pipeline parallelism," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[77] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Ganger, P. B. Gibbons, and M. Zaharia, "PipeDream: Generalized pipeline parallelism for DNN training," in *Proc. ACM Symposium on Operating Systems Principles (SOSP)*, 2019, pp. 1–15.

[78] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia, "Efficient large-scale language model training on GPU clusters using Megatron-LM," in *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.

[79] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, "GShard: Scaling giant models with con-

ditional computation and automatic sharding," in *Proc. International Conference on Learning Representations (ICLR)*, 2021.

[80] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[81] T. Chen, B. Xu, C. Zhang, and C. Guestrin, "Training deep nets with sublinear memory cost," arXiv:1604.06174, 2016.

[82] P. Jain, A. Jain, A. Nrusimha, A. Gholami, P. Abbeel, K. Keutzer, I. Stoica, and J. E. Gonzalez, "Checkmate: Breaking the memory wall with optimal tensor rematerialization," in *Proc. Machine Learning and Systems (MLSys)*, 2020.

[83] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," in *Proc. International Conference on Learning Representations (ICLR)*, 2018.

[84] D. Kalamkar, D. Mudigere, N. Mellempudi *et al.*, "A study of BFLOAT16 for deep learning training," arXiv:1905.12322, 2019.

[85] J. Ren, S. Rajbhandari, R. Y. Aminabadi, O. Ruwase, S. Yang, M. Zhang, D. Li, and Y. He, "ZeRO-Offload: Democratizing billion-scale model training," in *Proc. USENIX Annual Technical Conference (ATC)*, 2021, pp. 551–564.

[86] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning," in *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.

[87] P. Micikevicius, D. Stosic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu, N. Mellempudi, S. Oberman, M. Shoeybi, M. Siu, and H. Wu, "FP8 formats for deep learning," arXiv:2209.05433, 2022.

[88] DeepSeek-AI, "DeepSeek-V3 technical report," arXiv:2412.19437, 2024.

[89] P. Qi, X. Wan, G. Huang, and M. Lin, "Zero bubble (almost) pipeline parallelism," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[90] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," in *Proc. Machine Learning and Systems (MLSys)*, 2023.

[91] H. Touvron, L. Martin, K. Stone *et al.*, "Llama 2: Open foundation and fine-tuned chat models," arXiv:2307.09288, 2023.

[92] N. Shazeer, "Fast transformer decoding: One write-head is all you need," arXiv:1911.02150, 2019.

[93] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[94] DeepSeek-AI, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," arXiv:2405.04434, 2024.

[95] A. Agrawal, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, and R. Ramjee, "SARATHI: Efficient LLM inference by piggybacking decodes with chunked prefills," arXiv:2308.16369, 2023.

[96] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. International Conference on Machine Learning (ICML)*, 2023.

[97] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," arXiv:2302.01318, 2023.

[98] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," in *Proc. International Conference on Machine Learning (ICML)*, 2024.

[99] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE: Speculative sampling requires rethinking feature uncertainty," in *Proc. International Conference on Machine Learning (ICML)*, 2024.

[100] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[101] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *Proc. International Conference on Machine Learning (ICML)*, 2023.

[102] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in *Proc. International Conference on Learning Representations (ICLR)*, 2023.

[103] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proc. Machine Learning and Systems (MLSys)*, 2024.

[104] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[105] Y. Sheng, S. Cao, D. Li, C. Hooper, N. Lee, S. Yang, C. Chou, B. Zhu, L. Zheng, K. Keutzer, J. E. Gonzalez, and I. Stoica, "S-LoRA: Serving thousands of concurrent LoRA adapters," in *Proc. Machine Learning and Systems (MLSys)*, 2024.

[106] L. Chen, Z. Ye, Y. Wu, D. Zhuo, L. Ceze, and A. Krishnamurthy, "Punica: Multi-tenant LoRA serving," in *Proc. Machine Learning and Systems (MLSys)*, 2024.

[107] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2022.

[108] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *Proc. International Conference on Learning Representations (ICLR)*, 2016.

[109] E. Frantar and D. Alistarh, "SparseGPT: Massive language models can be accurately pruned in one-shot," in *Proc. International Conference on Machine Learning (ICML)*, 2023.

[110] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv:1503.02531, 2015.

[111] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. International Conference on Learning Representations (ICLR)*, 2017.

[112] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He, "DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale," in *Proc. International Conference on Machine Learning (ICML)*, 2022.

[113] T. Gale, D. Narayanan, C. Young, and M. Zaharia, "MegaBlocks: Efficient sparse training with mixture-of-experts," in *Proc. Machine Learning and Systems (MLSys)*, 2023.

[114] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[115] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, "Transformers are RNNs: Fast autoregressive transformers with linear attention," in *Proc. International Conference on Machine Learning (ICML)*, 2020.

[116] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," in *Proc. International Conference on Learning Representations (ICLR)*, 2022.

[117] M. Poli, S. Massaroli, E. Nguyen, D. Y. Fu, T. Dao, S. Baccus, Y. Bengio, S. Ermon, and C. Ré, "Hyena hierarchy: Towards larger convolutional language models," in *Proc. International Conference on Machine Learning (ICML)*, 2023.

[118] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," arXiv:2312.00752, 2023.

[119] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," arXiv:2001.08361, 2020.

[120] J. Hoffmann, S. Borgeaud, A. Mensch *et al.*, "Training compute-optimal large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[121] M. Jeon, S. Venkataraman, A. Phanishayee, J. Qian, W. Xiao, and F. Yang, "Analysis of large-scale multi-tenant GPU clusters for DNN training workloads," in *Proc. USENIX Annual Technical Conference (ATC)*, 2019, pp. 947–960.

[122] W. Xiao, R. Bhardwaj, R. Ramjee, M. Sivathanu, N. Kwatra, Z. Han, P. Patel, X. Peng, H. Zhao, Q. Zhang, F. Yang, and L. Zhou, "Gandiva: Introspective cluster scheduling for deep learning," in *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018, pp. 595–610.

[123] J. Gu, M. Chowdhury, K. G. Shin, Y. Zhu, M. Jeon, J. Qian, H. Liu, and C. Guo, “Tiresias: A GPU cluster manager for distributed deep learning,” in *Proc. USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2019, pp. 485–500.

[124] A. Qiao, S. K. Choe, S. J. Subramanya, W. Neiswanger, Q. Ho, H. Zhang, G. R. Ganger, and E. P. Xing, “Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning,” in *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021.

[125] Gemini Team, “Gemini: A family of highly capable multimodal models,” arXiv:2312.11805, 2023.

[126] J. W. Young, “A first order approximation to the optimum checkpoint interval,” *Communications of the ACM*, vol. 17, no. 9, pp. 530–531, 1974.

[127] J. Mohan, A. Phanishayee, and V. Chidambaram, “CheckFreq: Frequent, fine-grained DNN checkpointing,” in *Proc. USENIX Conference on File and Storage Technologies (FAST)*, 2021, pp. 203–216.

[128] J. Mohan, A. Phanishayee, A. Raniwala, and V. Chidambaram, “Analyzing and mitigating data stalls in DNN training,” *Proceedings of the VLDB Endowment*, vol. 14, no. 5, pp. 771–784, 2021.

[129] D. G. Murray, J. Šimša, A. Klimovic, and I. Indyk, “tf.data: A machine learning data processing framework,” *Proceedings of the VLDB Endowment*, vol. 14, no. 12, pp. 2945–2958, 2021.

[130] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, “Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing,” in *Proc. USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2012, pp. 15–28.

[131] A. Mirhoseini, H. Pham, Q. V. Le, B. Steiner, R. Larsen, Y. Zhou, N. Kumar, M. Norouzi, S. Bengio, and J. Dean, “Device placement optimization with reinforcement learning,” in *Proc. International Conference on Machine Learning (ICML)*, 2017.